



(19) **United States**

(12) **Patent Application Publication**

Kuo et al.

(10) **Pub. No.: US 2025/0287006 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **CHROMA CODING ENHANCEMENT IN CROSS-COMPONENT SAMPLE ADAPTIVE OFFSET WITH VIRTUAL BOUNDARY**

(71) Applicant: **BEIJING DAJIA INTERNET INFORMATION TECHNOLOGY CO., LTD.**, Beijing (CN)

(72) Inventors: **Che-Wei Kuo**, Beijing (CN); **Xiaoyu Xiu**, Beijing (CN); **Yi-Wen Chen**, Beijing (CN); **Wei Chen**, Beijing (CN); **Tsung-Chuan Ma**, Beijing (CN); **Hong-Jheng Jhu**, Beijing (CN); **Bing Yu**, Beijing (CN); **Xianglin Wang**, Beijing (CN)

(73) Assignee: **BEIJING DAJIA INTERNET INFORMATION TECHNOLOGY CO., LTD.**, Beijing (CN)

(21) Appl. No.: **19/219,778**

(22) Filed: **May 27, 2025**

Related U.S. Application Data

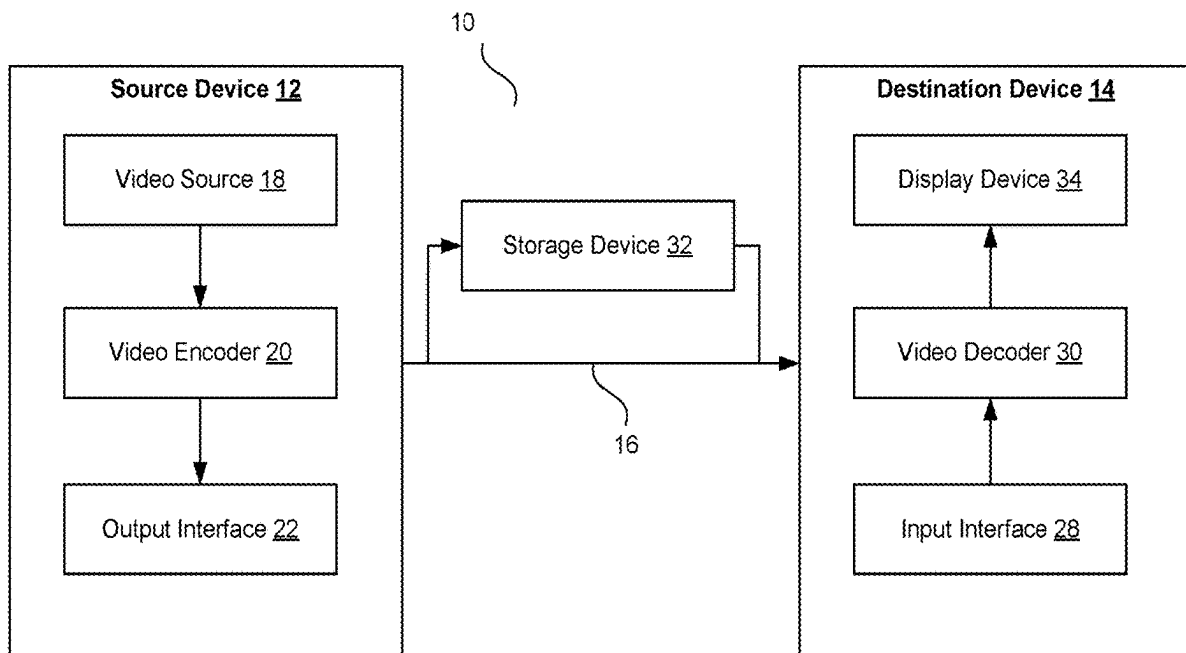
- (63) Continuation of application No. 18/125,568, filed on Mar. 23, 2023, which is a continuation of application No. PCT/US2021/051573, filed on Sep. 22, 2021.
- (60) Provisional application No. 63/082,356, filed on Sep. 23, 2020, provisional application No. 63/086,504, filed on Oct. 1, 2020.

Publication Classification

- (51) **Int. Cl.**
H04N 19/132 (2014.01)
H04N 19/105 (2014.01)
H04N 19/169 (2014.01)
H04N 19/176 (2014.01)
- (52) **U.S. Cl.**
CPC *H04N 19/132* (2014.11); *H04N 19/105* (2014.11); *H04N 19/176* (2014.11); *H04N 19/1883* (2014.11)

(57) **ABSTRACT**

An electronic apparatus performs a method of encoding video data. The method includes: in response to a current Coding Tree Unit (CTU) being not located within a bottom row of CTUs of a frame, slice, tile, subpicture or patch, determining a set of one or more samples of a first component of a picture frame according to a positional relationship between a respective sample of a second component of the picture frame and a virtual boundary; determining a class index for the second component from the set of one or more samples of the first component associated with the respective sample of the second component; selecting a sample offset from a plurality of sample offsets for the respective sample of the second component according to the class index; and obtaining a cross-component offsetted sample value of the second component based on the selected sample offset.



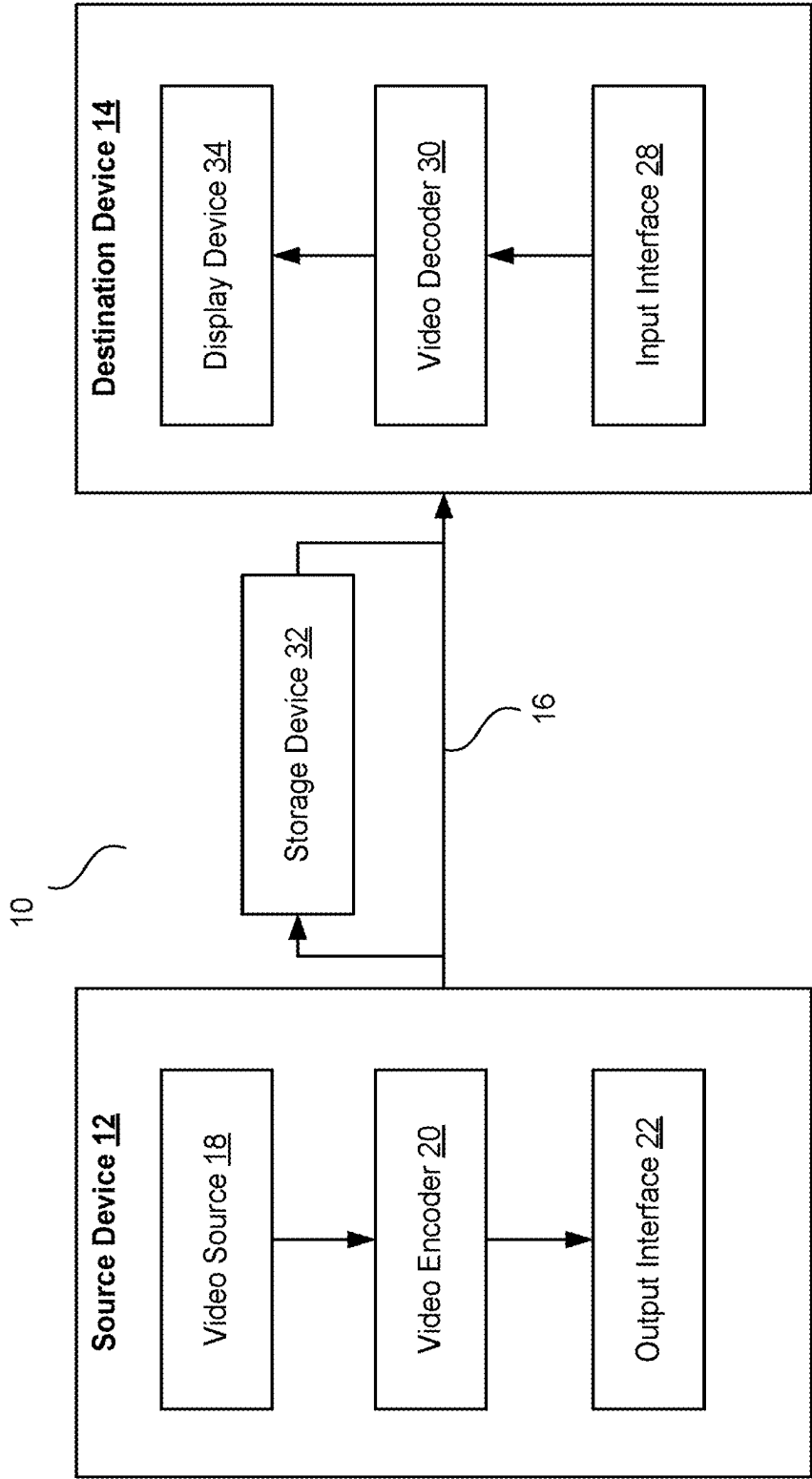


FIG. 1

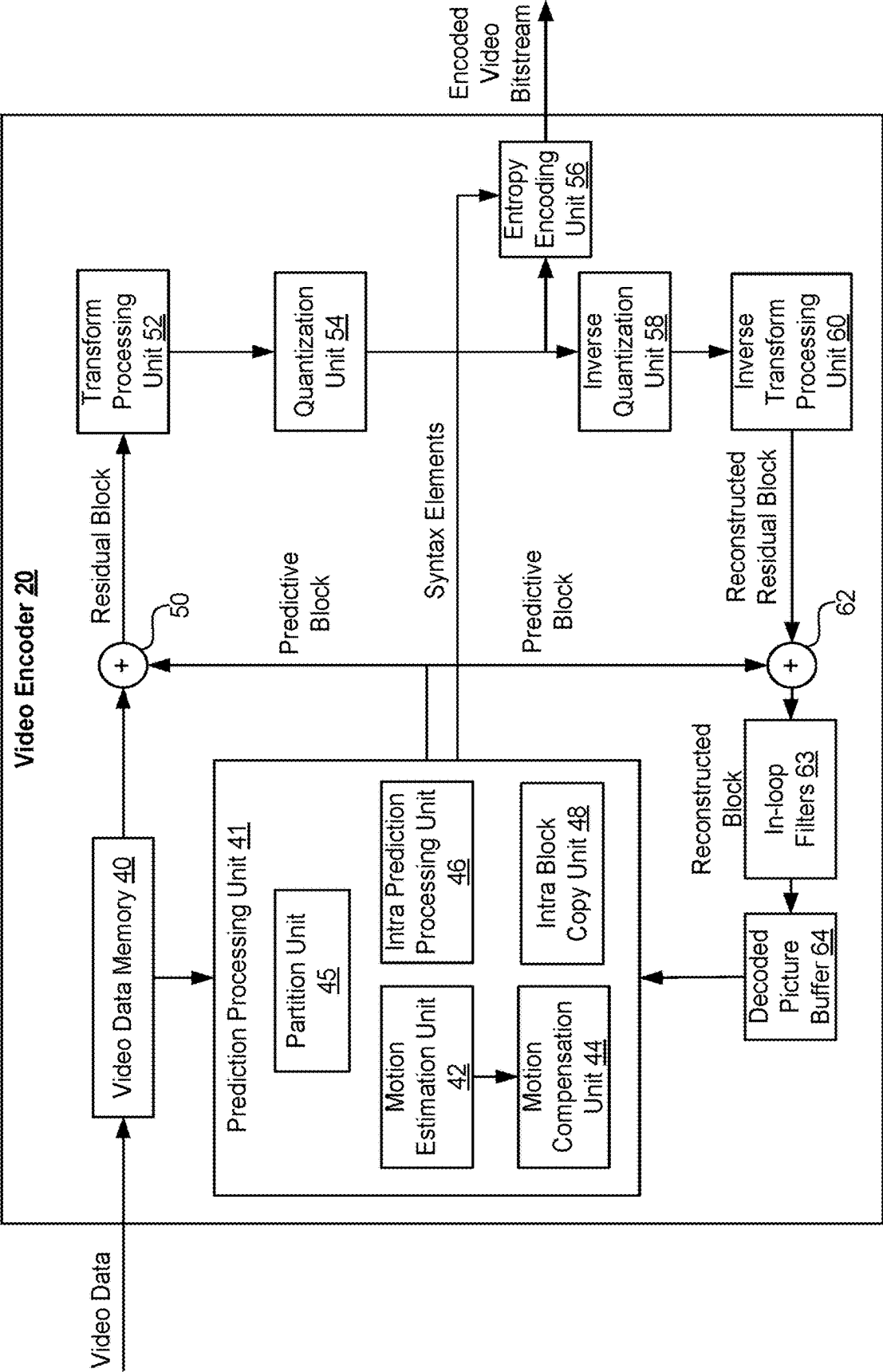


FIG. 2

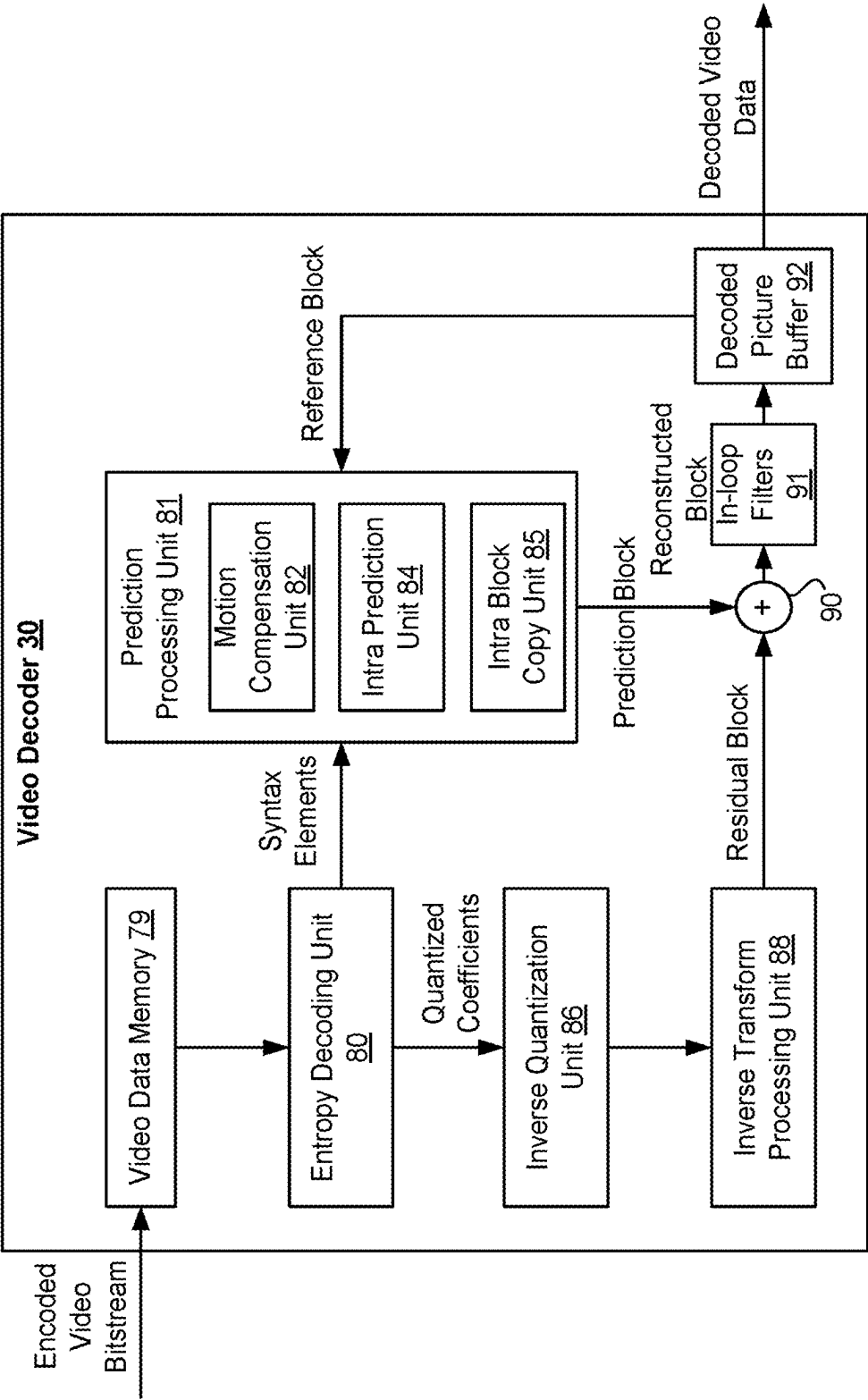


FIG. 3

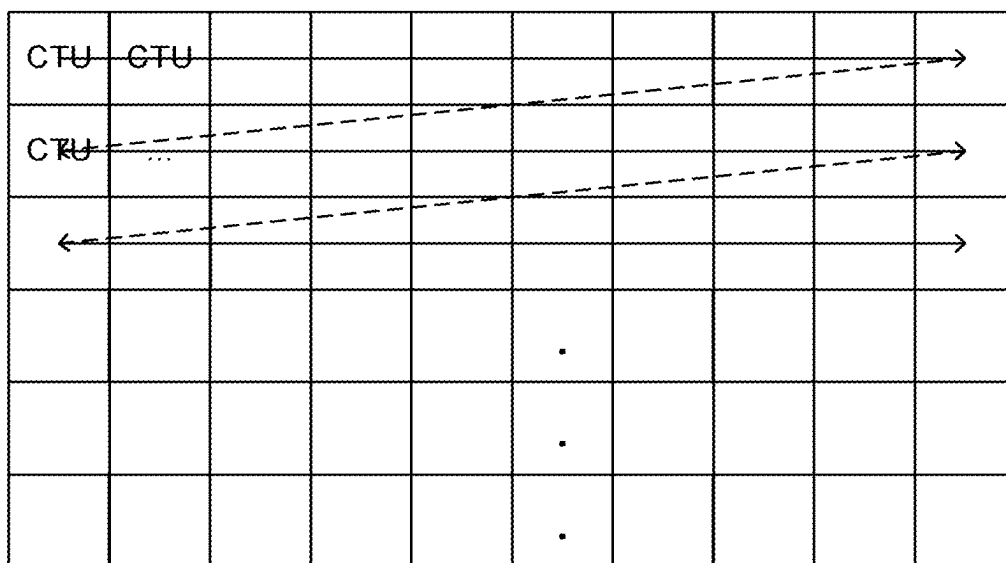


FIG. 4A

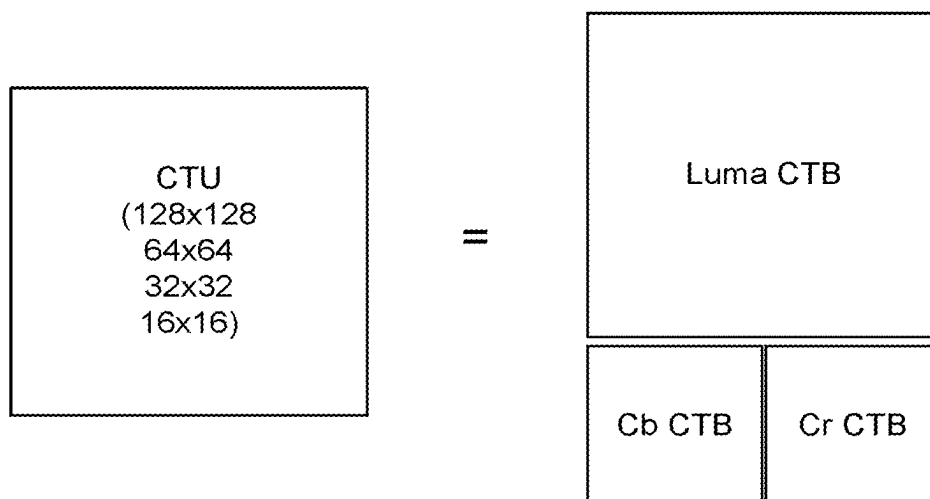


FIG. 4B

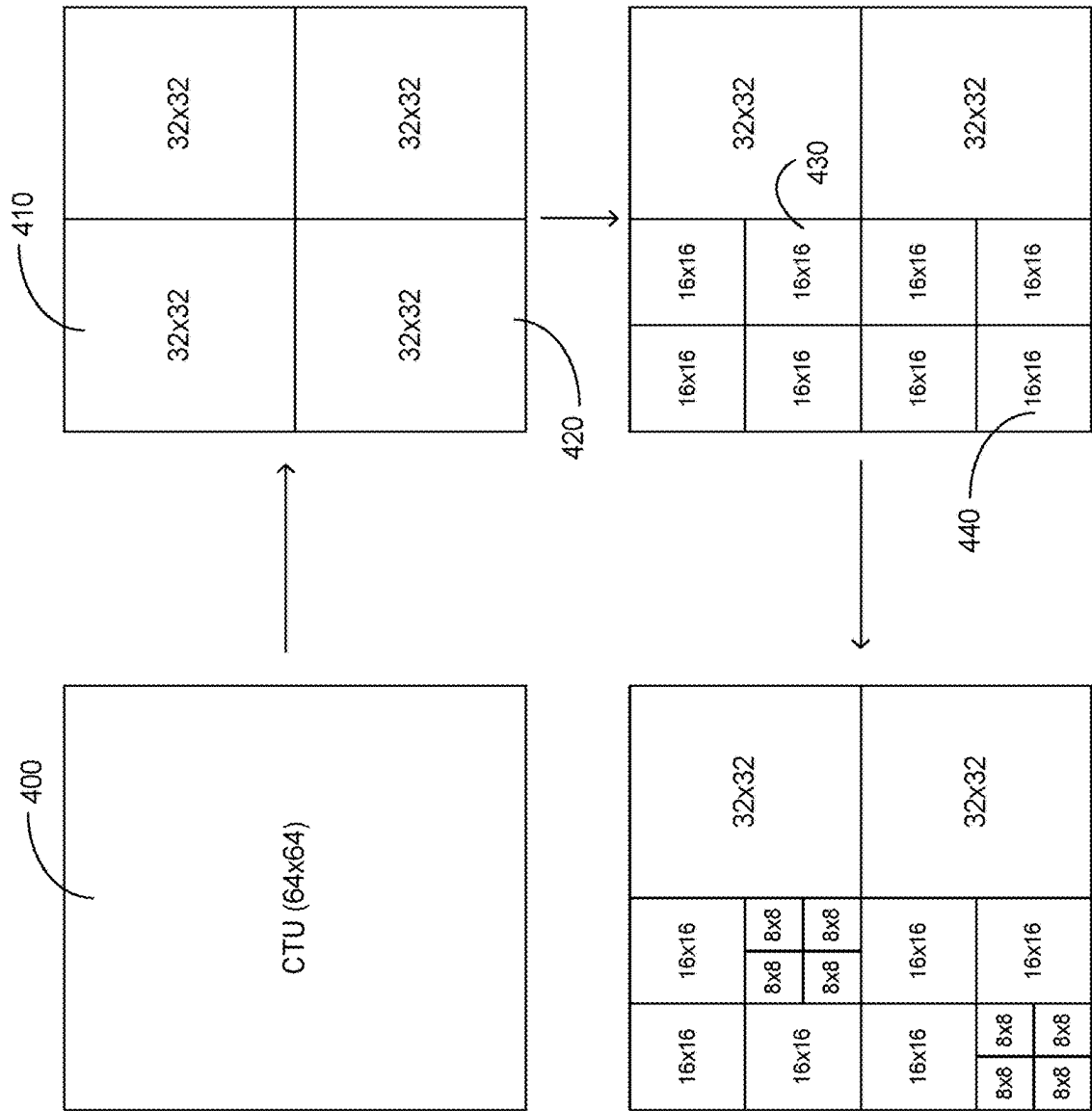


FIG. 4C

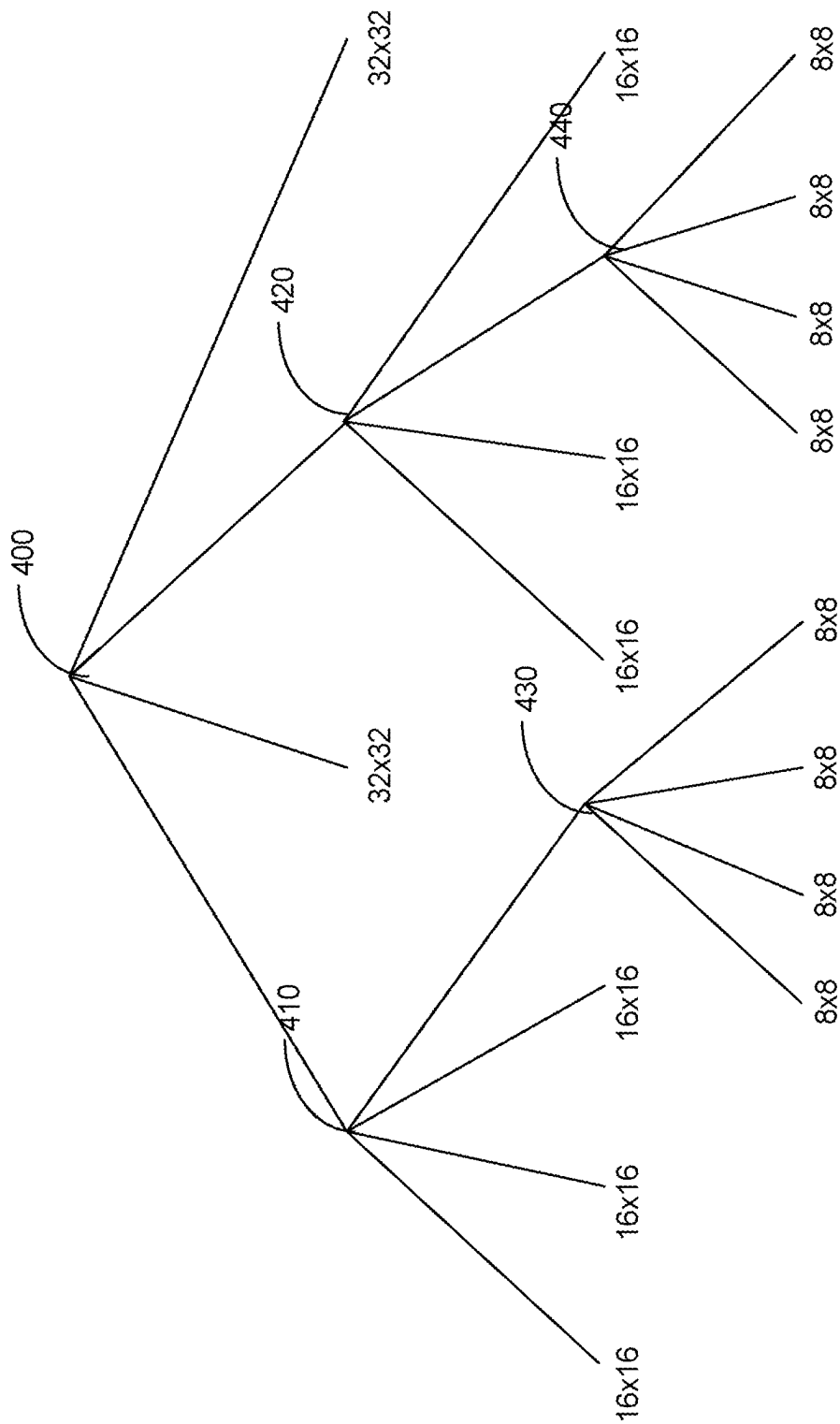


FIG. 4D

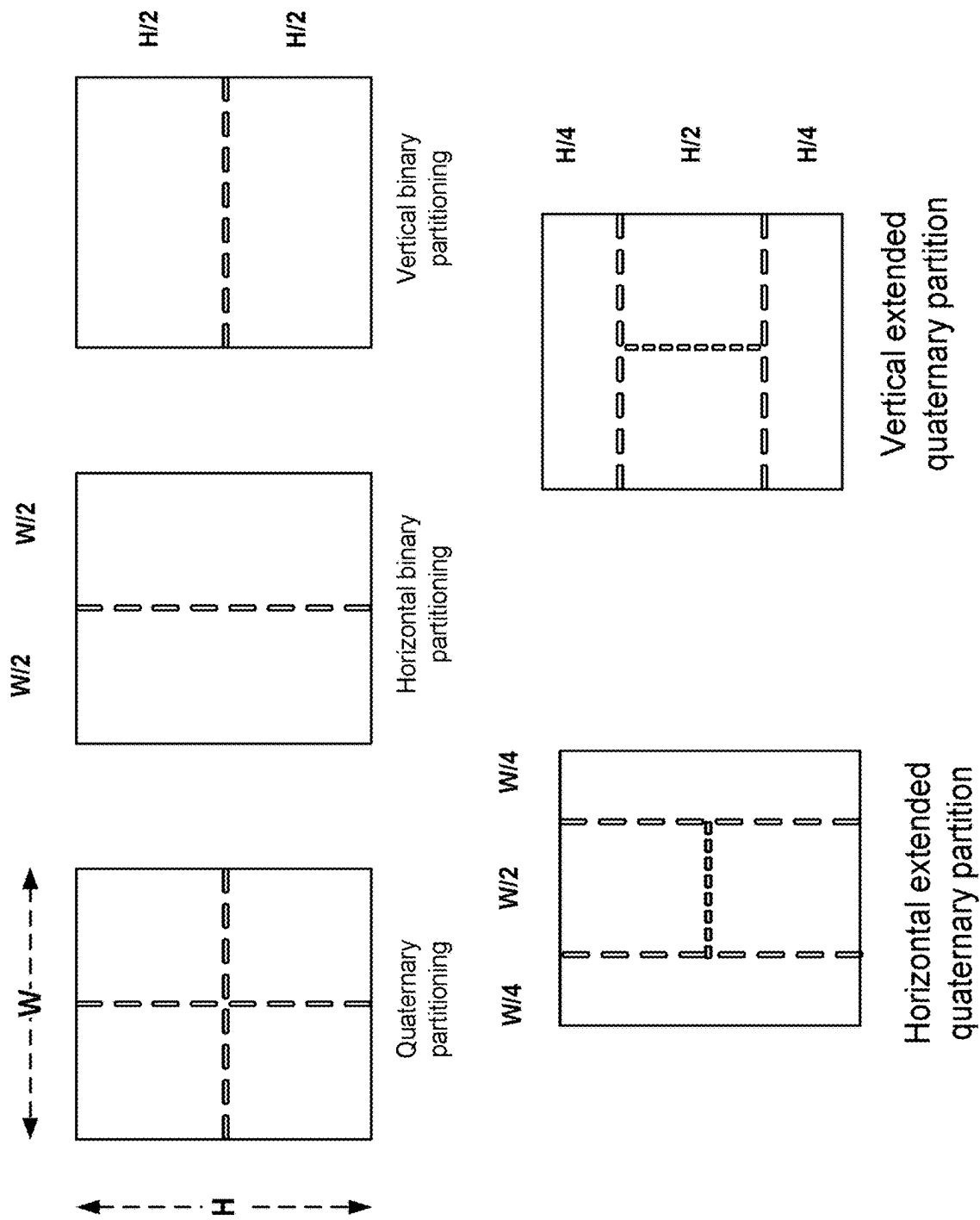


FIG. 4E

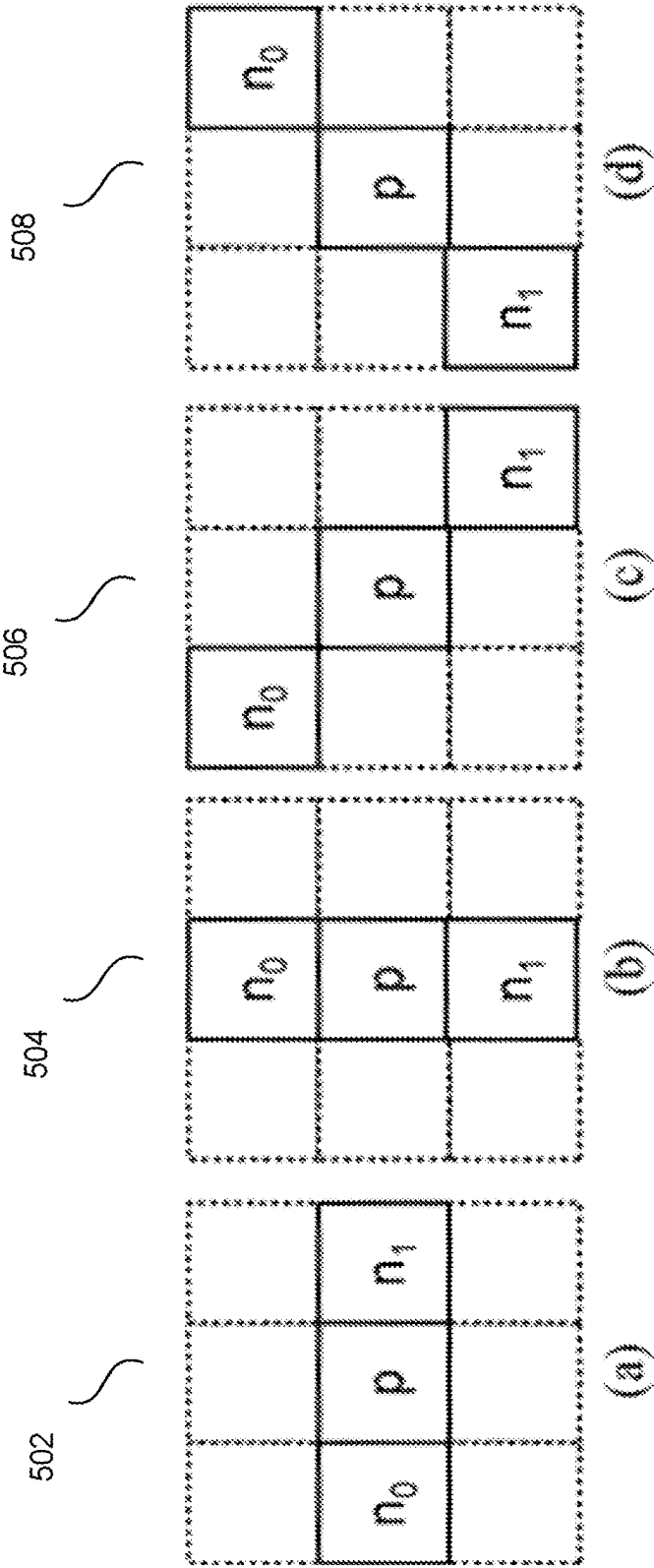


FIG. 5

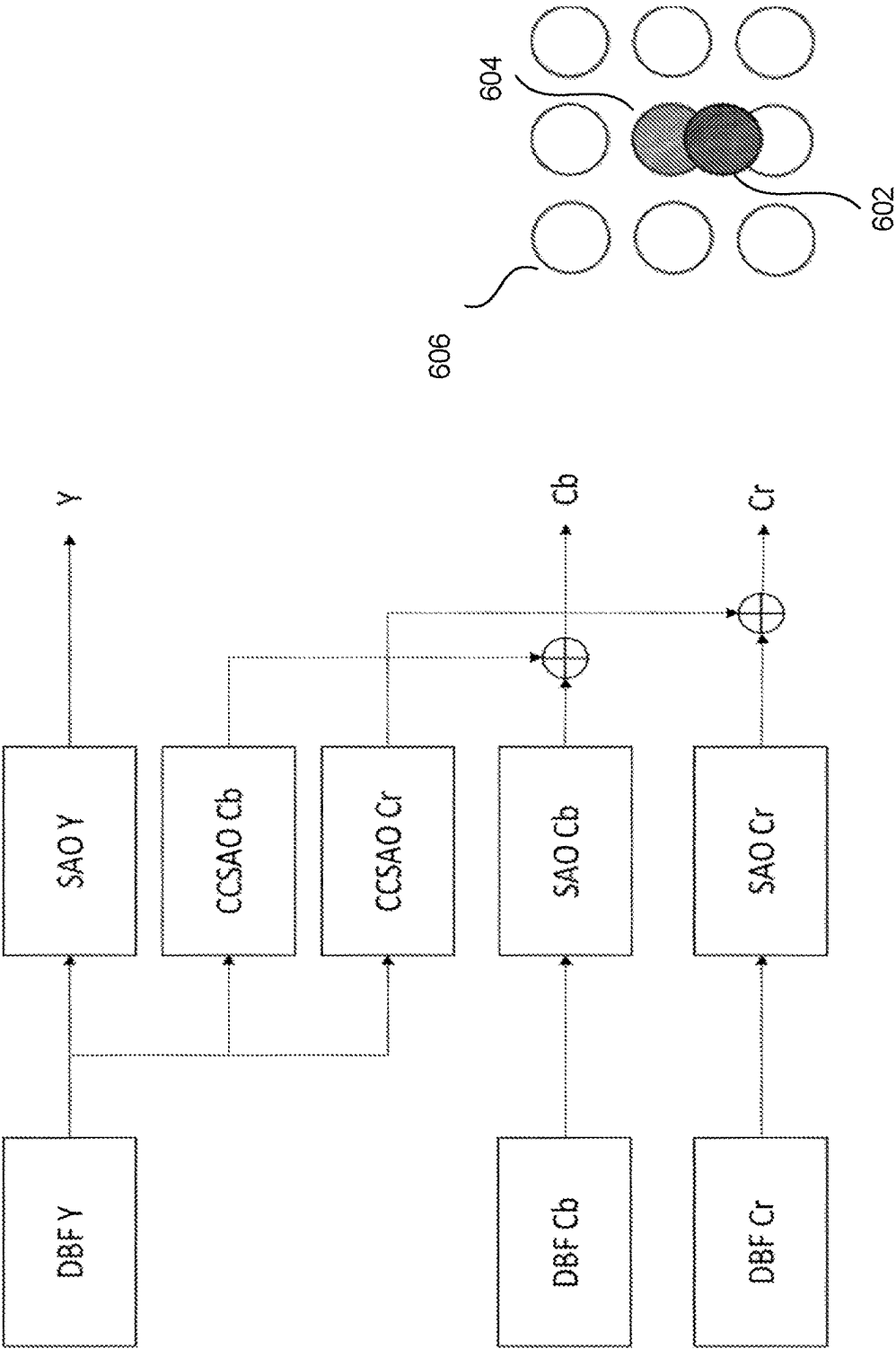


FIG. 6A

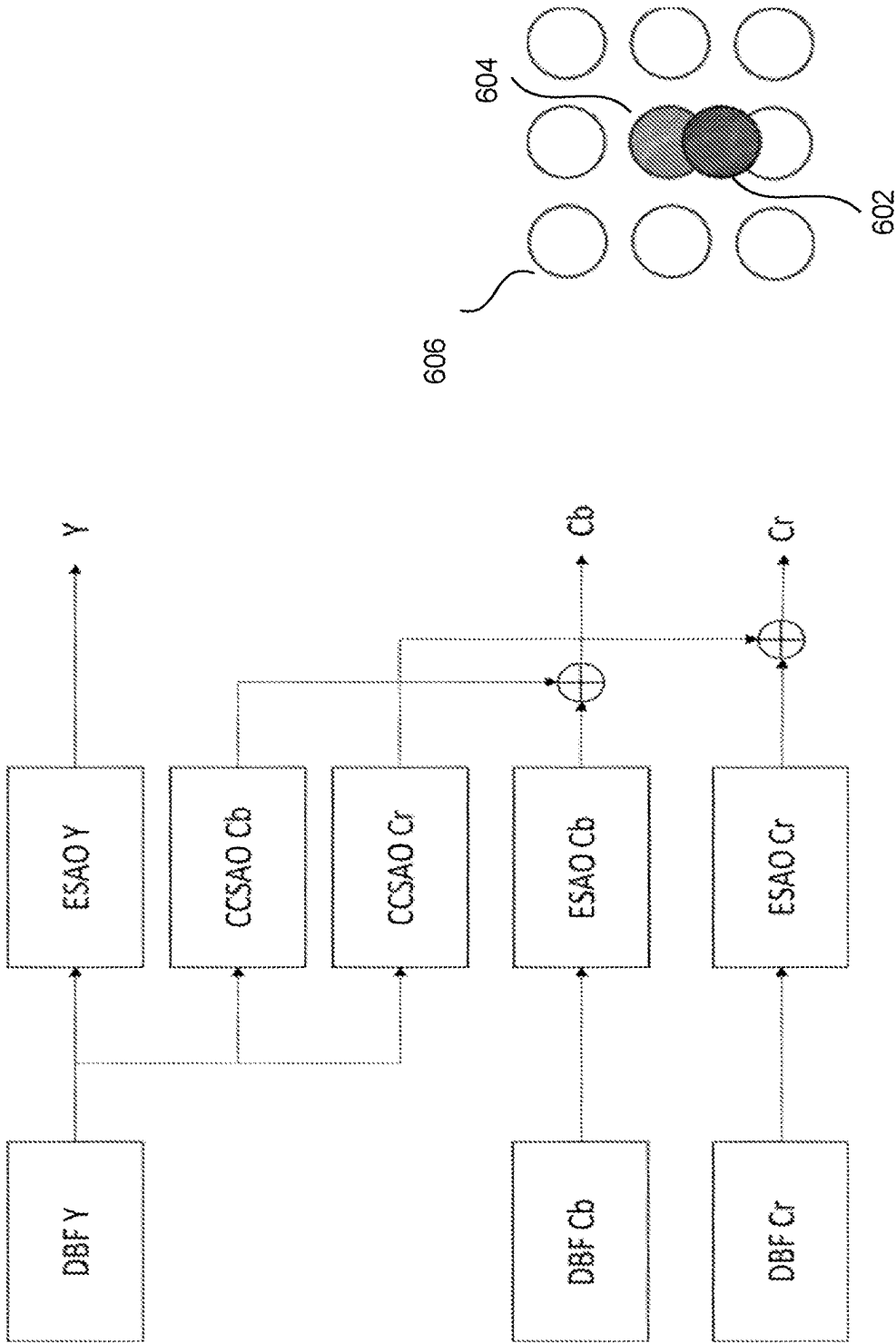


FIG. 6B

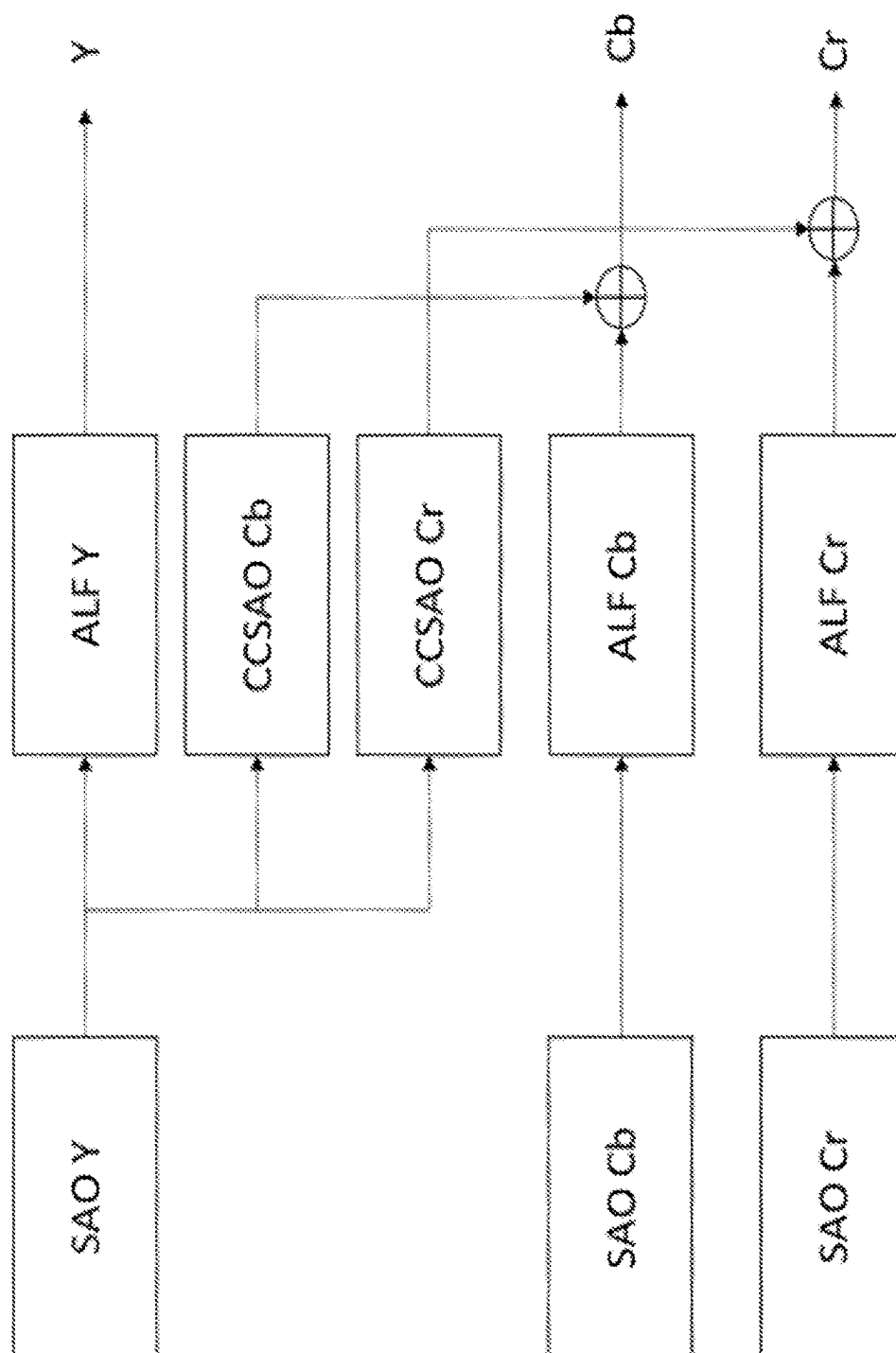


FIG. 6C

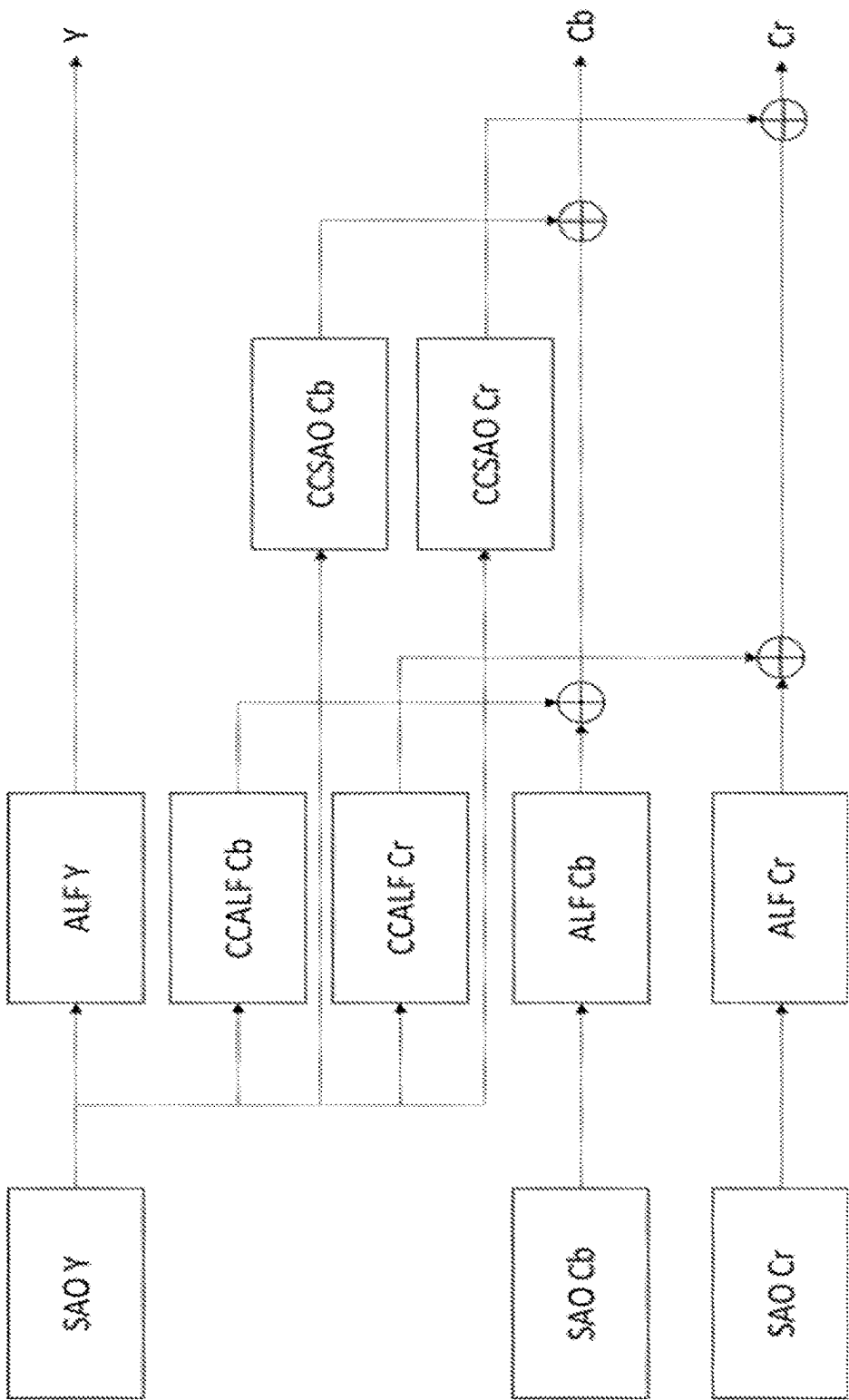


FIG. 6D

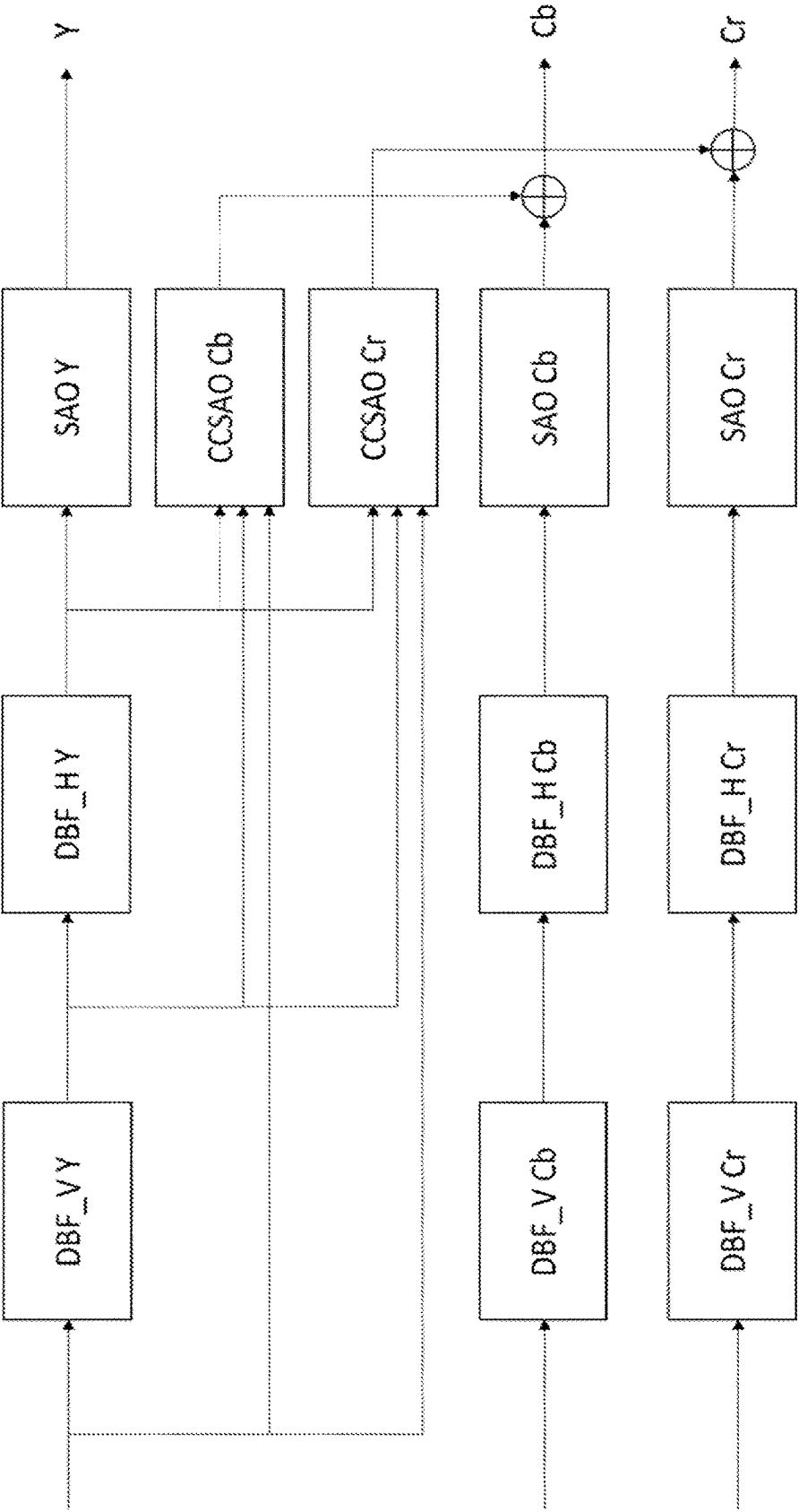


FIG. 7

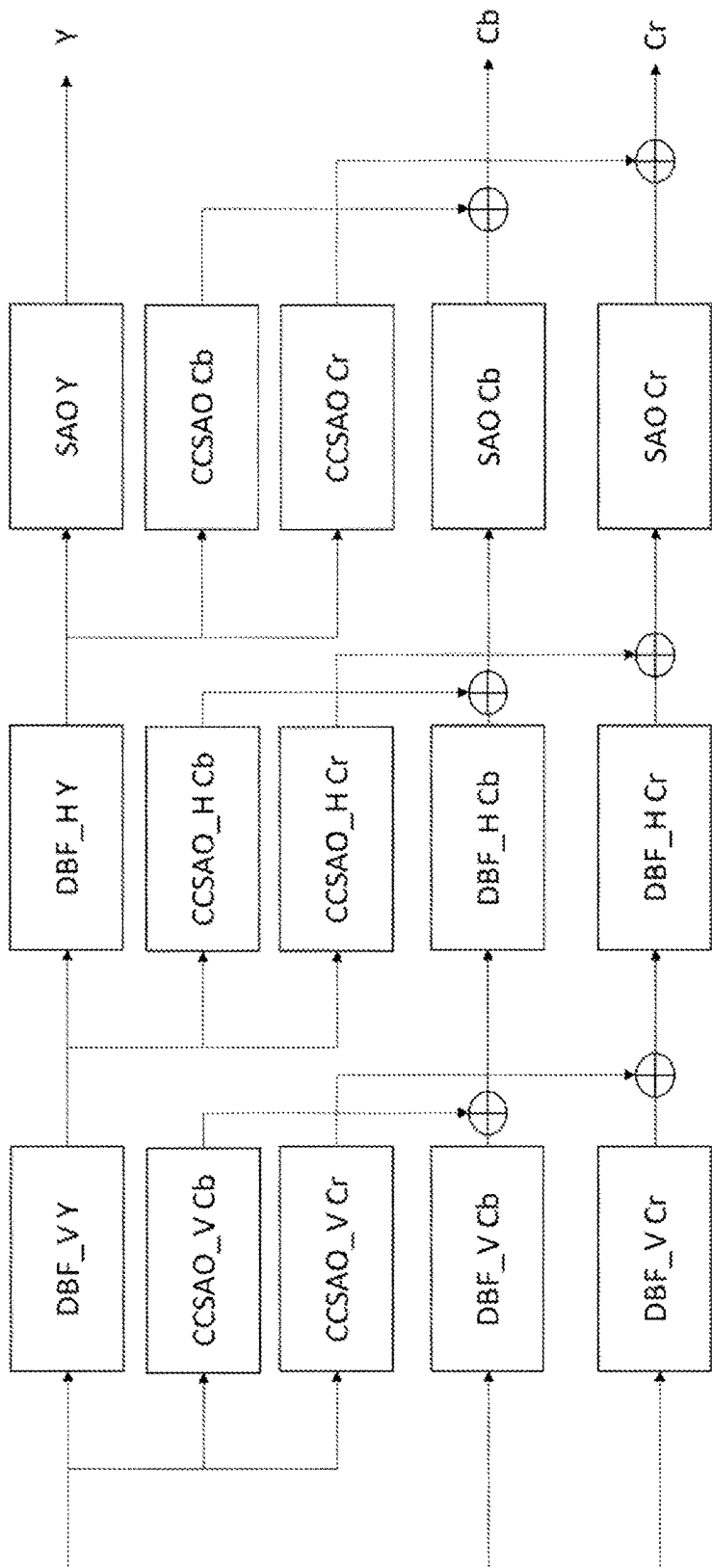


FIG. 8

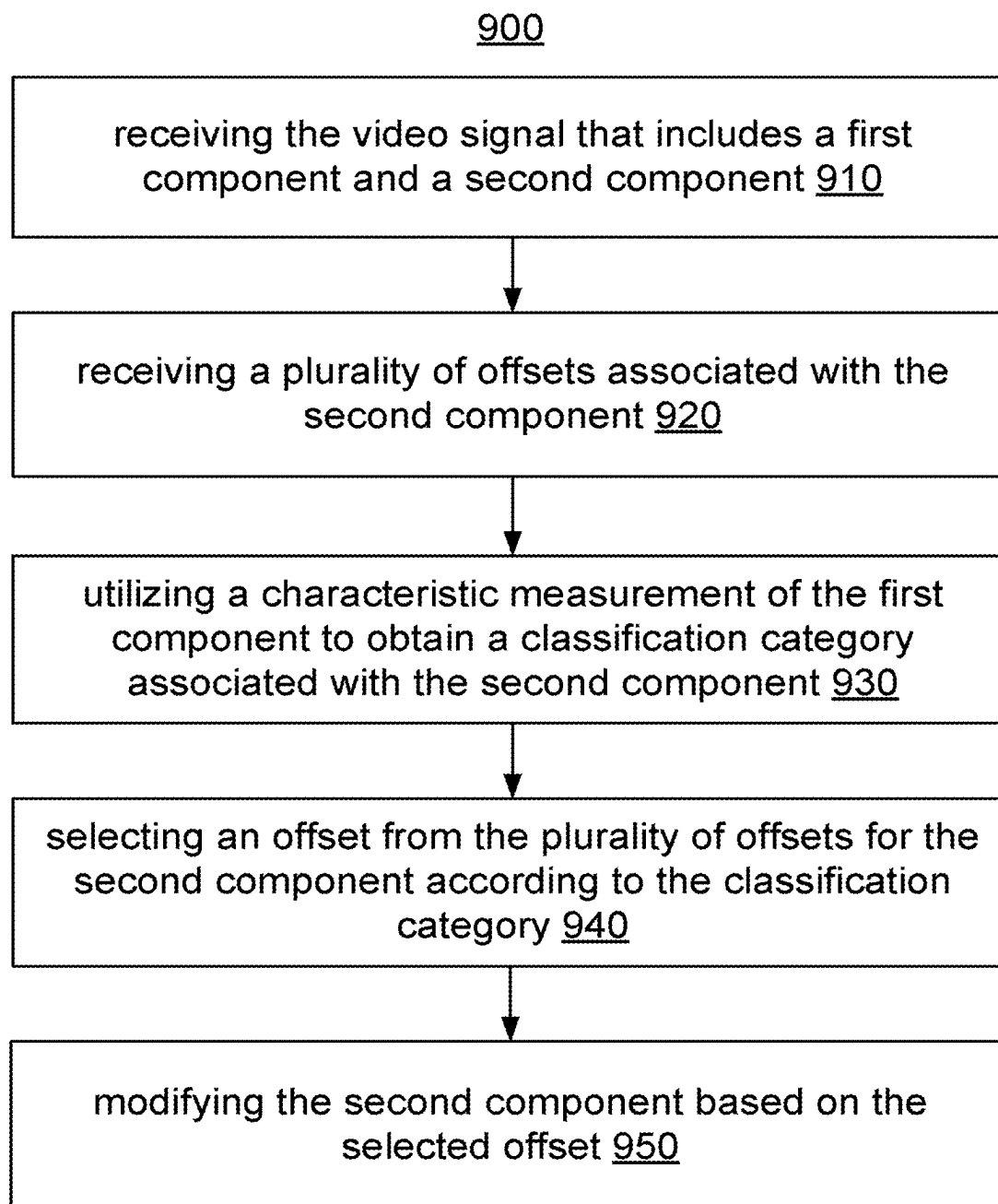


FIG. 9

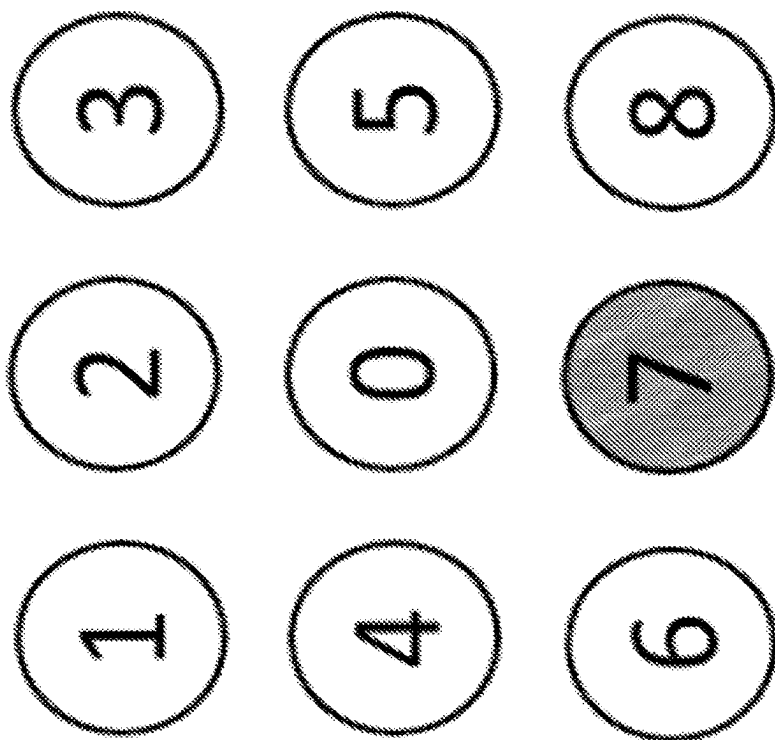


FIG. 10A

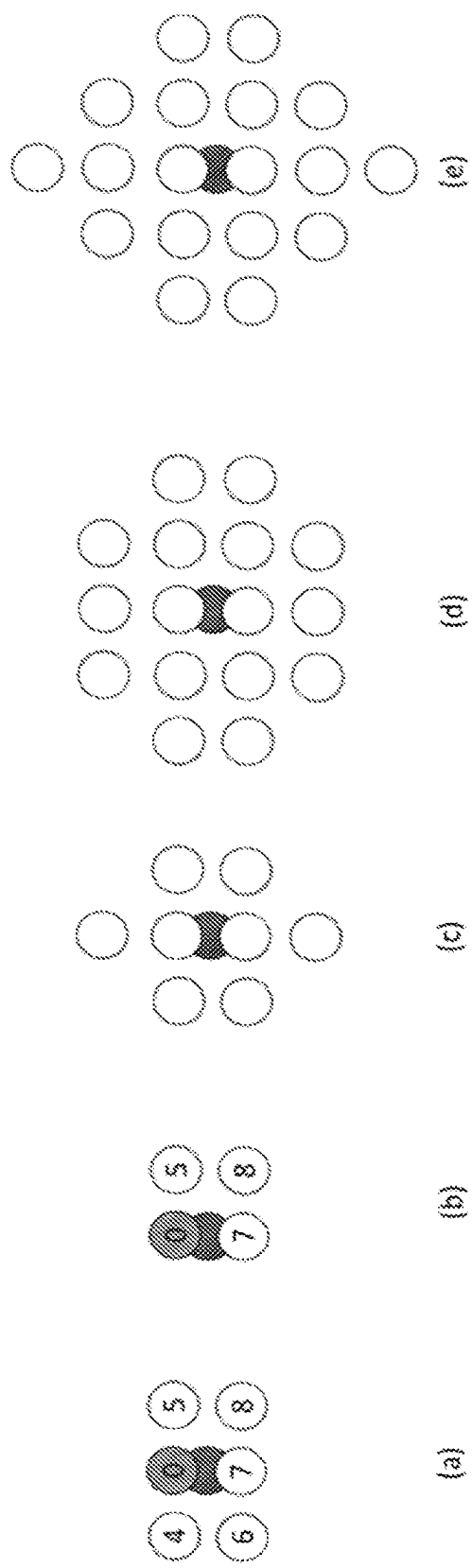


FIG. 10B

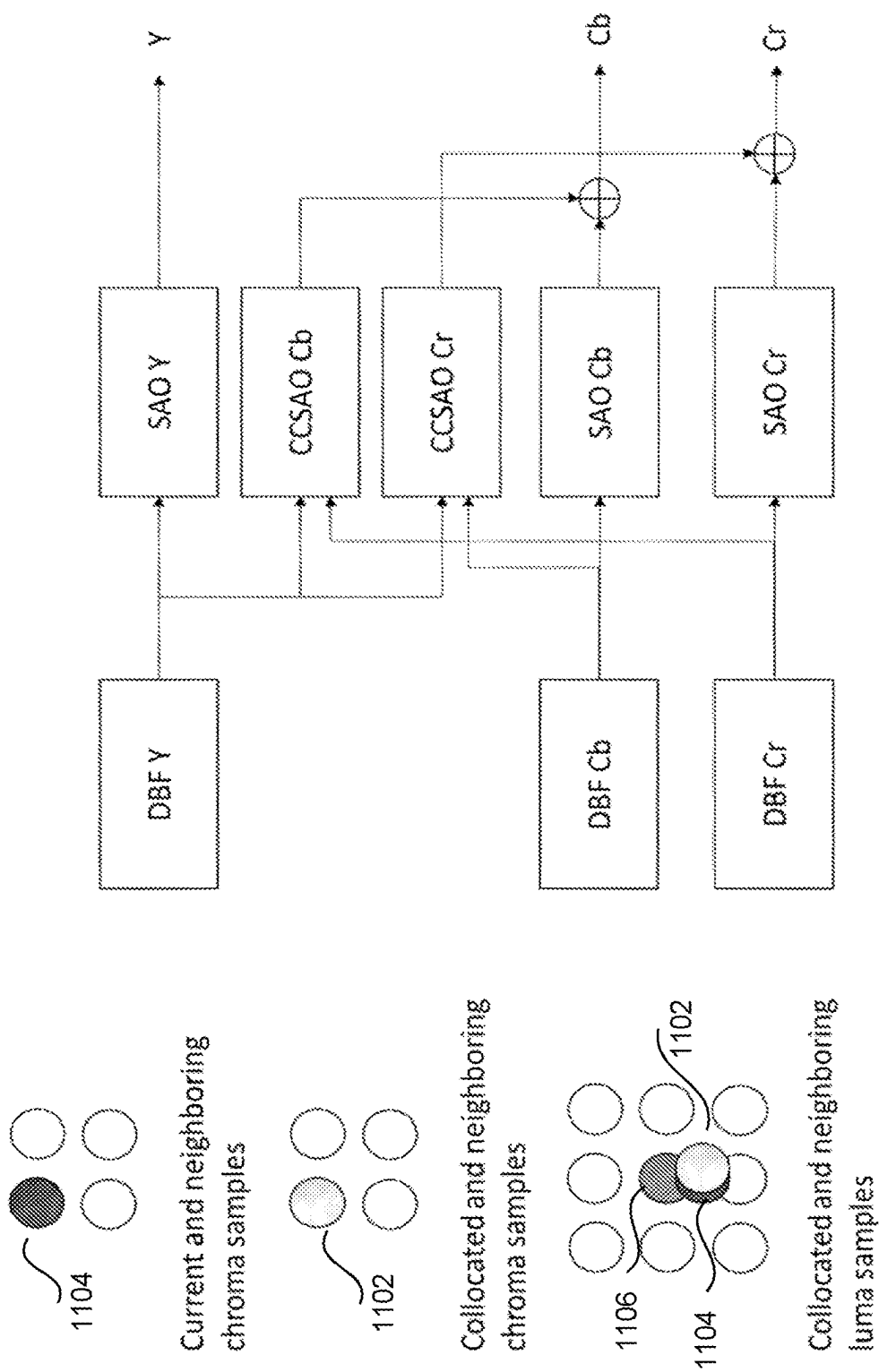


FIG. 11

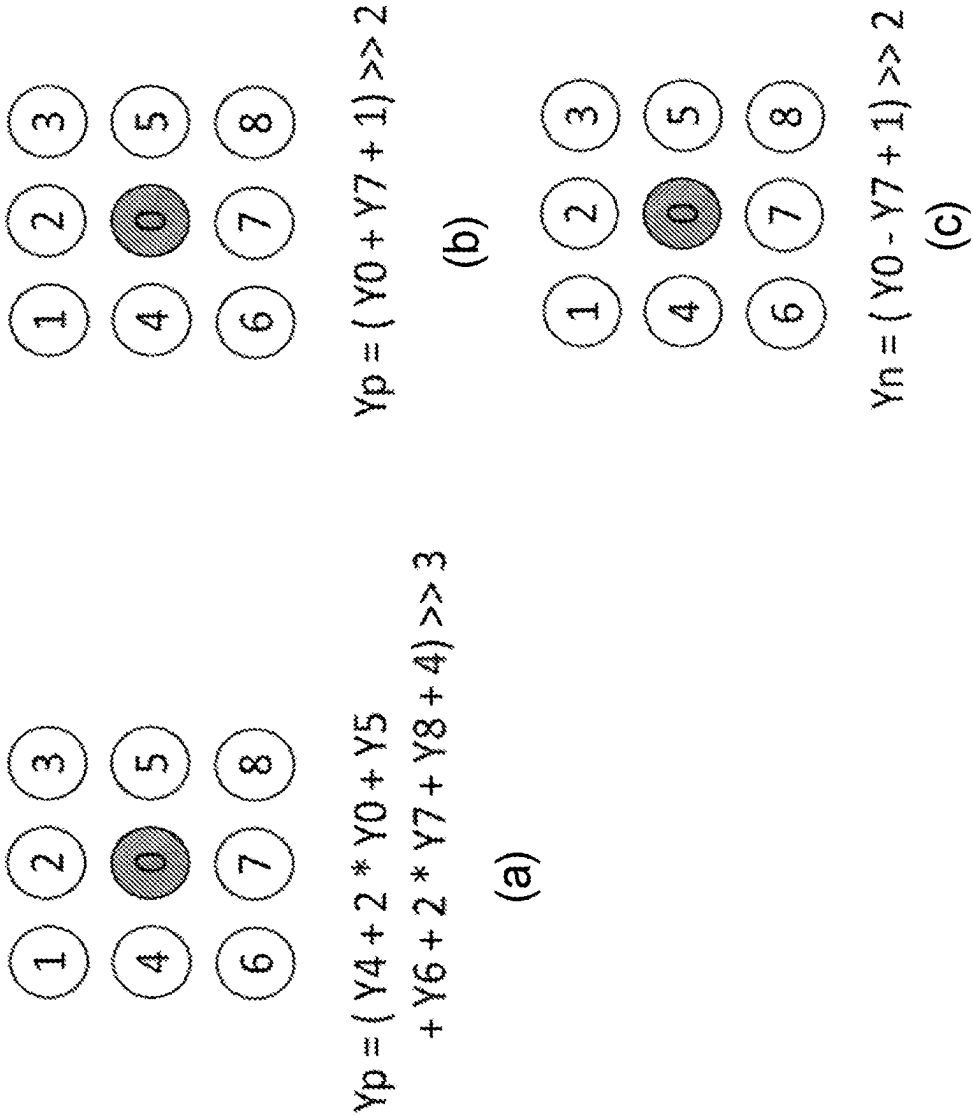


FIG. 12

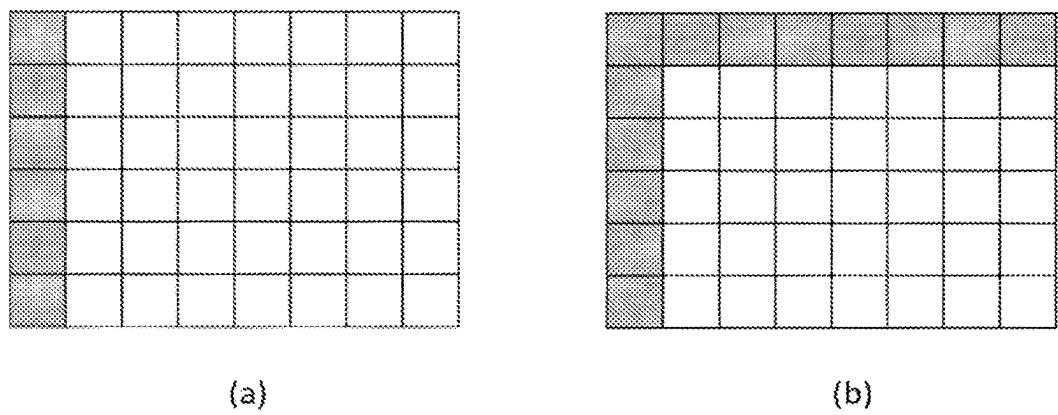


FIG. 13A

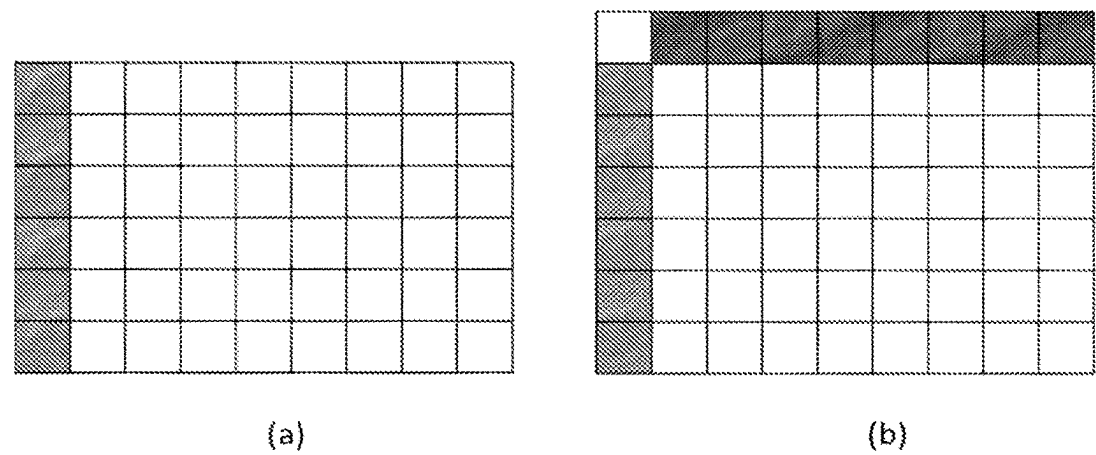


FIG. 13B

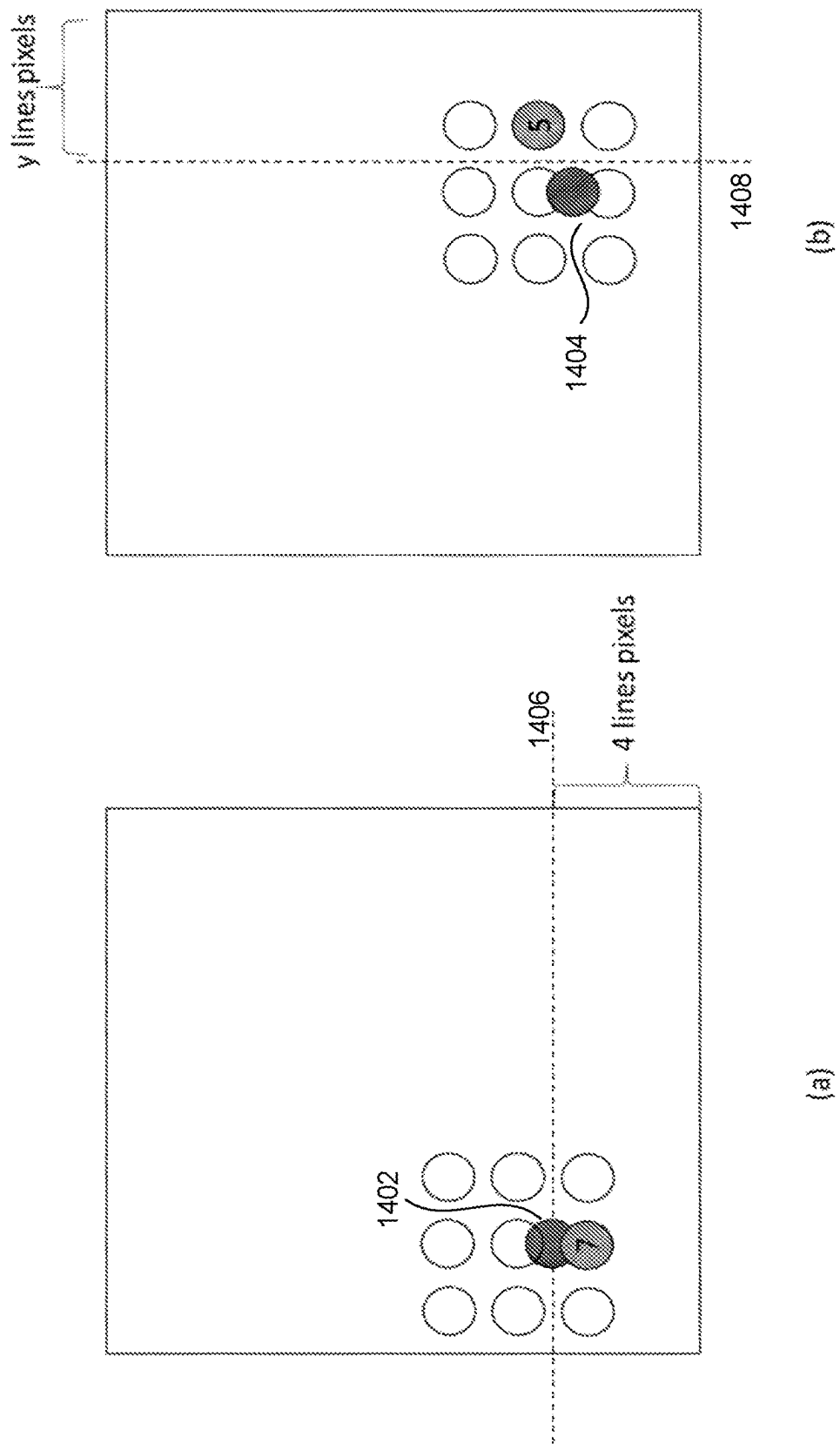
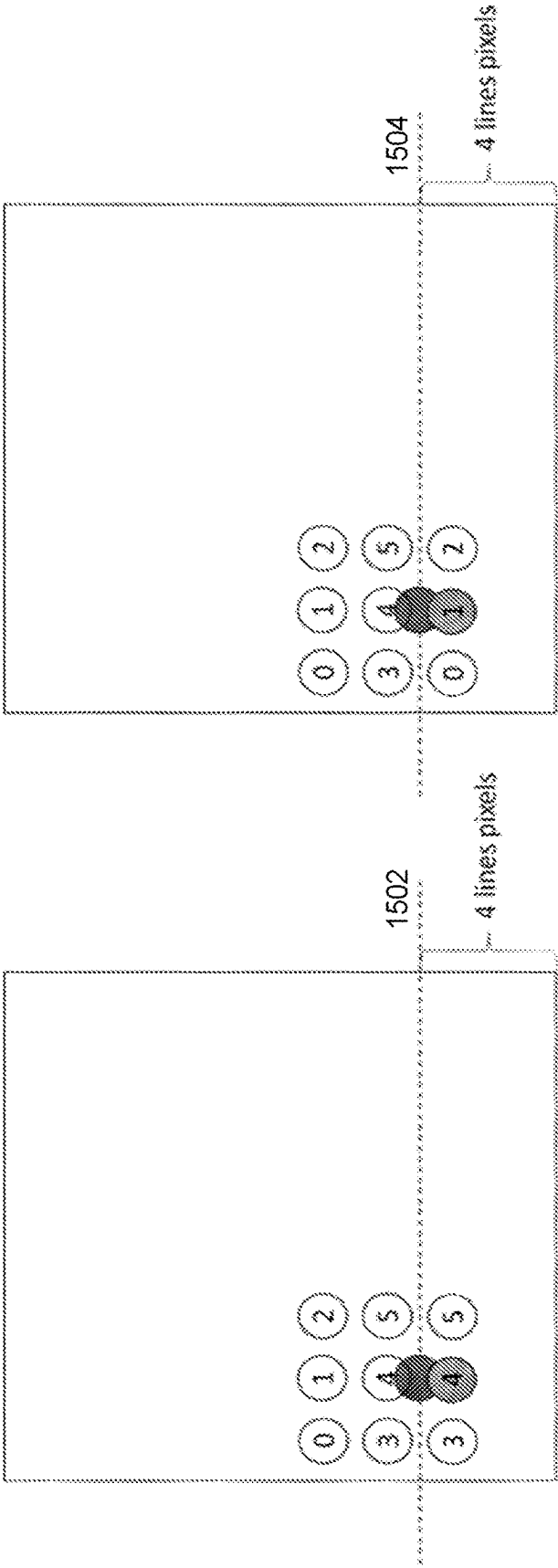


FIG. 14



(a)

(b)

FIG. 15

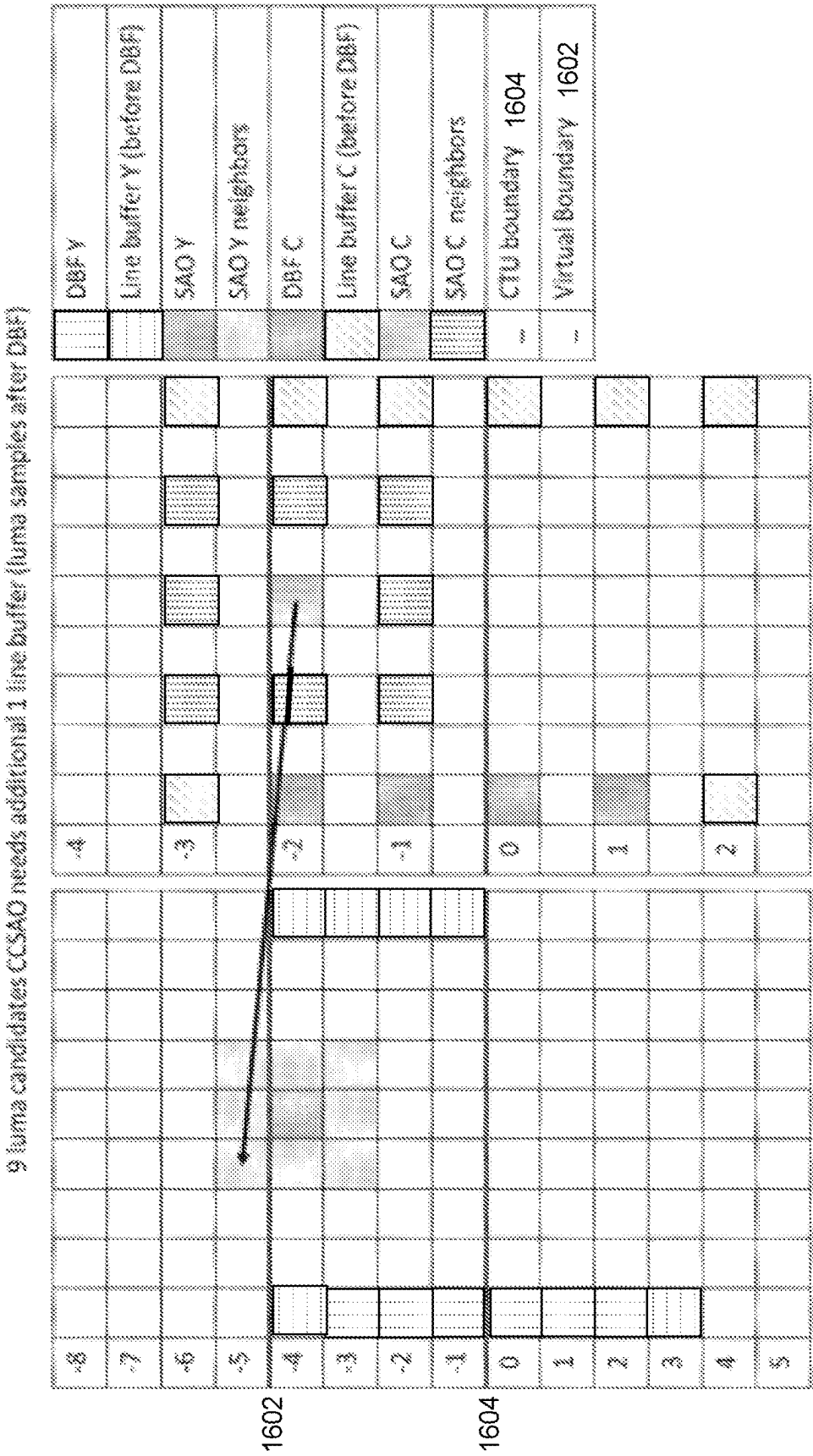


FIG. 16

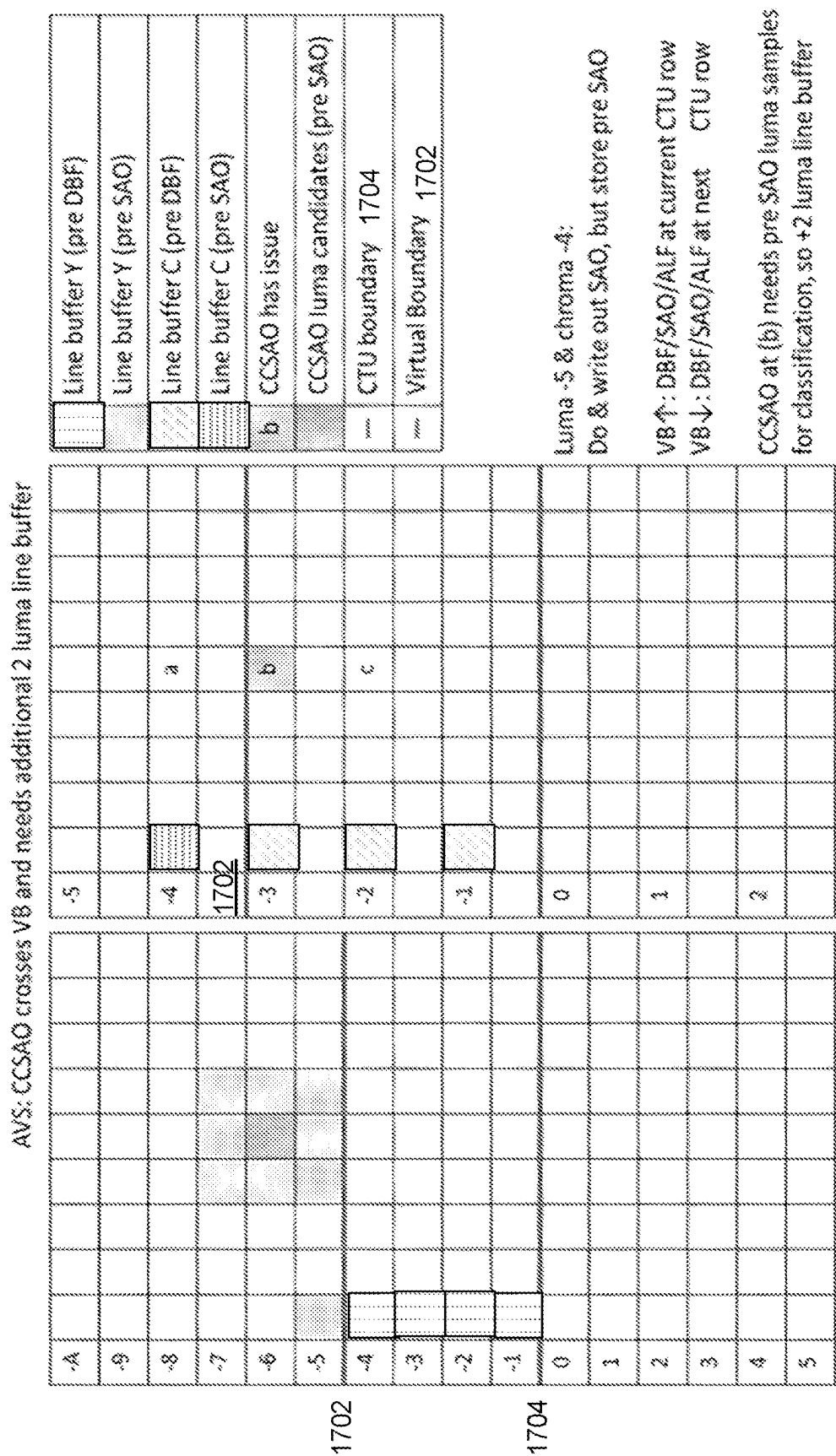


FIG. 17

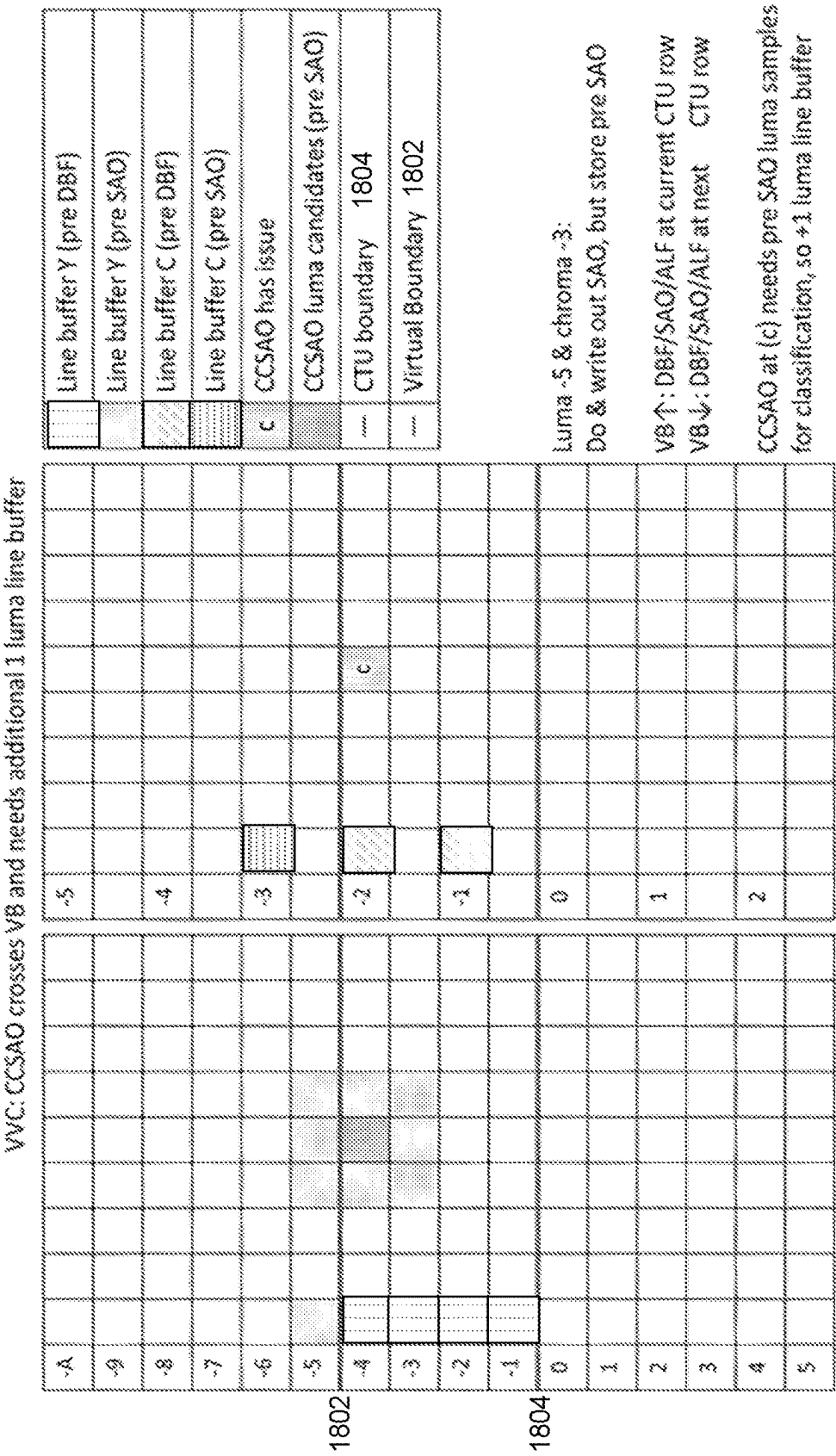
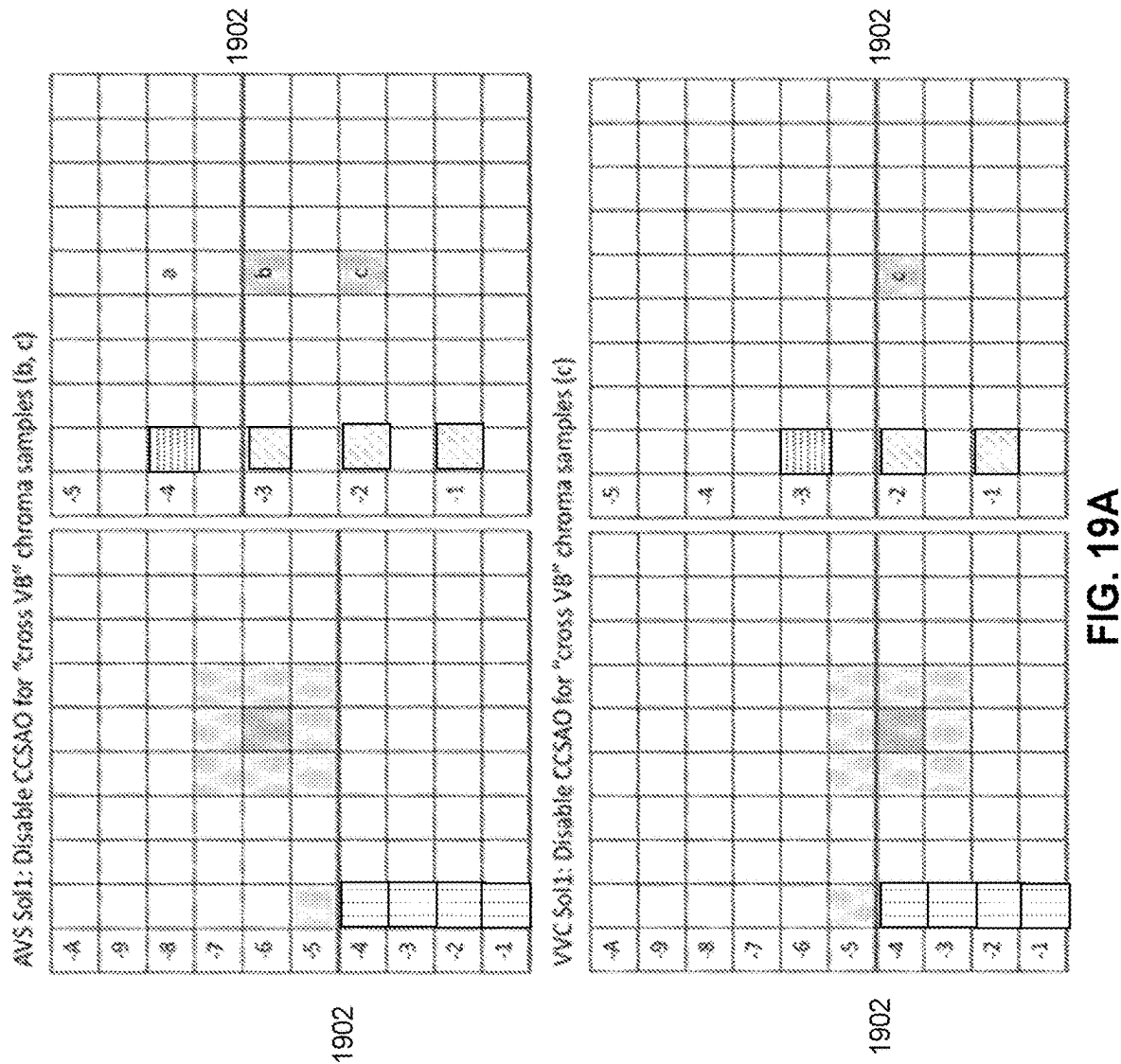
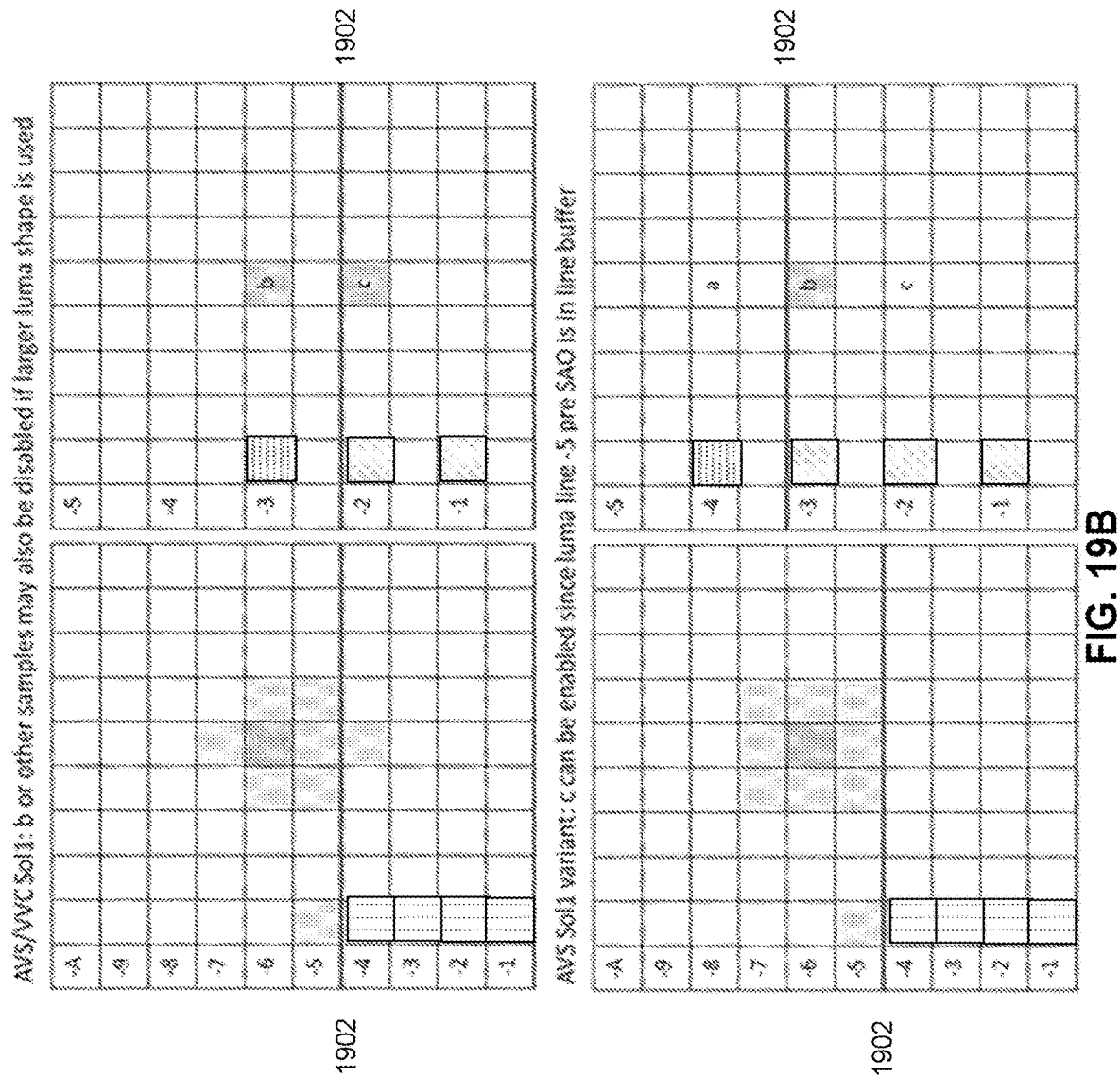
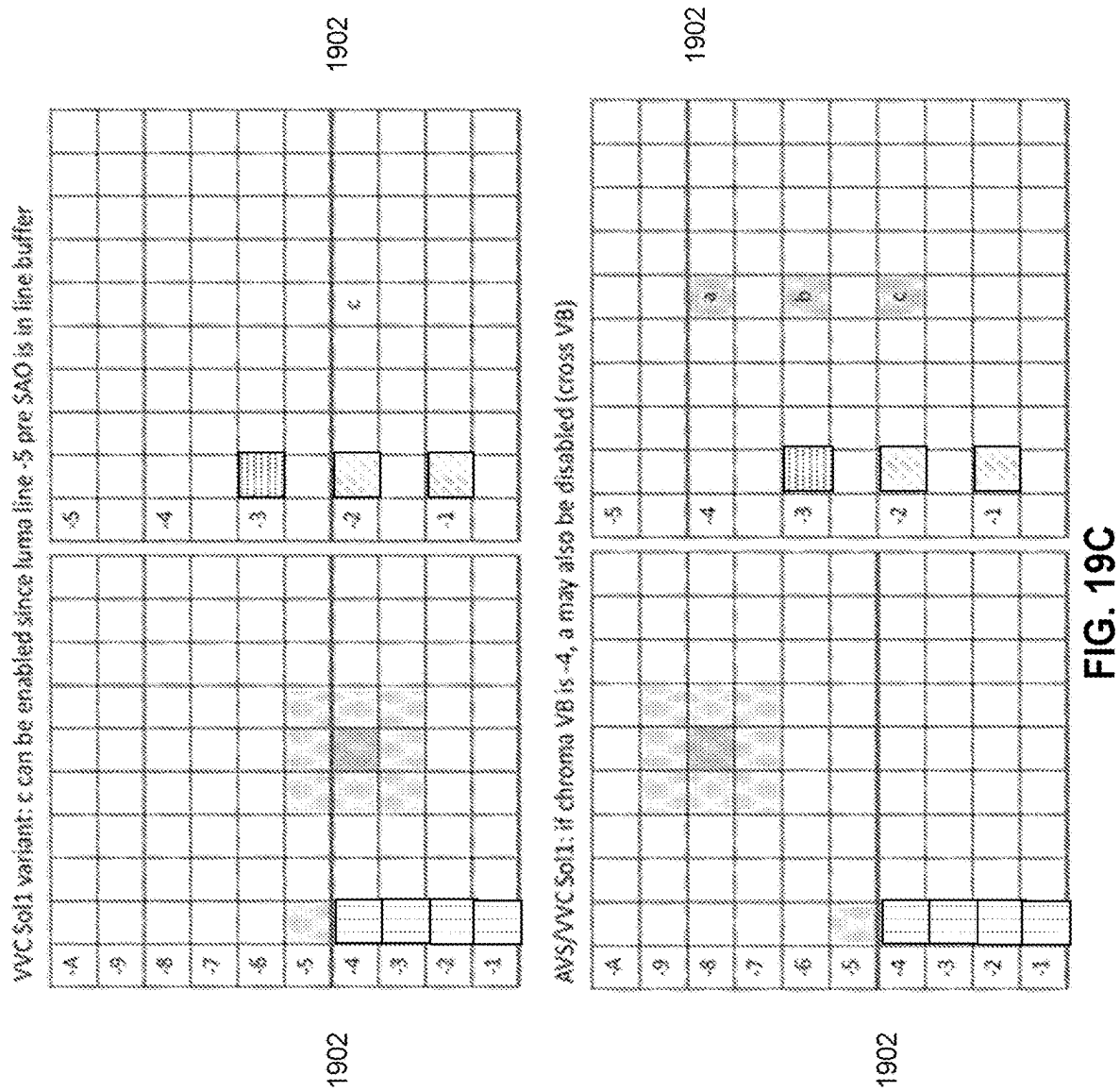


FIG. 18







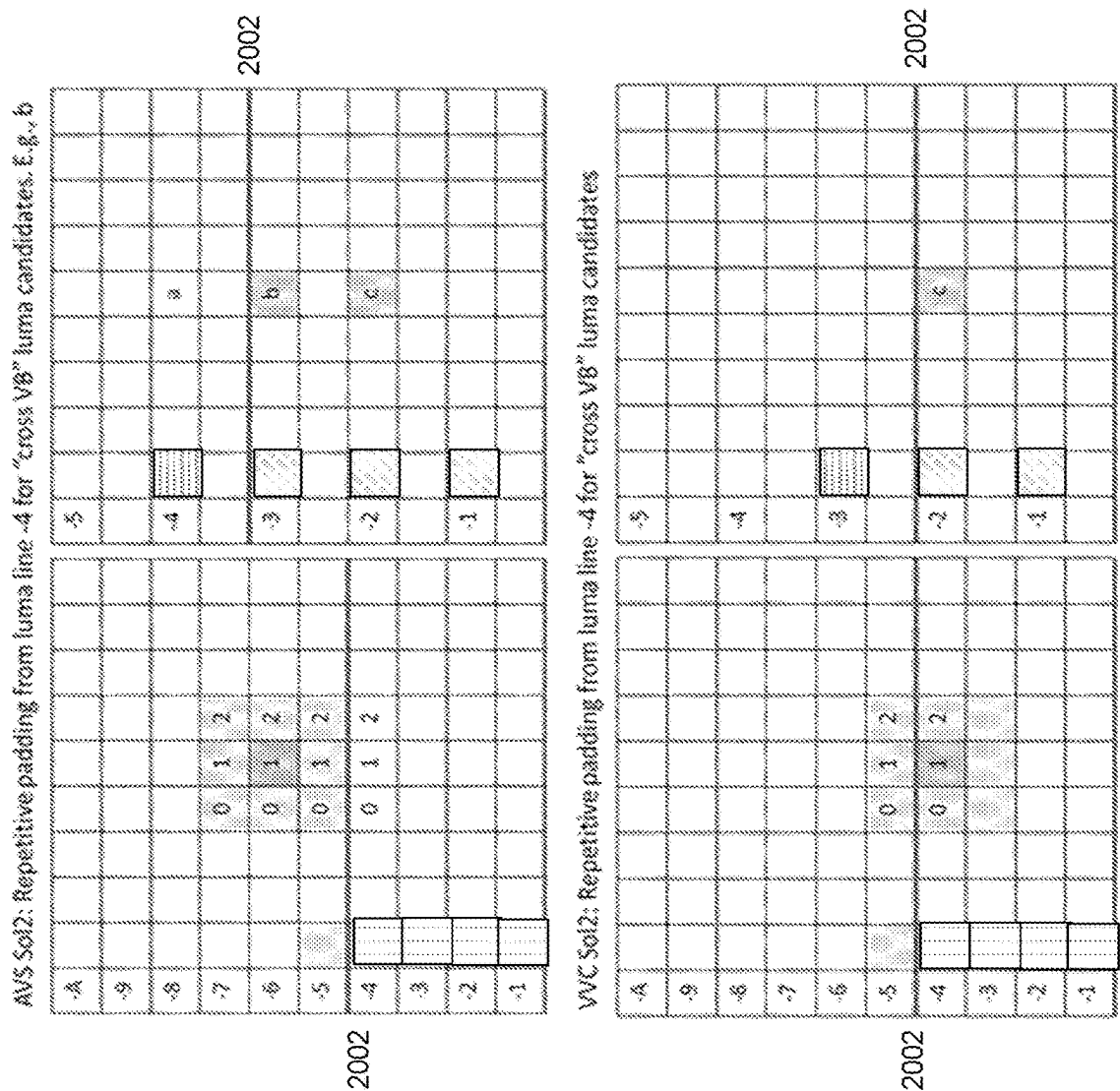


FIG. 20A

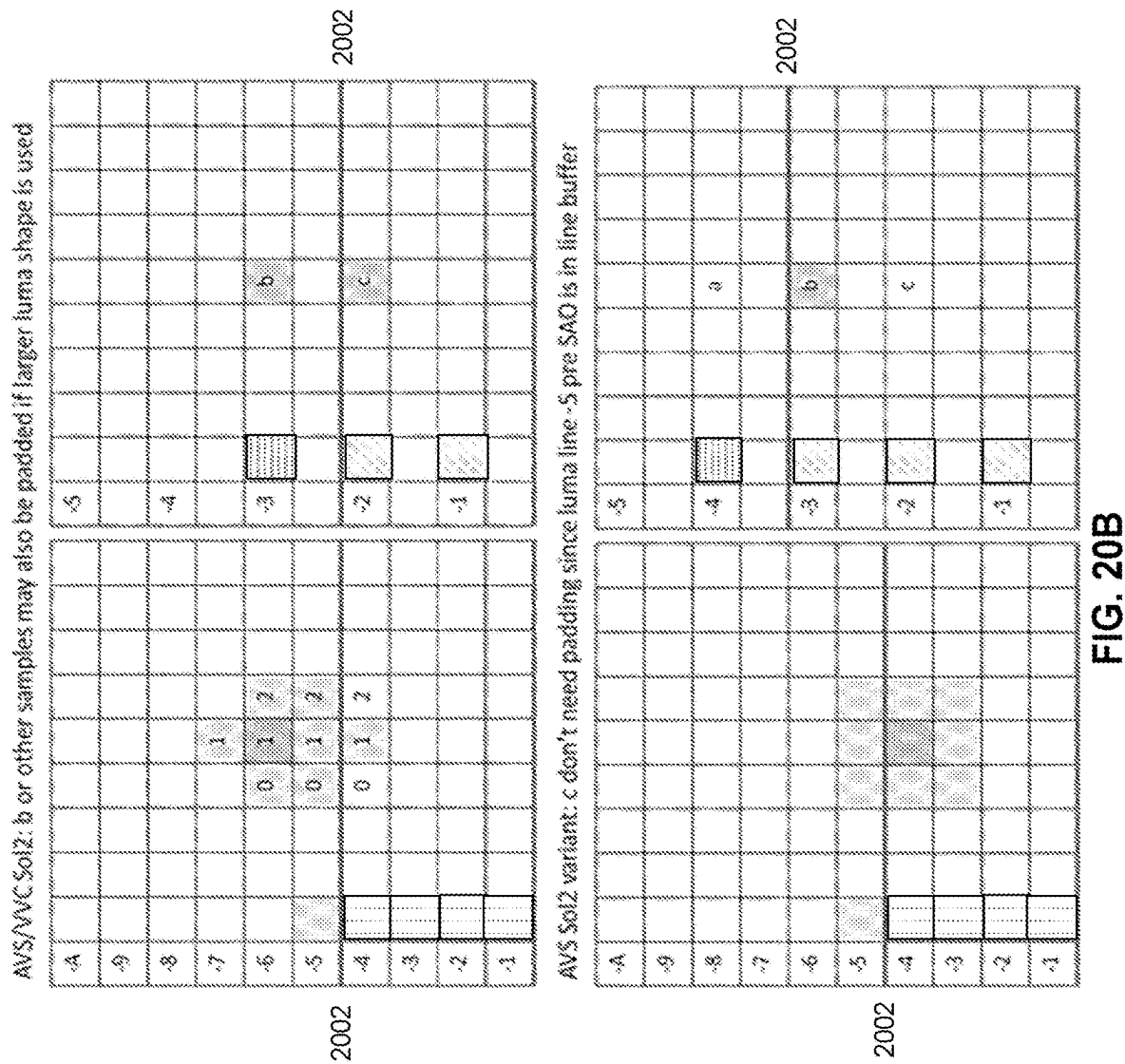
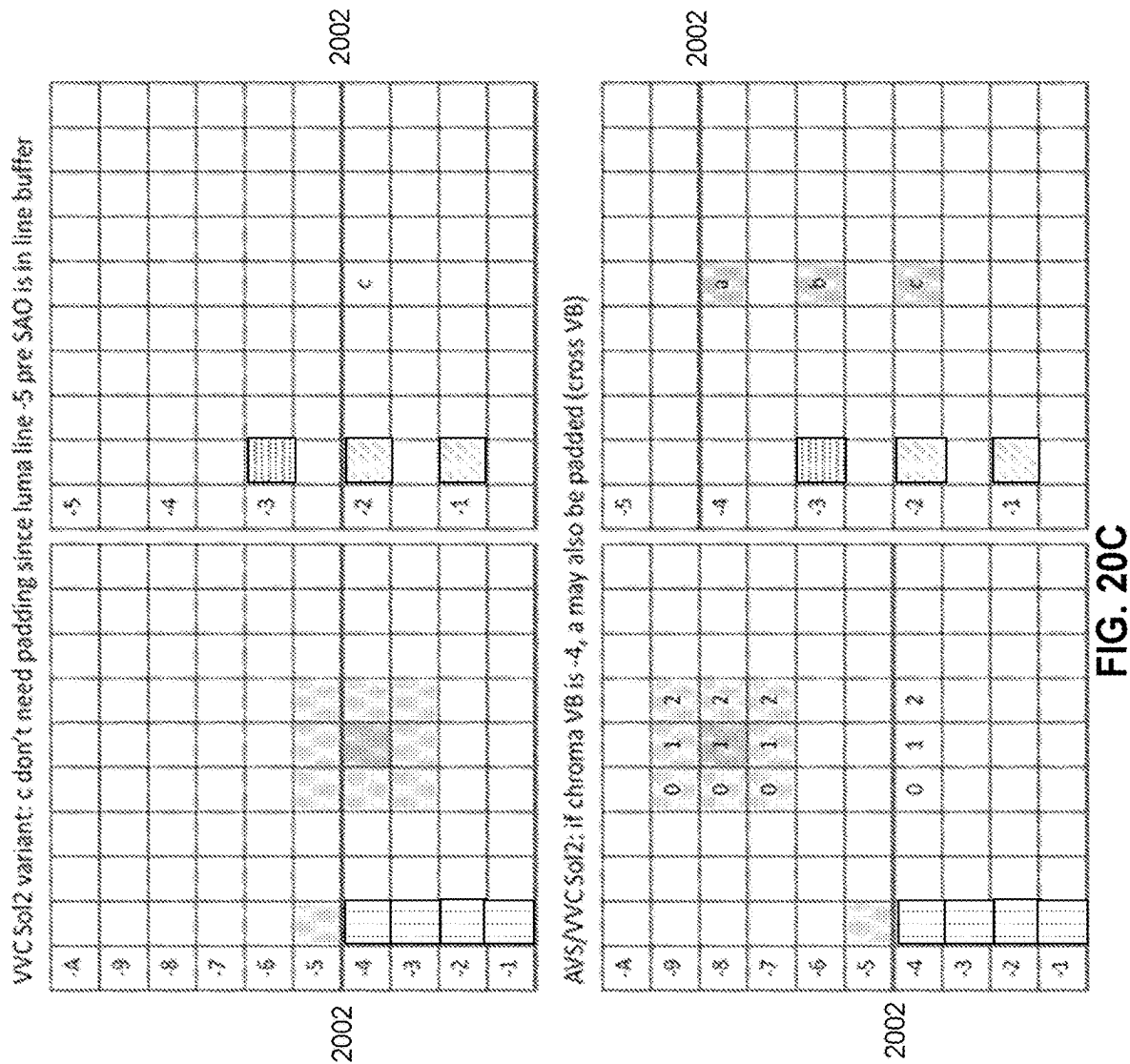
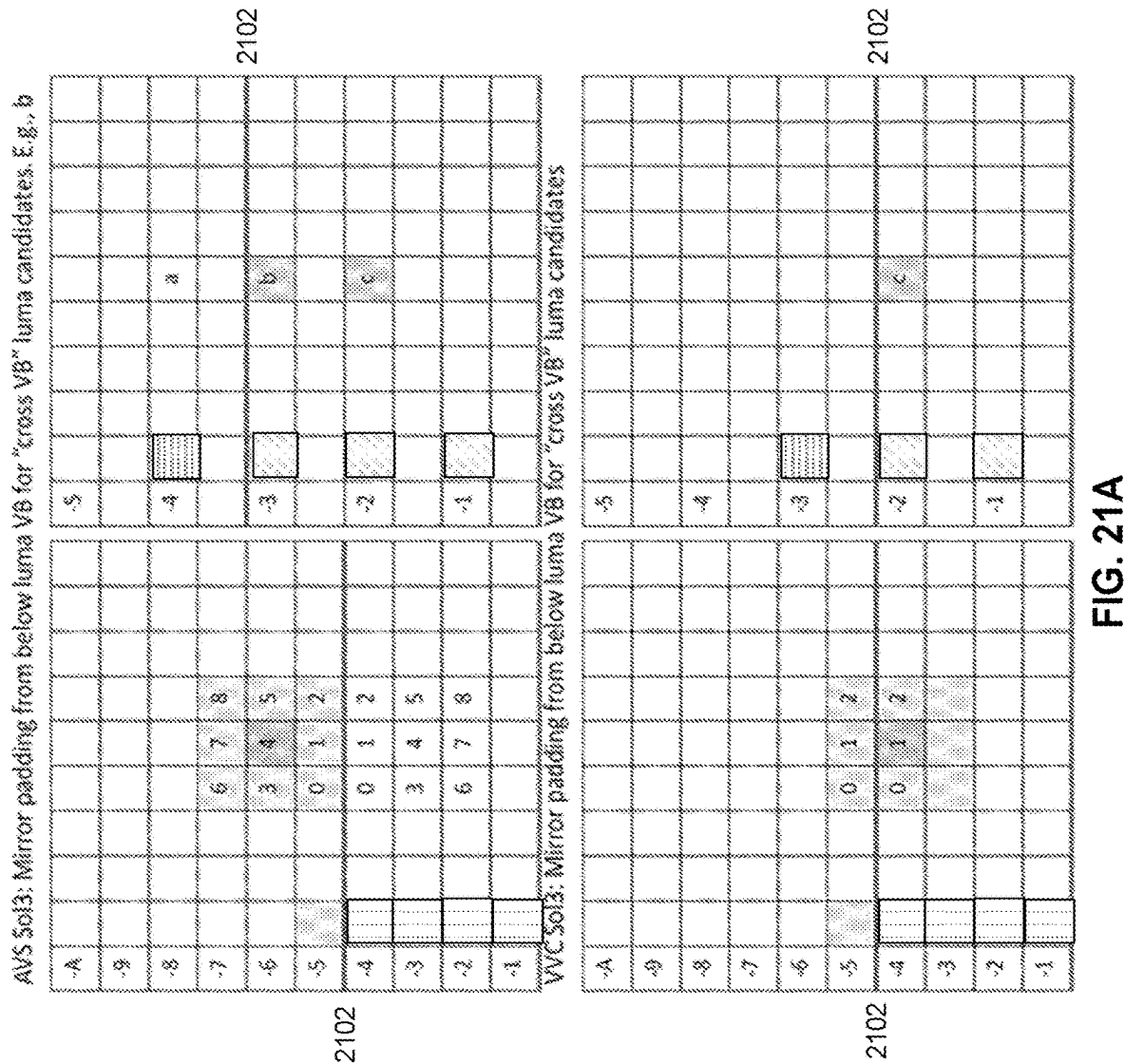
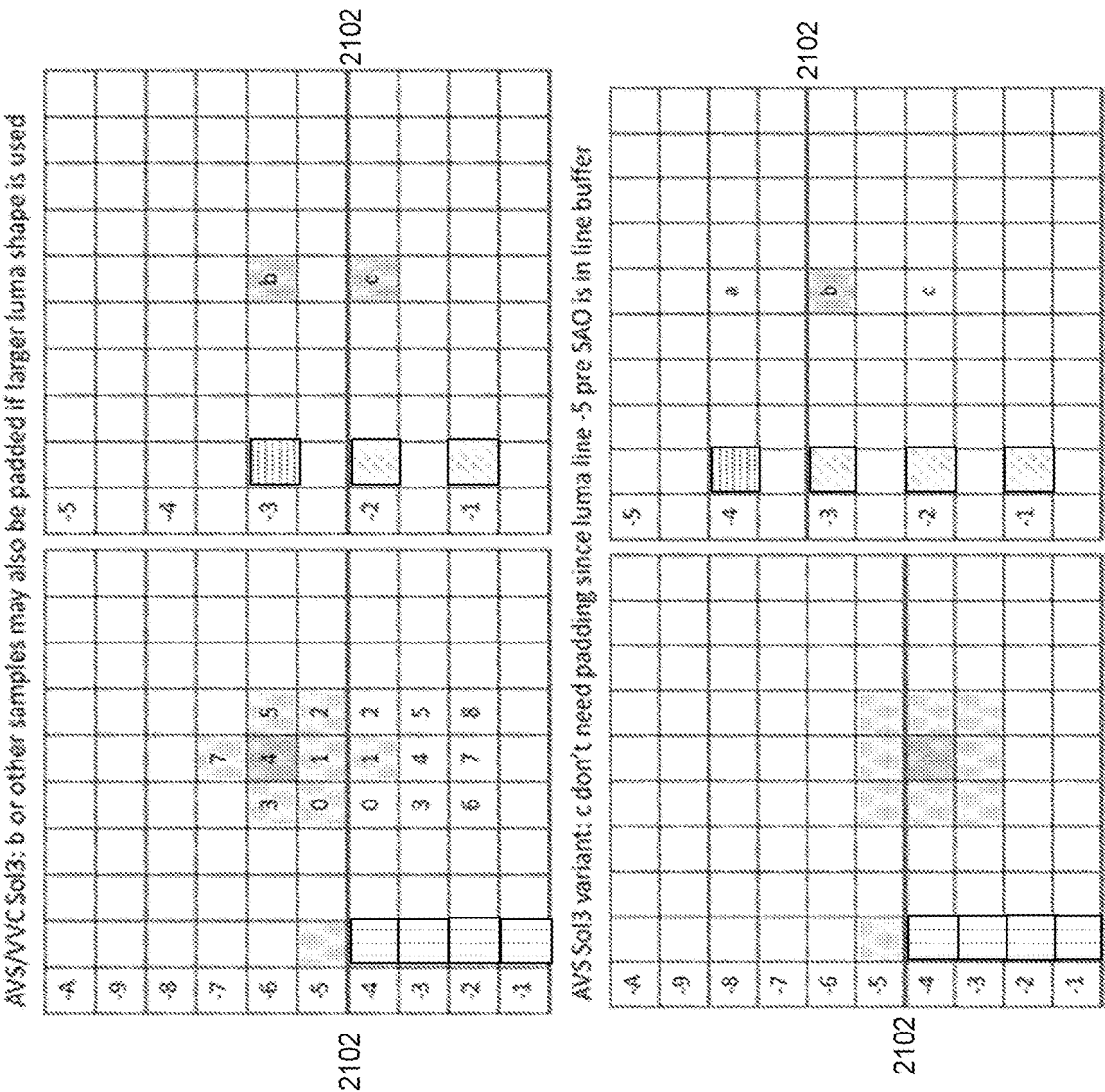
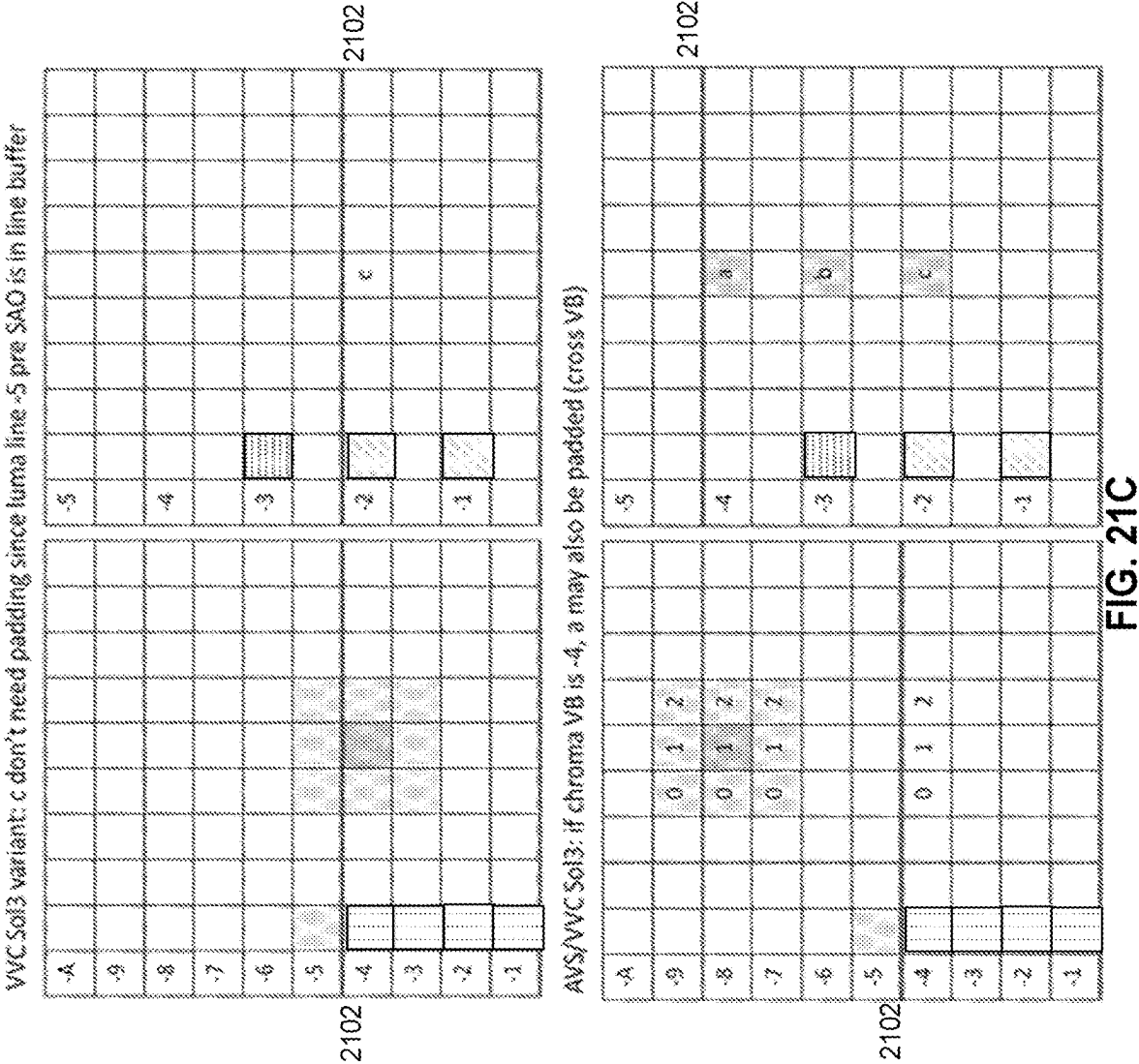


FIG. 20B



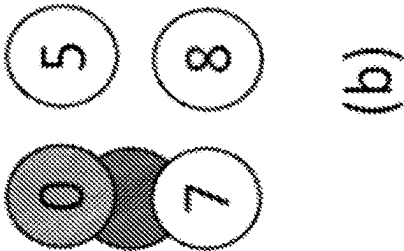








(a)



(b)

FIG. 22

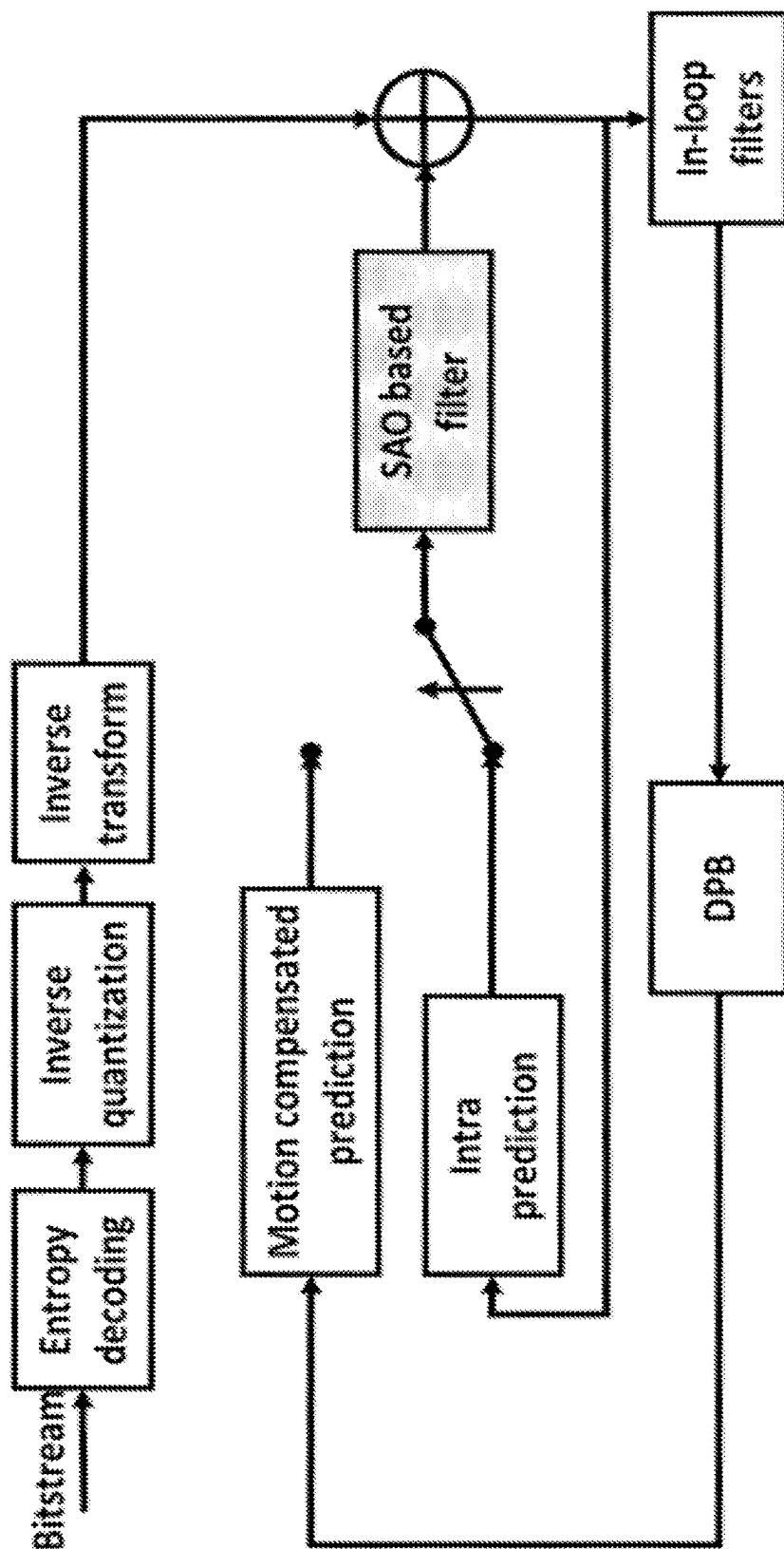


FIG. 23

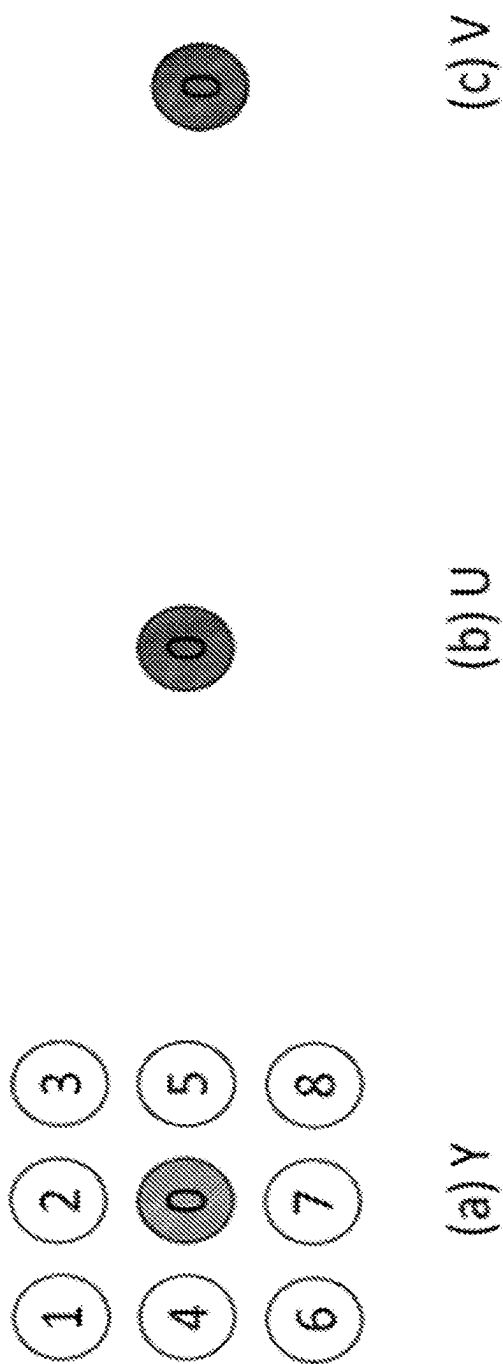


FIG. 24

2500

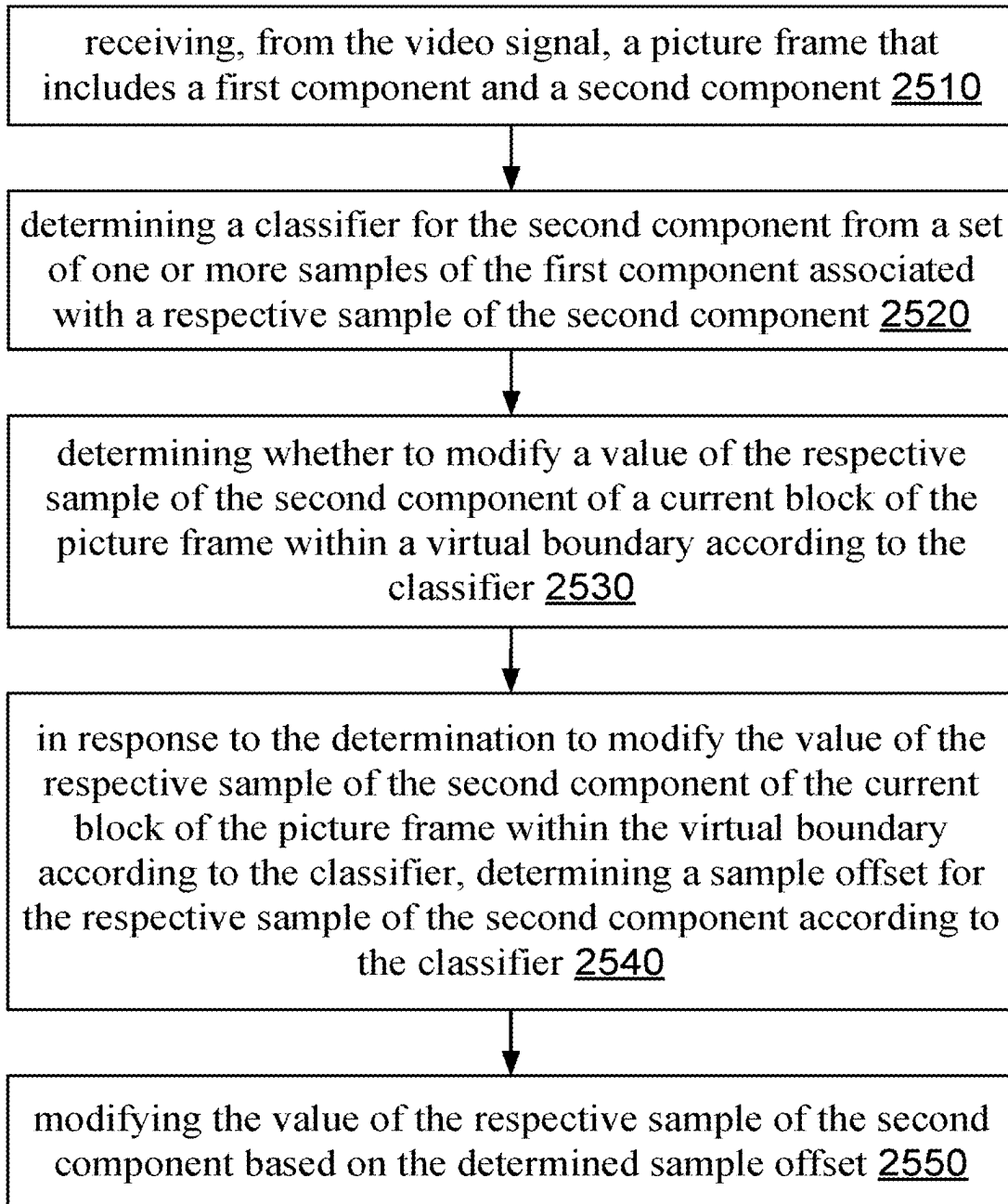


FIG. 25

CHROMA CODING ENHANCEMENT IN CROSS-COMPONENT SAMPLE ADAPTIVE OFFSET WITH VIRTUAL BOUNDARY

RELATED APPLICATIONS

[0001] The present application is a continuation of U.S. patent application Ser. No. 18/125,568, filed Mar. 23, 2023, which is a continuation of international application No. PCT/US2021/051573, filed on Sep. 22, 2021, which claims priority to U.S. Provisional Patent Application No. 63/082,356, entitled “Cross-component Sample Adaptive Offset” filed Sep. 23, 2020, and U.S. Provisional Patent Application No. 63/086,504, entitled “Cross-component Sample Adaptive Offset” filed Oct. 1, 2020. All of the above applications are incorporated by reference in their entirety.

TECHNICAL FIELD

[0002] The present application generally relates to video coding and compression, and more specifically, to methods and apparatus on improving the chroma coding efficiency.

BACKGROUND

[0003] Digital video is supported by a variety of electronic devices, such as digital televisions, laptop or desktop computers, tablet computers, digital cameras, digital recording devices, digital media players, video gaming consoles, smart phones, video teleconferencing devices, video streaming devices, etc. The electronic devices transmit, receive, encode, decode, and/or store digital video data by implementing video compression/decompression standards. Some well-known video coding standards include Versatile Video Coding (VVC), High Efficiency Video Coding (HEVC, also known as H.265 or MPEG-H Part 2) and Advanced Video Coding (AVC, also known as H.264 or MPEG-4 Part 10), which are jointly developed by ISO/IEC MPEG and ITU-T VCEG. AOMedia Video 1 (AV1) was developed by Alliance for Open Media (AOM) as a successor to its preceding standard VP9. Audio Video Coding (AVS), which refers to digital audio and digital video compression standard, is another video compression standard series developed by the Audio and Video Coding Standard Workgroup of China.

[0004] Video compression typically includes performing spatial (intra frame) prediction and/or temporal (inter frame) prediction to reduce or remove redundancy inherent in the video data. For block-based video coding, a video frame is partitioned into one or more slices, each slice having multiple video blocks, which may also be referred to as coding tree units (CTUs). Each CTU may contain one coding unit (CU) or recursively split into smaller CUs until the pre-defined minimum CU size is reached. Each CU (also named leaf CU) contains one or multiple transform units (TUs) and each CU also contains one or multiple prediction units (PUs). Each CU can be coded in either intra, inter or IBC modes. Video blocks in an intra coded (I) slice of a video frame are encoded using spatial prediction with respect to reference samples in neighboring blocks within the same video frame. Video blocks in an inter coded (P or B) slice of a video frame may use spatial prediction with respect to reference samples in neighboring blocks within the same video frame or temporal prediction with respect to reference samples in other previous and/or future reference video frames.

[0005] Spatial or temporal prediction based on a reference block that has been previously encoded, e.g., a neighboring block, results in a predictive block for a current video block to be coded. The process of finding the reference block may be accomplished by block matching algorithm. Residual data representing pixel differences between the current block to be coded and the predictive block is referred to as a residual block or prediction errors. An inter-coded block is encoded according to a motion vector that points to a reference block in a reference frame forming the predictive block, and the residual block. The process of determining the motion vector is typically referred to as motion estimation. An intra coded block is encoded according to an intra prediction mode and the residual block. For further compression, the residual block is transformed from the pixel domain to a transform domain, e.g., frequency domain, resulting in residual transform coefficients, which may then be quantized. The quantized transform coefficients, initially arranged in a two-dimensional array, may be scanned to produce a one-dimensional vector of transform coefficients, and then entropy encoded into a video bitstream to achieve even more compression.

[0006] The encoded video bitstream is then saved in a computer-readable storage medium (e.g., flash memory) to be accessed by another electronic device with digital video capability or directly transmitted to the electronic device wired or wirelessly. The electronic device then performs video decompression (which is an opposite process to the video compression described above) by, e.g., parsing the encoded video bitstream to obtain syntax elements from the bitstream and reconstructing the digital video data to its original format from the encoded video bitstream based at least in part on the syntax elements obtained from the bitstream, and renders the reconstructed digital video data on a display of the electronic device.

[0007] With digital video quality going from high definition, to 4K×2K or even 8K×4K, the amount of video data to be encoded/decoded grows exponentially. It is a constant challenge in terms of how the video data can be encoded/decoded more efficiently while maintaining the image quality of the decoded video data.

SUMMARY

[0008] The present application describes implementations related to video data encoding and decoding and, more particularly, to methods and apparatus on improving the coding efficiency of chroma coding, including improving the coding efficiency by exploring cross-component relationship between luma and chroma components.

[0009] According to a first aspect of the present application, a method of decoding a video signal comprises: receiving, from the video signal, a picture frame that includes a first component and a second component; in response to a current Coding Tree Unit (CTU) being not located within a bottom row of CTUs of a frame, slice, tile, subpicture or patch, determining a set of one or more samples of the first component according to a positional relationship between a respective sample of the second component and a virtual boundary; determining a class index for the second component from the set of one or more samples of the first component associated with the respective sample of the second component; determining a sample offset for the respective sample of the second component according to the

class index; and obtaining a cross-component offsetted sample value of the second component based on the determined sample offset.

[0010] In at least one embodiment, the method further comprises: in response to the current CTU being located within the bottom row of CTUs of the frame, slice, tile, subpicture or patch, determining the set of one or more samples of the first component regardless of the virtual boundary, wherein the set of one or more samples comprises a collocated sample of the first component relative to the respective sample of the second component or a neighboring sample of the collocated sample of the first component relative to the respective sample of the second component.

[0011] In at least one embodiment, determining the set of one or more samples of the first component according to the positional relationship between the respective sample of the second component and the virtual boundary comprises: determining a first set of one or more samples of the first component as the set of one or more samples of the first component when the respective sample of the second component is above the virtual boundary; and determining a second set of one or more samples of the first component as the set of one or more samples of the first component when the respective sample of the second component is on or below the virtual boundary, wherein the second set of one or more samples is different from the first set of one or more samples.

[0012] In at least one embodiment, the first set of one or more samples comprises a collocated sample of the first component relative to the respective sample of the second component or a neighboring sample of the collocated sample of the first component relative to the respective sample of the second component.

[0013] In at least one embodiment, the virtual boundary comprises a virtual boundary within a Coding Tree Block (CTB) of the second component.

[0014] In at least one embodiment, the virtual boundary is parallel with a block boundary and separated from the block boundary by at least one row or column of samples.

[0015] According to a second aspect of the present application, a method of decoding video signal comprises: receiving, from the video signal, a picture frame that includes a first component and a second component; determining a classifier for the second component from a set of one or more samples of the first component associated with a respective sample of the second component; determining whether to modify a value of the respective sample of the second component of a current block of the picture frame within a virtual boundary according to the classifier; in response to the determination to modify the value of the respective sample of the second component of the current block of the picture frame within the virtual boundary according to the classifier, determining a sample offset for the respective sample of the second component according to the classifier; and modifying the value of the respective sample of the second component based on the determined sample offset.

[0016] According to a third aspect of the present application, an electronic apparatus includes one or more processing units, memory and a plurality of programs stored in the memory. The programs, when executed by the one or more processing units, cause the electronic apparatus to perform the method of coding video data as described above.

[0017] According to a fourth aspect of the present application, a non-transitory computer readable storage medium stores a plurality of programs for execution by an electronic apparatus having one or more processing units. The programs, when executed by the one or more processing units, cause the electronic apparatus to perform the method of coding video data as described above.

BRIEF DESCRIPTION OF DRAWINGS

[0018] The accompanying drawings, which are included to provide a further understanding of the implementations and are incorporated herein and constitute a part of the specification, illustrate the described implementations and together with the description serve to explain the underlying principles. Like reference numerals refer to corresponding parts.

[0019] FIG. 1 is a block diagram illustrating an exemplary video encoding and decoding system in accordance with some implementations of the present disclosure.

[0020] FIG. 2 is a block diagram illustrating an exemplary video encoder in accordance with some implementations of the present disclosure.

[0021] FIG. 3 is a block diagram illustrating an exemplary video decoder in accordance with some implementations of the present disclosure.

[0022] FIGS. 4A through 4E are block diagrams illustrating how a frame is recursively partitioned into multiple video blocks of different sizes and shapes in accordance with some implementations of the present disclosure.

[0023] FIG. 5 is a block diagram depicting the four gradient patterns used in Sample Adaptive Offset (SAO) in accordance with some implementations of the present disclosure.

[0024] FIG. 6A is a block diagram illustrating the system and process of Cross-Component Sample Adaptive Offset (CCSAO) according to some implementations of the present disclosure.

[0025] FIG. 6B is a block diagram illustrating the system and process of CCSAO applied in parallel with Enhanced Sample Adaptive Offset (ESAO) in the AVS standard according to some implementations of the present disclosure.

[0026] FIG. 6C is a block diagram illustrating the system and process of CCSAO applied after SAO according to some implementations of the present disclosure.

[0027] FIG. 6D is a block diagram illustrating the system and process of CCSAO applied in parallel with Cross-Component Adaptive Loop Filter (CCALF) according to some implementations of the present disclosure.

[0028] FIG. 7 is a block diagram illustrating a sample process using CCSAO in accordance with some implementations of the present disclosure.

[0029] FIG. 8 is a block diagram illustrating that CCSAO process is interleaved to vertical and horizontal deblocking filter (DBF) in accordance with some implementations of the present disclosure.

[0030] FIG. 9 is a flowchart illustrating an exemplary process of decoding video signal using cross-component correlation in accordance with some implementations of the present disclosure.

[0031] FIG. 10A is a block diagram showing a classifier using different luma sample position for classification in accordance with some implementations of the present disclosure.

[0032] FIG. 10B illustrates some examples of different shapes for luma candidates, in accordance with some implementations of the present disclosure.

[0033] FIG. 11 is a block diagram of a sample process illustrating that besides luma, the other cross-component collocated and neighboring chroma samples are also fed into CCSAO classification in accordance with some implementations of the present disclosure.

[0034] FIG. 12 illustrates exemplary classifiers by replacing the collocated luma sample value with a value obtained by weighing collocated and neighboring luma samples in accordance with some implementations of the present disclosure.

[0035] FIG. 13A is a block diagram illustrating CCSAO is not applied on the current chroma sample if any of the collocated and neighboring luma samples used for classification is outside the current picture in accordance with some implementations of the present disclosure.

[0036] FIG. 13B is a block diagram illustrating CCSAO is applied on the current chroma sample if any of the collocated and neighboring luma samples used for classification is outside the current picture in accordance with some implementations of the present disclosure.

[0037] FIG. 14 is a block diagram illustrating CCSAO is not applied on the current chroma sample if a corresponding selected collocated or neighboring luma sample used for classification is outside a virtual space defined by a virtual boundary (VB) in accordance with some implementations of the present disclosure.

[0038] FIG. 15 shows repetitive or mirror padding is applied on the luma samples that are outside the virtual boundary in accordance with some implementations of the present disclosure.

[0039] FIG. 16 shows additional 1 luma line buffer is required if all 9 collocated neighboring luma samples are used for classification in accordance with some implementations of the present disclosure.

[0040] FIG. 17 shows an illustration in AVS that 9 luma candidates CCSAO crossing VB may increase 2 additional luma line buffers in accordance with some implementations of the present disclosure.

[0041] FIG. 18 shows an illustration in VVC that 9 luma candidates CCSAO crossing VB may increase 1 additional luma line buffer in accordance with some implementations of the present disclosure.

[0042] FIGS. 19A-19C show in AVS and VVC, CCSAO is disabled for a chroma sample if any of the chroma sample's luma candidates is across VB (outside the current chroma sample VB) in accordance with some implementations of the present disclosure.

[0043] FIGS. 20A-20C show in AVS and VVC, CCSAO is enabled using repetitive padding for a chroma sample if any of the chroma sample's luma candidates is across VB (outside the current chroma sample VB) in accordance with some implementations of the present disclosure.

[0044] FIGS. 21A-21C show in AVS and VVC, CCSAO is enabled using mirror padding for a chroma sample if any of the chroma sample's luma candidates is across VB (outside the current chroma sample VB) in accordance with some implementations of the present disclosure.

[0045] FIG. 22 shows the restrictions of using a limited number of luma candidates for classification in accordance with some implementations of the present disclosure.

[0046] FIG. 23 is a block diagram illustrating that the SAO classification methods disclosed in the present disclosure serve as a post prediction filter in accordance with some implementations of the present disclosure.

[0047] FIG. 24 is a block diagram illustrating that for post prediction SAO filter, each component can use the current and neighboring samples for classification in accordance with some implementations of the present disclosure.

[0048] FIG. 25 is a flowchart illustrating an exemplary process of decoding video signal using cross-component correlation when Virtual Boundary is present in accordance with some implementations of the present disclosure.

DETAILED DESCRIPTION

[0049] Reference will now be made in detail to specific implementations, examples of which are illustrated in the accompanying drawings. In the following detailed description, numerous non-limiting specific details are set forth in order to assist in understanding the subject matter presented herein. But it will be apparent to one of ordinary skill in the art that various alternatives may be used without departing from the scope of claims and the subject matter may be practiced without these specific details. For example, it will be apparent to one of ordinary skill in the art that the subject matter presented herein can be implemented on many types of electronic devices with digital video capabilities.

[0050] The first generation AVS standard includes Chinese national standard "Information Technology, Advanced Audio Video Coding, Part 2: Video" (known as AVS1) and "Information Technology, Advanced Audio Video Coding Part 16: Radio Television Video" (known as AVS+). It can offer around 50% bit-rate saving at the same perceptual quality compared to MPEG-2 standard. The second generation AVS standard includes the series of Chinese national standard "Information Technology, Efficient Multimedia Coding" (known as AVS2), which is mainly targeted at the transmission of extra HD TV programs. The coding efficiency of the AVS2 is double of that of the AVS+. Meanwhile, the AVS2 standard video part was submitted by Institute of Electrical and Electronics Engineers (IEEE) as one international standard for applications. The AVS3 standard is one new generation video coding standard for UHD video application aiming at surpassing the coding efficiency of the latest international standard HEVC, which provides approximately 30% bit-rate savings over the HEVC standard. In March 2019, at the 68-th AVS meeting, the AVS3-P2 baseline was finished, which provides approximately 30% bit-rate savings over the HEVC standard. Currently, one reference software, called high performance model (HPM), is maintained by the AVS group to demonstrate a reference implementation of the AVS3 standard. Like the HEVC, the AVS3 standard is built upon the block-based hybrid video coding framework.

[0051] FIG. 1 is a block diagram illustrating an exemplary system 10 for encoding and decoding video blocks in parallel in accordance with some implementations of the present disclosure. As shown in FIG. 1, system 10 includes a source device 12 that generates and encodes video data to be decoded at a later time by a destination device 14. Source device 12 and destination device 14 may comprise any of a wide variety of electronic devices, including desktop or laptop computers, tablet computers, smart phones, set-top boxes, digital televisions, cameras, display devices, digital media players, video gaming consoles, video streaming

device, or the like. In some implementations, source device 12 and destination device 14 are equipped with wireless communication capabilities.

[0052] In some implementations, destination device 14 may receive the encoded video data to be decoded via a link 16. Link 16 may comprise any type of communication medium or device capable of moving the encoded video data from source device 12 to destination device 14. In one example, link 16 may comprise a communication medium to enable source device 12 to transmit the encoded video data directly to destination device 14 in real-time. The encoded video data may be modulated according to a communication standard, such as a wireless communication protocol, and transmitted to destination device 14. The communication medium may comprise any wireless or wired communication medium, such as a radio frequency (RF) spectrum or one or more physical transmission lines. The communication medium may form part of a packet-based network, such as a local area network, a wide-area network, or a global network such as the Internet. The communication medium may include routers, switches, base stations, or any other equipment that may be useful to facilitate communication from source device 12 to destination device 14.

[0053] In some other implementations, the encoded video data may be transmitted from output interface 22 to a storage device 32. Subsequently, the encoded video data in storage device 32 may be accessed by destination device 14 via input interface 28. Storage device 32 may include any of a variety of distributed or locally accessed data storage media such as a hard drive, Blu-ray discs, DVDs, CD-ROMs, flash memory, volatile or non-volatile memory, or any other suitable digital storage media for storing encoded video data. In a further example, storage device 32 may correspond to a file server or another intermediate storage device that may hold the encoded video data generated by source device 12. Destination device 14 may access the stored video data from storage device 32 via streaming or downloading. The file server may be any type of computer capable of storing encoded video data and transmitting the encoded video data to destination device 14. Exemplary file servers include a web server (e.g., for a website), an

[0054] FTP server, network attached storage (NAS) devices, or a local disk drive. Destination device 14 may access the encoded video data through any standard data connection, including a wireless channel (e.g., a Wi-Fi connection), a wired connection (e.g., DSL, cable modem, etc.), or a combination of both that is suitable for accessing encoded video data stored on a file server. The transmission of encoded video data from storage device 32 may be a streaming transmission, a download transmission, or a combination of both.

[0055] As shown in FIG. 1, source device 12 includes a video source 18, a video encoder 20 and an output interface 22. Video source 18 may include a source such as a video capture device, e.g., a video camera, a video archive containing previously captured video, a video feed interface to receive video from a video content provider, and/or a computer graphics system for generating computer graphics data as the source video, or a combination of such sources. As one example, if video source 18 is a video camera of a security surveillance system, source device 12 and destination device 14 may form camera phones or video phones. However, the implementations described in the present

application may be applicable to video coding in general, and may be applied to wireless and/or wired applications.

[0056] The captured, pre-captured, or computer-generated video may be encoded by video encoder 20. The encoded video data may be transmitted directly to destination device 14 via output interface 22 of source device 12. The encoded video data may also (or alternatively) be stored onto storage device 32 for later access by destination device 14 or other devices, for decoding and/or playback. Output interface 22 may further include a modem and/or a transmitter.

[0057] Destination device 14 includes an input interface 28, a video decoder 30, and a display device 34. Input interface 28 may include a receiver and/or a modem and receive the encoded video data over link 16. The encoded video data communicated over link 16, or provided on storage device 32, may include a variety of syntax elements generated by video encoder 20 for use by video decoder 30 in decoding the video data. Such syntax elements may be included within the encoded video data transmitted on a communication medium, stored on a storage medium, or stored a file server.

[0058] In some implementations, destination device 14 may include a display device 34, which can be an integrated display device and an external display device that is configured to communicate with destination device 14. Display device 34 displays the decoded video data to a user, and may comprise any of a variety of display devices such as a liquid crystal display (LCD), a plasma display, an organic light emitting diode (OLED) display, or another type of display device.

[0059] Video encoder 20 and video decoder 30 may operate according to proprietary or industry standards, such as VVC, HEVC, MPEG-4, Part 10, Advanced Video Coding (AVC), AVS, or extensions of such standards. It should be understood that the present application is not limited to a specific video coding/decoding standard and may be applicable to other video coding/decoding standards. It is generally contemplated that video encoder 20 of source device 12 may be configured to encode video data according to any of these current or future standards. Similarly, it is also generally contemplated that video decoder 30 of destination device 14 may be configured to decode video data according to any of these current or future standards.

[0060] Video encoder 20 and video decoder 30 each may be implemented as any of a variety of suitable encoder circuitry, such as one or more microprocessors, digital signal processors (DSPs), application specific integrated circuits (ASICs), field programmable gate arrays (FPGAs), discrete logic, software, hardware, firmware or any combinations thereof. When implemented partially in software, an electronic device may store instructions for the software in a suitable, non-transitory computer-readable medium and execute the instructions in hardware using one or more processors to perform the video coding/decoding operations disclosed in the present disclosure. Each of video encoder 20 and video decoder 30 may be included in one or more encoders or decoders, either of which may be integrated as part of a combined encoder/decoder (CODEC) in a respective device.

[0061] FIG. 2 is a block diagram illustrating an exemplary video encoder 20 in accordance with some implementations described in the present application. Video encoder 20 may perform intra and inter predictive coding of video blocks within video frames. Intra predictive coding relies on spatial

prediction to reduce or remove spatial redundancy in video data within a given video frame or picture. Inter predictive coding relies on temporal prediction to reduce or remove temporal redundancy in video data within adjacent video frames or pictures of a video sequence.

[0062] As shown in FIG. 2, video encoder 20 includes video data memory 40, prediction processing unit 41, decoded picture buffer (DPB) 64, summer 50, transform processing unit 52, quantization unit 54, and entropy encoding unit 56. Prediction processing unit 41 further includes motion estimation unit 42, motion compensation unit 44, partition unit 45, intra prediction processing unit 46, and intra block copy (BC) unit 48. In some implementations, video encoder 20 also includes inverse quantization unit 58, inverse transform processing unit 60, and summer 62 for video block reconstruction. An in-loop filter 63, such as a deblocking filter may be positioned between summer 62 and DPB 64 to filter block boundaries to remove blockiness artifacts from reconstructed video. Another in-loop filter 63 may also be used in addition to the deblocking filter to filter the output of summer 62. Further in-loop filtering 63, such as sample adaptive offset (SAO) and adaptive in-loop filter (ALF) may be applied on the reconstructed CU before it is put in the reference picture store and used as reference to code future video blocks. Video encoder 20 may take the form of a fixed or programmable hardware unit or may be divided among one or more of the illustrated fixed or programmable hardware units.

[0063] Video data memory 40 may store video data to be encoded by the components of video encoder 20. The video data in video data memory 40 may be obtained, for example, from video source 18. DPB 64 is a buffer that stores reference video data for use in encoding video data by video encoder 20 (e.g., in intra or inter predictive coding modes). Video data memory 40 and DPB 64 may be formed by any of a variety of memory devices. In various examples, video data memory 40 may be on-chip with other components of video encoder 20, or off-chip relative to those components.

[0064] As shown in FIG. 2, after receiving video data, partition unit 45 within prediction processing unit 41 partitions the video data into video blocks. This partitioning may also include partitioning a video frame into slices, tiles, or other larger coding units (CUs) according to a predefined splitting structures such as quad-tree structure associated with the video data. The video frame may be divided into multiple video blocks (or sets of video blocks referred to as tiles). Prediction processing unit 41 may select one of a plurality of possible predictive coding modes, such as one of a plurality of intra predictive coding modes or one of a plurality of inter predictive coding modes, for the current video block based on error results (e.g., coding rate and the level of distortion). Prediction processing unit 41 may provide the resulting intra or inter prediction coded block to summer 50 to generate a residual block and to summer 62 to reconstruct the encoded block for use as part of a reference frame subsequently. Prediction processing unit 41 also provides syntax elements, such as motion vectors, intra-mode indicators, partition information, and other such syntax information, to entropy encoding unit 56.

[0065] In order to select an appropriate intra predictive coding mode for the current video block, intra prediction processing unit 46 within prediction processing unit 41 may perform intra predictive coding of the current video block relative to one or more neighboring blocks in the same frame

as the current block to be coded to provide spatial prediction. Motion estimation unit 42 and motion compensation unit 44 within prediction processing unit 41 perform inter predictive coding of the current video block relative to one or more predictive blocks in one or more reference frames to provide temporal prediction. Video encoder 20 may perform multiple coding passes, e.g., to select an appropriate coding mode for each block of video data.

[0066] In some implementations, motion estimation unit 42 determines the inter prediction mode for a current video frame by generating a motion vector, which indicates the displacement of a prediction unit (PU) of a video block within the current video frame relative to a predictive block within a reference video frame, according to a predetermined pattern within a sequence of video frames. Motion estimation, performed by motion estimation unit 42, is the process of generating motion vectors, which estimate motion for video blocks. A motion vector, for example, may indicate the displacement of a PU of a video block within a current video frame or picture relative to a predictive block within a reference frame (or other coded unit) relative to the current block being coded within the current frame (or other coded unit). The predetermined pattern may designate video frames in the sequence as P frames or B frames. Intra BC unit 48 may determine vectors, e.g., block vectors, for intra BC coding in a manner similar to the determination of motion vectors by motion estimation unit 42 for inter prediction, or may utilize motion estimation unit 42 to determine the block vector.

[0067] A predictive block is a block of a reference frame that is deemed as closely matching the PU of the video block to be coded in terms of pixel difference, which may be determined by sum of absolute difference (SAD), sum of square difference (SSD), or other difference metrics. In some implementations, video encoder 20 may calculate values for sub-integer pixel positions of reference frames stored in DPB 64. For example, video encoder 20 may interpolate values of one-quarter pixel positions, one-eighth pixel positions, or other fractional pixel positions of the reference frame. Therefore, motion estimation unit 42 may perform a motion search relative to the full pixel positions and fractional pixel positions and output a motion vector with fractional pixel precision.

[0068] Motion estimation unit 42 calculates a motion vector for a PU of a video block in an inter prediction coded frame by comparing the position of the PU to the position of a predictive block of a reference frame selected from a first reference frame list (List 0) or a second reference frame list (List 1), each of which identifies one or more reference frames stored in DPB 64. Motion estimation unit 42 sends the calculated motion vector to motion compensation unit 44 and then to entropy encoding unit 56.

[0069] Motion compensation, performed by motion compensation unit 44, may involve fetching or generating the predictive block based on the motion vector determined by motion estimation unit 42. Upon receiving the motion vector for the PU of the current video block, motion compensation unit 44 may locate a predictive block to which the motion vector points in one of the reference frame lists, retrieve the predictive block from DPB 64, and forward the predictive block to summer 50. Summer 50 then forms a residual video block of pixel difference values by subtracting pixel values of the predictive block provided by motion compensation unit 44 from the pixel values of the current video block

being coded. The pixel difference values forming the residual video block may include luma or chroma difference components or both. Motion compensation unit 44 may also generate syntax elements associated with the video blocks of a video frame for use by video decoder 30 in decoding the video blocks of the video frame. The syntax elements may include, for example, syntax elements defining the motion vector used to identify the predictive block, any flags indicating the prediction mode, or any other syntax information described herein. Note that motion estimation unit 42 and motion compensation unit 44 may be highly integrated, but are illustrated separately for conceptual purposes.

[0070] In some implementations, intra BC unit 48 may generate vectors and fetch predictive blocks in a manner similar to that described above in connection with motion estimation unit 42 and motion compensation unit 44, but with the predictive blocks being in the same frame as the current block being coded and with the vectors being referred to as block vectors as opposed to motion vectors. In particular, intra BC unit 48 may determine an intra-prediction mode to use to encode a current block. In some examples, intra BC unit 48 may encode a current block using various intra-prediction modes, e.g., during separate encoding passes, and test their performance through rate-distortion analysis. Next, intra BC unit 48 may select, among the various tested intra-prediction modes, an appropriate intra-prediction mode to use and generate an intra-mode indicator accordingly. For example, intra BC unit 48 may calculate rate-distortion values using a rate-distortion analysis for the various tested intra-prediction modes, and select the intra-prediction mode having the best rate-distortion characteristics among the tested modes as the appropriate intra-prediction mode to use. Rate-distortion analysis generally determines an amount of distortion (or error) between an encoded block and an original, unencoded block that was encoded to produce the encoded block, as well as a bitrate (i.e., a number of bits) used to produce the encoded block. Intra BC unit 48 may calculate ratios from the distortions and rates for the various encoded blocks to determine which intra-prediction mode exhibits the best rate-distortion value for the block.

[0071] In other examples, intra BC unit 48 may use motion estimation unit 42 and motion compensation unit 44, in whole or in part, to perform such functions for Intra BC prediction according to the implementations described herein. In either case, for Intra block copy, a predictive block may be a block that is deemed as closely matching the block to be coded, in terms of pixel difference, which may be determined by sum of absolute difference (SAD), sum of squared difference (SSD), or other difference metrics, and identification of the predictive block may include calculation of values for sub-integer pixel positions.

[0072] Whether the predictive block is from the same frame according to intra prediction, or a different frame according to inter prediction, video encoder 20 may form a residual video block by subtracting pixel values of the predictive block from the pixel values of the current video block being coded, forming pixel difference values. The pixel difference values forming the residual video block may include both luma and chroma component differences.

[0073] Intra prediction processing unit 46 may intra-predict a current video block, as an alternative to the inter-prediction performed by motion estimation unit 42 and motion compensation unit 44, or the intra block copy prediction performed by intra BC unit 48, as described above. In particular, intra prediction processing unit 46 may determine an intra prediction mode to use to encode a current block. To do so, intra prediction processing unit 46 may encode a current block using various intra prediction modes, e.g., during separate encoding passes, and intra prediction processing unit 46 (or a mode select unit, in some examples) may select an appropriate intra prediction mode to use from the tested intra prediction modes. Intra prediction processing unit 46 may provide information indicative of the selected intra-prediction mode for the block to entropy encoding unit 56. Entropy encoding unit 56 may encode the information indicating the selected intra-prediction mode in the bitstream.

[0074] After prediction processing unit 41 determines the predictive block for the current video block via either inter prediction or intra prediction, summer 50 forms a residual video block by subtracting the predictive block from the current video block. The residual video data in the residual block may be included in one or more transform units (TUs) and is provided to transform processing unit 52. Transform processing unit 52 transforms the residual video data into residual transform coefficients using a transform, such as a discrete cosine transform (DCT) or a conceptually similar transform.

[0075] Transform processing unit 52 may send the resulting transform coefficients to quantization unit 54. Quantization unit 54 quantizes the transform coefficients to further reduce bit rate. The quantization process may also reduce the bit depth associated with some or all of the coefficients. The degree of quantization may be modified by adjusting a quantization parameter. In some examples, quantization unit 54 may then perform a scan of a matrix including the quantized transform coefficients. Alternatively, entropy encoding unit 56 may perform the scan.

[0076] Following quantization, entropy encoding unit 56 entropy encodes the quantized transform coefficients into a video bitstream using, e.g., context adaptive variable length coding (CAVLC), context adaptive binary arithmetic coding (CABAC), syntax-based context-adaptive binary arithmetic coding (SBAC), probability interval partitioning entropy (PIPE) coding or another entropy encoding methodology or technique. The encoded bitstream may then be transmitted to video decoder 30, or archived in storage device 32 for later transmission to or retrieval by video decoder 30. Entropy encoding unit 56 may also entropy encode the motion vectors and the other syntax elements for the current video frame being coded.

[0077] Inverse quantization unit 58 and inverse transform processing unit 60 apply inverse quantization and inverse transformation, respectively, to reconstruct the residual video block in the pixel domain for generating a reference block for prediction of other video blocks. As noted above, motion compensation unit 44 may generate a motion compensated predictive block from one or more reference blocks of the frames stored in DPB 64. Motion compensation unit 44 may also apply one or more interpolation filters to the predictive block to calculate sub-integer pixel values for use in motion estimation.

[0078] Summer 62 adds the reconstructed residual block to the motion compensated predictive block produced by motion compensation unit 44 to produce a reference block for storage in DPB 64. The reference block may then be used by intra BC unit 48, motion estimation unit 42 and motion compensation unit 44 as a predictive block to inter predict another video block in a subsequent video frame.

[0079] FIG. 3 is a block diagram illustrating an exemplary video decoder 30 in accordance with some implementations of the present application. Video decoder 30 includes video data memory 79, entropy decoding unit 80, prediction processing unit 81, inverse quantization unit 86, inverse transform processing unit 88, summer 90, and DPB 92. Prediction processing unit 81 further includes motion compensation unit 82, intra prediction processing unit 84, and intra BC unit 85. Video decoder 30 may perform a decoding process generally reciprocal to the encoding process described above with respect to video encoder 20 in connection with FIG. 2. For example, motion compensation unit 82 may generate prediction data based on motion vectors received from entropy decoding unit 80, while intra-prediction unit 84 may generate prediction data based on intra-prediction mode indicators received from entropy decoding unit 80.

[0080] In some examples, a unit of video decoder 30 may be tasked to perform the implementations of the present application. Also, in some examples, the implementations of the present disclosure may be divided among one or more of the units of video decoder 30. For example, intra BC unit 85 may perform the implementations of the present application, alone, or in combination with other units of video decoder 30, such as motion compensation unit 82, intra prediction processing unit 84, and entropy decoding unit 80. In some examples, video decoder 30 may not include intra BC unit 85 and the functionality of intra BC unit 85 may be performed by other components of prediction processing unit 81, such as motion compensation unit 82.

[0081] Video data memory 79 may store video data, such as an encoded video bitstream, to be decoded by the other components of video decoder 30. The video data stored in video data memory 79 may be obtained, for example, from storage device 32, from a local video source, such as a camera, via wired or wireless network communication of video data, or by accessing physical data storage media (e.g., a flash drive or hard disk). Video data memory 79 may include a coded picture buffer (CPB) that stores encoded video data from an encoded video bitstream. Decoded picture buffer (DPB) 92 of video decoder 30 stores reference video data for use in decoding video data by video decoder 30 (e.g., in intra or inter predictive coding modes). Video data memory 79 and DPB 92 may be formed by any of a variety of memory devices, such as dynamic random access memory (DRAM), including synchronous DRAM (SDRAM), magneto-resistive RAM (MRAM), resistive RAM (RRAM), or other types of memory devices. For illustrative purpose, video data memory 79 and DPB 92 are depicted as two distinct components of video decoder 30 in FIG. 3. But it will be apparent to one skilled in the art that video data memory 79 and DPB 92 may be provided by the same memory device or separate memory devices. In some examples, video data memory 79 may be on-chip with other components of video decoder 30, or off-chip relative to those components.

[0082] During the decoding process, video decoder 30 receives an encoded video bitstream that represents video blocks of an encoded video frame and associated syntax elements. Video decoder 30 may receive the syntax elements at the video frame level and/or the video block level. Entropy decoding unit 80 of video decoder 30 entropy decodes the bitstream to generate quantized coefficients, motion vectors or intra-prediction mode indicators, and other syntax elements. Entropy decoding unit 80 then forwards the motion vectors and other syntax elements to prediction processing unit 81.

[0083] When the video frame is coded as an intra predictive coded (I) frame or for intra coded predictive blocks in other types of frames, intra prediction processing unit 84 of prediction processing unit 81 may generate prediction data for a video block of the current video frame based on a signaled intra prediction mode and reference data from previously decoded blocks of the current frame.

[0084] When the video frame is coded as an inter-predictive coded (i.e., B or P) frame, motion compensation unit 82 of prediction processing unit 81 produces one or more predictive blocks for a video block of the current video frame based on the motion vectors and other syntax elements received from entropy decoding unit 80. Each of the predictive blocks may be produced from a reference frame within one of the reference frame lists. Video decoder 30 may construct the reference frame lists, List 0 and List 1, using default construction techniques based on reference frames stored in DPB 92.

[0085] In some examples, when the video block is coded according to the intra BC mode described herein, intra BC unit 85 of prediction processing unit 81 produces predictive blocks for the current video block based on block vectors and other syntax elements received from entropy decoding unit 80. The predictive blocks may be within a reconstructed region of the same picture as the current video block defined by video encoder 20.

[0086] Motion compensation unit 82 and/or intra BC unit 85 determines prediction information for a video block of the current video frame by parsing the motion vectors and other syntax elements, and then uses the prediction information to produce the predictive blocks for the current video block being decoded. For example, motion compensation unit 82 uses some of the received syntax elements to determine a prediction mode (e.g., intra or inter prediction) used to code video blocks of the video frame, an inter prediction frame type (e.g., B or P), construction information for one or more of the reference frame lists for the frame, motion vectors for each inter predictive encoded video block of the frame, inter prediction status for each inter predictive coded video block of the frame, and other information to decode the video blocks in the current video frame.

[0087] Similarly, intra BC unit 85 may use some of the received syntax elements, e.g., a flag, to determine that the current video block was predicted using the intra BC mode, construction information of which video blocks of the frame are within the reconstructed region and should be stored in DPB 92, block vectors for each intra BC predicted video block of the frame, intra BC prediction status for each intra BC predicted video block of the frame, and other information to decode the video blocks in the current video frame.

[0088] Motion compensation unit **82** may also perform interpolation using the interpolation filters as used by video encoder **20** during encoding of the video blocks to calculate interpolated values for sub-integer pixels of reference blocks. In this case, motion compensation unit **82** may determine the interpolation filters used by video encoder **20** from the received syntax elements and use the interpolation filters to produce predictive blocks.

[0089] Inverse quantization unit **86** inverse quantizes the quantized transform coefficients provided in the bitstream and entropy decoded by entropy decoding unit **80** using the same quantization parameter calculated by video encoder **20** for each video block in the video frame to determine a degree of quantization. Inverse transform processing unit **88** applies an inverse transform, e.g., an inverse DCT, an inverse integer transform, or a conceptually similar inverse transform process, to the transform coefficients in order to reconstruct the residual blocks in the pixel domain.

[0090] After motion compensation unit **82** or intra BC unit **85** generates the predictive block for the current video block based on the vectors and other syntax elements, summer **90** reconstructs decoded video block for the current video block by summing the residual block from inverse transform processing unit **88** and a corresponding predictive block generated by motion compensation unit **82** and intra BC unit **85**. An in-loop filter **91** may be positioned between summer **90** and DPB **92** to further process the decoded video block. The in-loop filtering **91**, such as deblocking filter, sample adaptive offset (SAO) and adaptive in-loop filter (ALF) may be applied on the reconstructed CU before it is put in the reference picture store. The decoded video blocks in a given frame are then stored in DPB **92**, which stores reference frames used for subsequent motion compensation of next video blocks. DPB **92**, or a memory device separate from DPB **92**, may also store decoded video for later presentation on a display device, such as display device **34** of FIG. 1.

[0091] In a typical video coding process, a video sequence typically includes an ordered set of frames or pictures. Each frame may include three sample arrays, denoted SL, SCb, and SCr. SL is a two-dimensional array of luma samples. SCb is a two-dimensional array of Cb chroma samples. SCr is a two-dimensional array of Cr chroma samples. In other instances, a frame may be monochrome and therefore includes only one two-dimensional array of luma samples.

[0092] Like the HEVC, the AVS3 standard is built upon the block-based hybrid video coding framework. The input video signal is processed block by block (called coding units (CUs)). Different from the HEVC which partitions blocks only based on quad-trees, in the AVS3, one coding tree unit (CTU) is split into CUs to adapt to varying local characteristics based on quad/binary/extended-quad-tree. Additionally, the concept of multiple partition unit type in the HEVC is removed, i.e., the separation of CU, prediction unit (PU) and transform unit (TU) does not exist in the AVS3. Instead, each CU is always used as the basic unit for both prediction and transform without further partitions. In the tree partition structure of the AVS3, one CTU is firstly partitioned based on a quad-tree structure. Then, each quad-tree leaf node can be further partitioned based on a binary and extended-quad-tree structure.

[0093] As shown in FIG. 4A, video encoder **20** (or more specifically partition unit **45**) generates an encoded representation of a frame by first partitioning the frame into a set of coding tree units (CTUs). A video frame may include an

integer number of CTUs ordered consecutively in a raster scan order from left to right and from top to bottom. Each CTU is a largest logical coding unit and the width and height of the CTU are signaled by the video encoder **20** in a sequence parameter set, such that all the CTUs in a video sequence have the same size being one of 128×128, 64×64, 32×32, and 16×16. But it should be noted that the present application is not necessarily limited to a particular size. As shown in FIG. 4B, each CTU may comprise one coding tree block (CTB) of luma samples, two corresponding coding tree blocks of chroma samples, and syntax elements used to code the samples of the coding tree blocks. The syntax elements describe properties of different types of units of a coded block of pixels and how the video sequence can be reconstructed at the video decoder **30**, including inter or intra prediction, intra prediction mode, motion vectors, and other parameters. In monochrome pictures or pictures having three separate color planes, a CTU may comprise a single coding tree block and syntax elements used to code the samples of the coding tree block. A coding tree block may be an N×N block of samples.

[0094] To achieve a better performance, video encoder **20** may recursively perform tree partitioning such as binary-tree partitioning, ternary-tree partitioning, quad-tree partitioning or a combination of both on the coding tree blocks of the CTU and divide the CTU into smaller coding units (CUs). As depicted in FIG. 4C, the 64×64 CTU **400** is first divided into four smaller CU, each having a block size of 32×32. Among the four smaller CUs, CU **410** and CU **420** are each divided into four CUs of 16×16 by block size. The two 16×16 CUs **430** and **440** are each further divided into four CUs of 8×8 by block size. FIG. 4D depicts a quad-tree data structure illustrating the end result of the partition process of the CTU **400** as depicted in FIG. 4C, each leaf node of the quad-tree corresponding to one CU of a respective size ranging from 32×32 to 8×8. Like the CTU depicted in FIG. 4B, each CU may comprise a coding block (CB) of luma samples and two corresponding coding blocks of chroma samples of a frame of the same size, and syntax elements used to code the samples of the coding blocks. In monochrome pictures or pictures having three separate color planes, a CU may comprise a single coding block and syntax structures used to code the samples of the coding block. It should be noted that the quad-tree partitioning depicted in FIGS. 4C and 4D is only for illustrative purposes and one CTU can be split into CUs to adapt to varying local characteristics based on quad/ternary/binary-tree partitions. In the multi-type tree structure, one CTU is partitioned by a quad-tree structure and each quad-tree leaf CU can be further partitioned by a binary and ternary tree structure. As shown in FIG. 4E, there are five splitting/partitioning types in the AVS3, i.e., quaternary partitioning, horizontal binary partitioning, vertical binary partitioning, horizontal extended quad-tree partitioning, and vertical extended quad-tree partitioning.

[0095] In some implementations, video encoder **20** may further partition a coding block of a CU into one or more M×N prediction blocks (PB). A prediction block is a rectangular (square or non-square) block of samples on which the same prediction, inter or intra, is applied. A prediction unit (PU) of a CU may comprise a prediction block of luma samples, two corresponding prediction blocks of chroma samples, and syntax elements used to predict the prediction blocks. In monochrome pictures or pictures having three

separate color planes, a PU may comprise a single prediction block and syntax structures used to predict the prediction block. Video encoder **20** may generate predictive luma, Cb, and Cr blocks for luma, Cb, and Cr prediction blocks of each PU of the CU.

[0096] Video encoder **20** may use intra prediction or inter prediction to generate the predictive blocks for a PU. If video encoder **20** uses intra prediction to generate the predictive blocks of a PU, video encoder **20** may generate the predictive blocks of the PU based on decoded samples of the frame associated with the PU. If video encoder **20** uses inter prediction to generate the predictive blocks of a PU, video encoder **20** may generate the predictive blocks of the PU based on decoded samples of one or more frames other than the frame associated with the PU.

[0097] After video encoder **20** generates predictive luma, Cb, and Cr blocks for one or more PUs of a CU, video encoder **20** may generate a luma residual block for the CU by subtracting the CU's predictive luma blocks from its original luma coding block such that each sample in the CU's luma residual block indicates a difference between a luma sample in one of the CU's predictive luma blocks and a corresponding sample in the CU's original luma coding block. Similarly, video encoder **20** may generate a Cb residual block and a Cr residual block for the CU, respectively, such that each sample in the CU's Cb residual block indicates a difference between a Cb sample in one of the CU's predictive Cb blocks and a corresponding sample in the CU's original Cb coding block and each sample in the CU's Cr residual block may indicate a difference between a Cr sample in one of the CU's predictive Cr blocks and a corresponding sample in the CU's original Cr coding block.

[0098] Furthermore, as illustrated in FIG. 4C, video encoder **20** may use quad-tree partitioning to decompose the luma, Cb, and Cr residual blocks of a CU into one or more luma, Cb, and Cr transform blocks. A transform block is a rectangular (square or non-square) block of samples on which the same transform is applied. A transform unit (TU) of a CU may comprise a transform block of luma samples, two corresponding transform blocks of chroma samples, and syntax elements used to transform the transform block samples. Thus, each TU of a CU may be associated with a luma transform block, a Cb transform block, and a Cr transform block. In some examples, the luma transform block associated with the TU may be a sub-block of the CU's luma residual block. The Cb transform block may be a sub-block of the CU's Cb residual block. The Cr transform block may be a sub-block of the CU's Cr residual block. In monochrome pictures or pictures having three separate color planes, a TU may comprise a single transform block and syntax structures used to transform the samples of the transform block.

[0099] Video encoder **20** may apply one or more transforms to a luma transform block of a TU to generate a luma coefficient block for the TU. A coefficient block may be a two-dimensional array of transform coefficients. A transform coefficient may be a scalar quantity. Video encoder **20** may apply one or more transforms to a Cb transform block of a TU to generate a Cb coefficient block for the TU. Video encoder **20** may apply one or more transforms to a Cr transform block of a TU to generate a Cr coefficient block for the TU.

[0100] After generating a coefficient block (e.g., a luma coefficient block, a Cb coefficient block or a Cr coefficient block), video encoder **20** may quantize the coefficient block. Quantization generally refers to a process in which transform coefficients are quantized to possibly reduce the amount of data used to represent the transform coefficients, providing further compression. After video encoder **20** quantizes a coefficient block, video encoder **20** may entropy encode syntax elements indicating the quantized transform coefficients. For example, video encoder **20** may perform Context-Adaptive Binary Arithmetic Coding (CABAC) on the syntax elements indicating the quantized transform coefficients. Finally, video encoder **20** may output a bitstream that includes a sequence of bits that forms a representation of coded frames and associated data, which is either saved in storage device **32** or transmitted to destination device **14**.

[0101] After receiving a bitstream generated by video encoder **20**, video decoder **30** may parse the bitstream to obtain syntax elements from the bitstream. Video decoder **30** may reconstruct the frames of the video data based at least in part on the syntax elements obtained from the bitstream. The process of reconstructing the video data is generally reciprocal to the encoding process performed by video encoder **20**. For example, video decoder **30** may perform inverse transforms on the coefficient blocks associated with TUs of a current CU to reconstruct residual blocks associated with the TUs of the current CU. Video decoder **30** also reconstructs the coding blocks of the current CU by adding the samples of the predictive blocks for PUs of the current CU to corresponding samples of the transform blocks of the TUs of the current CU. After reconstructing the coding blocks for each CU of a frame, video decoder **30** may reconstruct the frame.

[0102] SAO is a process that modifies the decoded samples by conditionally adding an offset value to each sample after the application of the deblocking filter, based on values in look-up tables transmitted by the encoder. SAO filtering is performed on a region basis, based on a filtering type selected per CTB by a syntax element sao-type-idx. A value of 0 for sao-type-idx indicates that the SAO filter is not applied to the CTB, and the values 1 and 2 signal the use of the band offset and edge offset filtering types, respectively. In the band offset mode specified by sao-type-idx equal to 1, the selected offset value directly depends on the sample amplitude. In this mode, the full sample amplitude range is uniformly split into 32 segments called bands, and the sample values belonging to four of these bands (which are consecutive within the 32 bands) are modified by adding transmitted values denoted as band offsets, which can be positive or negative. The main reason for using four consecutive bands is that in the smooth areas where banding artifacts can appear, the sample amplitudes in a CTB tend to be concentrated in only few of the bands. In addition, the design choice of using four offsets is unified with the edge offset mode of operation which also uses four offset values. In the edge offset mode specified by sao-type-idx equal to 2, a syntax element sao-eo-class with values from 0 to 3 signals whether a horizontal, vertical or one of two diagonal gradient directions is used for the edge offset classification in the CTB.

[0103] FIG. 5 is a block diagram depicting the four gradient patterns used in SAO in accordance with some implementations of the present disclosure. The four gradient patterns **502**, **504**, **506**, and **508** are for the respective

sao-eo-class in the edge offset mode. Sample labelled “p” indicates a center sample to be considered. Two samples labeled “n0” and “n1” specify two neighboring samples along the (a) horizontal (sao-eo-class=0), (b) vertical (sao-eo-class=1), (c) 135° diagonal (sao-eo-class=2), and (d) 45° (sao-eo-class=3) gradient patterns. Each sample in the CTB is classified into one of five EdgIdx categories by comparing the sample value p located at some position with the values n0 and n1 of two samples located at neighboring positions as shown in FIG. 5. This classification is done for each sample based on decoded sample values, so no additional signaling is required for the EdgIdx classification. Depending on the EdgIdx category at the sample position, for EdgIdx categories from 1 to 4, an offset value from a transmitted look-up table is added to the sample value. The offset values are always positive for categories 1 and 2 and negative for categories 3 and 4. Thus the filter generally has a smoothing effect in the edge offset mode. Table 1 below illustrates a sample EdgIdx categories in SAO edge classes.

TABLE 1

A sample EdgIdx categories in SAO edge classes.		
EdgIdx	Condition	Meaning
0	Cases not listed below	Monotonic area
1	$p < n_0$ and $p < n_1$	Local min
2	$p < n_0$ and $p = n_1$ or $p < n_1$ and $p = n_0$	Edge
3	$p > n_0$ and $p = n_1$ or $p > n_1$ and $p = n_0$	Edge
4	$p > n_0$ and $p > n_1$	Local max

[0104] For SAO types 1 and 2, a total of four amplitude offset values are transmitted to the decoder for each CTB. For type 1, the sign is also encoded. The offset values and related syntax elements such as sao-type-idx and sao-eo-class are determined by the encoder—typically using criteria that optimize rate-distortion performance. The SAO parameters can be indicated to be inherited from the left or above CTB using a merge flag to make the signaling efficient. In summary, SAO is a nonlinear filtering operation which allows additional refinement of the reconstructed signal, and it can enhance the signal representation in both smooth areas and around edges.

[0105] In some embodiments, methods and systems are disclosed herein to improve the coding efficiency or reduce the complexity of Sample Adaptive Offset (SAO) by introducing cross-component information. SAO is used in the HEVC, VVC, AVS2 and AVS3 standards. Although the existing SAO design in the HEVC, VVC, AVS2 and AVS3 standards is used as the basic SAO method in the following descriptions, to a person skilled in the art of video coding, the cross-component methods described in the disclosure can also be applied to other loop filter designs or other coding tools with the similar design spirits. For example, in the AVS3 standard, SAO is replaced by a coding tool called Enhanced Sample Adaptive Offset (ESAO). However, the CCSAO disclosed herein can also be applied in parallel with ESAO. In another example, CCSAO can be applied in parallel with Constrained Directional Enhancement Filter (CDEF) in the AV1 standard.

[0106] For the existing SAO design in the HEVC, VVC, AVS2 and AVS3 standards, the luma Y, chroma Cb and chroma Cr sample offset values are decided independently. That is, for example, the current chroma sample offset is

decided by only the current and neighboring chroma sample values, without taking collocated or neighboring luma samples into consideration. However, luma samples preserve more original picture detail information than chroma samples, and they can benefit the decision of the current chroma sample offset. Furthermore, since chroma samples usually lose high frequency details after color conversion from RGB to YCbCr, or after quantization and deblocking filter, introducing luma samples with high frequency detail preserved for chroma offset decision can benefit the chroma sample reconstruction. Hence, further gain can be expected by exploring cross-component correlation, for example, by using the methods and systems of Cross-Component Sample Adaptive Offset (CCSAO).

[0107] FIG. 6A is a block diagram illustrating the system and process of CCSAO according to some implementations of the present disclosure. The luma samples after luma deblocking filter (DBF Y) is used to determine additional offsets for chroma Cb and Cr after SAO Cb and SAO Cr. For example, the current chroma sample 602 is first classified using collocated 604 and neighboring (white) luma samples 606, and the corresponding CCSAO offset value of the corresponding class is added to the current chroma sample value.

[0108] In some embodiments, CCSAO can also be applied in parallel with other coding tools, for example, ESAO in the AVS standard, or CDEF in the AV1 standard. FIG. 6B is a block diagram illustrating the system and process of CCSAO applied in parallel with ESAO in the AVS standard according to some implementations of the present disclosure.

[0109] FIG. 6C is a block diagram illustrating the system and process of CCSAO applied after SAO according to some implementations of the present disclosure. In some embodiments, FIG. 6C shows that the location of CCSAO can be after SAO, i.e., the location of Cross-Component Adaptive Loop Filter (CCALF) in the VVC standard. In some embodiments, the SAO Y/Cb/Cr can be replaced by ESAO, for example, in the AVS3 standard.

[0110] FIG. 6D is a block diagram illustrating the system and process of CCSAO applied in parallel with CCALF according to some implementations of the present disclosure. In some embodiments, FIG. 6D shows that CCSAO can be applied in parallel with CCALF. In some embodiments, in FIG. 6D, the locations of CCALF and CCSAO can be switched. In some embodiments, in FIG. 6A to FIG. 6D, or throughout the present disclosure, the SAO Y/Cb/Cr blocks can be replaced by ESAO Y/Cb/Cr (in AVS3) or CDEF (in AV1).

[0111] In some embodiments, the current chroma sample classification is reusing the SAO type (EO or BO), class, and category of the collocated luma sample. The corresponding CCSAO offset can be signaled or derived from the decoder itself. For example, let h_Y be the collocated luma SAO offset, h_Cb and h_Cr be the CCSAO Cb and Cr offset, respectively. h_Cb (or h_Cr) = $w * h_Y$ where w can be selected in a limited table. For example, $+1/4$, $+1/2$, 0, -1 , $+2$, $+4$. . . etc., where $|w|$ only includes the power-of-2 values.

[0112] In some embodiments, the comparison score $[-8, 8]$ of the collocated luma samples (Y_0) and neighboring 8 luma samples are used, which yields 17 classes in total.

Initial Class = 0		
Loop over neighboring 8 luma samples (Y_i , $i=1$ to 8)		
if	$Y_0 > Y_i$	Class += 1
else if	$Y_0 < Y_i$	Class -= 1

[0113] In some embodiments, the abovementioned classification methods can be combined. For example, comparison score combined with SAO BO (32 bands classification) is used to increase diversity, which yields 17×32 classes in total. In some embodiments, the Cb and Cr can use the same class to reduce the complexity or saving bits.

[0114] FIG. 7 is a block diagram illustrating a sample process using CCSAO in accordance with some implementations of the present disclosure. Specifically, FIG. 7 shows the input of CCSAO can introduce the input of vertical and horizontal DBF, to simplify the class determination, or increase flexibility. For example, let $Y_0_DBF_V$, $Y_0_DBF_H$, and Y_0 be collocated luma samples at the input of DBF_V, DBF_H, and SAO, respectively. $Y_i_DBF_V$, $Y_i_DBF_H$, and Y_i are neighbouring 8 luma samples at the input of DBF_V, DBF_H, and SAO, respectively, where $i=1$ to 8.

$$\text{Max } Y_0 = \max(Y_0_DBF_V, Y_0_DBF_H, Y_0_DBF)$$

$$\text{Max } Y_i = \max(Y_i_DBF_V, Y_i_DBF_H, Y_i_DBF)$$

And feed max Y_0 and max Y_i to CCSAO classification.

[0115] FIG. 8 is a block diagram illustrating that CCSAO process is interleaved to vertical and horizontal DBF in accordance with some implementations of the present disclosure. In some embodiments, CCSAO blocks in FIGS. 6, 7 and 8 can be selective. For example, using $Y_0_DBF_V$ and $Y_i_DBF_V$ for the first CCSAO_V, which applies the same sample processing as in FIG. 6, while using the input of DBF_V luma samples as CCSAO input.

[0116] In some embodiments, CCSAO syntax implemented is shown in Table 2 below.

TABLE 2

An example of CCSAO syntax		
Level	Syntax element	Meaning
SPS	cc_sao_enabled_flag	whether CCSAO is enabled in the sequence
SH	slice_cc_sao_cb_flag	whether CCSAO is enabled for Cb or Cr
	slice_cc_sao_cr_flag	
CTU	cc_sao_merge_left_flag	whether CCSAO offset is merged from the left or up CTU
	cc_sao_merge_up_flag	
CTU	cc_sao_class_idx	CCSAO class index of this CTU
CTU	cc_sao_offset_sign_flag	CCSAO Cb and Cr offset values of this CTU class
	cc_sao_offset_abs	

[0117] In some embodiments, for signaling CCSAO Cb and Cr offset values, if one additional chroma offset is signaled, the other chroma component offset can be derived by plus or minus sign, or weighting to save bits overhead. For example, let h_Cb and h_Cr be the offset of CCSAO Cb

and Cr, respectively. With explicit signaling w , wherein $w = \pm |w|$ with limited $|w|$ candidates, h_Cr can be derived from h_Cb without explicit signaling h_Cr itself.

$$h_Cr = w * h_Cb$$

[0118] FIG. 9 is a flowchart illustrating an exemplary process 900 of decoding video signal using cross-component correlation in accordance with some implementations of the present disclosure.

[0119] The video decoder 30, receives the video signal that includes a first component and a second component (910). In some embodiments, the first component is a luma component, and the second component is a chroma component of the video signal.

[0120] The video decoder 30 also receives a plurality of offsets associated with the second component (920).

[0121] The video decoder 30 then utilizes a characteristic measurement of the first component to obtain a classification category associated with the second component (930). For example, in FIG. 6, the current chroma sample 602 is first classified using collocated 604 and neighboring (white) luma samples 606, and the corresponding CCSAO offset value is added to the current chroma sample.

[0122] The video decoder 30 further selects a first offset from the plurality of offsets for the second component according to the classification category (940).

[0123] The video decoder 30 additionally modifies the second component based on the selected first offset (950).

[0124] In some embodiments, utilizing the characteristic measurement of the first component to obtain the classification category associated with the second component (930) includes: utilizing a respective sample of the first component to obtain a respective classification category of a respective each sample of the second component, wherein the respective sample of the first component is a respective collocated sample of the first component to the respective each sample of the second component. For example, the current chroma sample classification is reusing the SAO type (EO or BO), class, and category of the collocated luma sample.

[0125] In some embodiments, utilizing the characteristic measurement of the first component to obtain the classification category associated with the second component (930) includes: utilizing a respective sample of the first component to obtain a respective classification category of a respective each sample of the second component, wherein the respective sample of the first component is reconstructed before being deblocked or is reconstructed after being deblocked. In some embodiment, the first component is being deblocked at a deblocking filter (DBF). In some embodiment, the first component is being deblocked at a luma deblocking filter (DBF Y). For example, alternative to FIG. 6 or 7, the CCSAO input can also be before DBF Y.

[0126] In some embodiments, the characteristic measurement is derived by dividing the range of sample values of the first component into several bands and selecting a band based on the intensity value of a sample in the first component. In some embodiments, the characteristic measurement is derived from Band Offset (BO).

[0127] In some embodiments, the characteristic measurement is derived based on the direction and strength of the edge information of a sample in the first component. In some embodiments, the characteristic measurement is derived from Edge Offset (EO).

[0128] In some embodiments, modifying the second component (950) comprises directly adding the selected first offset to the second component. For example, the corresponding CCSAO offset value is added to the current chroma component sample.

[0129] In some embodiments, modifying the second component (950) comprises mapping the selected first offset to a second offset and adding the mapped second offset to the second component. For example, for signaling CCSAO Cb and Cr offset values, if one additional chroma offset is signaled, the other chroma component offset can be derived by using a plus or minus sign, or weighting to save bits overhead.

[0130] In some embodiments, receiving the video signal (910) comprises receiving a syntax element that indicates whether the method of decoding video signal using CCSAO is enabled for the video signal in the Sequence Parameter Set (SPS). In some embodiments, cc_sao_enabled_flag indicates whether CCSAO is enabled in the sequence level.

[0131] In some embodiments, receiving the video signal (910) comprises receiving a syntax element that indicates whether the method of decoding video signal using CCSAO is enabled for the second component on the slice level. In some embodiments, slice_cc_sao_cb_flag or slice_cc_sao_cr_flag indicates whether CCSAO is enabled in the respective slice for Cb or Cr.

[0132] In some embodiments, receiving the plurality of offsets associated with the second component (920) comprises receiving different offsets for different Coding Tree Units (CTUs). In some embodiments, for a CTU, cc_sao_offset_sign_flag indicates a sign for an offset, and cc_sao_offset_abs indicates the CCSAO Cb and Cr offset values of the current CTU.

[0133] In some embodiments, receiving the plurality of offsets associated with the second component (920) comprises receiving a syntax element that indicates whether the received offsets of a CTU are the same as that of one of a neighboring CTU of the CTU, wherein the neighboring CTU is either a left or a top neighboring CTU. For example, cc_sao_merge_up_flag indicates whether CCSAO offset is merged from the left or up CTU.

[0134] In some embodiments, the video signal further includes a third component and the method of decoding the video signal using CCSAO further includes: receiving a second plurality of offsets associated with a third component; utilizing the characteristic measurement of the first component to obtain a second classification category associated with the third component; selecting a third offset from the second plurality of offsets for the third component according to the second classification category; and modifying the third component based on the selected third offset.

[0135] FIG. 11 is a block diagram of a sample process illustrating that besides luma, the other cross-component collocated (1102) and neighboring (white) chroma samples

are also fed into CCSAO classification in accordance with some implementations of the present disclosure. FIG. 6A, 6B and FIG. 11 show the input of CCSAO classification. In FIG. 11, current chroma sample is 1104, the cross-component collocated chroma sample is 1102, and the collocated luma sample is 1106.

[0136] In some embodiments, a classifier example (C0) uses the collocated luma sample value (Y0) for classification. Let band_num be the number of equally divided bands of luma dynamic range, and bit_depth be the sequence bit depth, the class index for the current chroma sample is:

$$\text{Class (C0)} = (Y0 * \text{band_num}) \gg \text{bit_depth}$$

[0137] In some embodiments, the classification takes rounding into account, for example:

$$\text{Class (C0)} = ((Y0 * \text{band_num}) + (1 \ll \text{bit_depth})) \gg \text{bit_depth}$$

[0138] Some band_num and bit_depth examples are listed below in Table 3. Table 3 shows three classification examples when the number of bands is different for each of the classification examples.

TABLE 3

Exemplary band_num and bit_depth for each class index.								
band_num		16	band_num		7	band_num		7
bit_depth		10	bit_depth		10	bit_depth		8
Class		Y0	Class		Y0	Class		Y0
0	0	63	0	0	145	0	0	36
1	64	127	1	146	292	1	37	72
2	128	191	2	293	438	2	73	109
3	192	255	3	439	584	3	110	145
4	256	319	4	585	730	4	146	182
5	320	383	5	731	877	5	183	218
6	384	447	6	878	1023	6	219	255
7	448	511						
8	512	575						
9	576	639						
10	640	703						
11	704	767						
12	768	831						
13	832	895						
14	896	959						
15	960	1023						

[0139] In some embodiments, a classifier uses different luma sample position for C0 classification. FIG. 10A is a block diagram showing a classifier using different luma sample position for C0 classification in accordance with some implementations of the present disclosure, for example, using the neighboring Y7 but not Y0 for C0 classification.

[0140] In some embodiments, different classifiers can be switched in Sequence Parameter Set (SPS)/Adaptation parameter set (APS)/Picture parameter set (PPS)/Picture header (PH)/Slice header (SH)/Coding tree unit (CTU)/Coding unit (CU) levels. For example, in FIG. 10, using Y0 for POC0 but using Y7 for POC1, as shown in Table 4 below.

TABLE 4

Different classifiers are applied to different pictures			
POC	Classifier	C0 band_num	Total classes
0	C0 using Y0 position	8	8
1	C0 using Y7 position	8	8

[0141] In some embodiments, FIG. 10B illustrates some examples of different shapes for luma candidates, in accordance with some implementations of the present disclosure. For example, a constraint can be applied to the shapes. In some instances, the total number of luma candidates must be the power of 2, as shown in FIG. 10B (b) (c) (d). In some instances, the number of luma candidates must be horizontal and vertical symmetric relative to the chroma sample (in the center), as shown in FIG. 10B (a) (c) (d) (e).

[0142] In some embodiments, the C0 position and C0 band_num can be combined and switched in SPS/APS/PPS/PH/SH/CTU/CU levels. Different combinations can be different classifiers as shown in Table 5 below.

TABLE 5

Different classifier and band number combinations are applied to different pictures			
POC	Classifier	C0 band_num	Total classes
0	C0 using Y0 position	16	16
1	C0 using Y7 position	8	8

[0143] In some embodiments, the collocated luma sample value (Y0) is replaced by a value (Yp) obtained by weighing collocated and neighboring luma samples. FIG. 12 illustrates exemplary classifiers by replacing the collocated luma sample value with a value obtained by weighing collocated and neighboring luma samples in accordance with some implementations of the present disclosure. The collocated luma sample value (Y0) can be replaced by a phase corrected value (Yp) obtained by weighing neighboring luma samples. Different Yp can be a different classifier.

[0144] In some embodiments, different Yp is applied on different chroma format. For example, in FIG. 12, the Yp of (a) is used for the 420 chroma format, the Yp of (b) is used for the 422 chroma format, and Y0 is used for the 444 chroma format.

[0145] In some embodiments, another classifier (C1) is the comparison score $[-8, 8]$ of the collocated luma samples (Y0) and neighboring 8 luma samples, which yields 17 classes in total as shown below.

Initial Class (C1) = 0, Loop over neighboring 8 luma samples (Yi, i=1 to 8)			
if	Y0 > Yi	Class += 1	
else if	Y0 < Yi	Class -= 1	

[0146] In some embodiments, a variation (C1') only counts comparison score $[0, 8]$, and this yields 8 classes.

(C1, C1') is a classifier group and a PH/SH level flag can be signaled to switch between C1 and C1'.

Initial Class (C1') = 0, Loop over neighboring 8 luma samples (Yi, i=1 to 8)		
if	Y0 >	Yi Class += 1

[0147] In some embodiments, different classifiers are combined to yield a general classifier. For example, for different pictures (different POC values), different classifiers are applied as shown in Table 6 below.

TABLE 6

Different general classifiers are applied to different pictures			
POC	Classifier	C0 band_num	Total classes
0	combine C0 and C1	16	16*17
1	combine C0 and C1'	16	16*9
2	combine C0 and C1	7	7*17

[0148] In some embodiments, a classifier example (C2) uses the difference (Yn) of collocated and neighboring luma samples. FIG. 12 (c) shows an example of Yn, which has a dynamic range of $[-1024, 1023]$ when bit depth is 10. Let C2 band_num be the number of equally divided bands of Yn dynamic range,

$$\text{Class (C2)} = (Yn + (1 \ll \text{bit_depth}) * \text{band_num}) \gg (\text{bit_depth} + 1).$$

[0149] In some embodiments, C0 and C2 are combined to yield a general classifier. For example, for different pictures (different POC), different classifiers are applied as shown in Table 7 below.

TABLE 7

Different general classifiers are applied to different pictures				
POC	Classifier	C0 band_num	C2 band_num	Total classes
0	combine C0 and C2	16	16	16*17
1	combine C0 and C2	8	7	8*7

[0150] In some embodiments, all above mentioned classifiers (C0, C1, C1', C2) are combined. For example, for different pictures (different POCs), different classifiers are applied as shown in Table 8 below.

TABLE 8

Different general classifiers are applied to different pictures				
POC	Classifier	C0 band_num	C2 band_num	Total classes
0	combine C0, C1 and C2	4	4	4*17*4
1	combine C0, C1' and C2	6	4	6*9*4

[0151] In some embodiments, plural classifiers are used in the same POC. The current frame is divided by several regions, and each region uses the same classifier. For example, 3 different classifiers are used in POCO, and which classifier (0, 1, or 2) is used is signaled in CTU level as shown in Table 9 below.

TABLE 9

Different general classifiers are applied to different regions in the same picture			
POC	Classifier	C0 band_num	Region
0	C0 using Y0 position	16	0
0	C0 using Y0 position	8	1
0	C0 using Y1 position	8	2

[0152] In some embodiments, the maximum number of plural classifiers (plural classifiers can also be called alternative offset sets) can be fixed or signaled in SPS/APS/PPS/PH/SH/CTU/CU levels. In one example, the fixed (pre-defined) maximum number of plural classifiers is 4. In that case, 4 different classifiers are used in POCO, and which classifier (0, 1, or 2) is used is signaled in the CTU level. Truncated-unary (TU) code can be used to indicate the classifier used for each chroma CTB. For example, as shown in Table 10 below, when TU code is 0: CCSAO is not applied; when TU code is 10: set 0 is applied; when TU code is 110, set 1 is applied; when TU code is 1110: set 2 is

applied; when TU code is 1111: set 3 is applied. Fixed-length code, golomb-rice code, and exponential-golomb code can also be used to indicate the classifier (offset set index) for CTB. 3 different classifiers are used in POC1.

TABLE 10

Truncated-unary (TU) code is used to indicate the classifier used for each chroma CTB				
POC	Classifier	C0 band_num	Region	TU code
0	C0 using Y3 position	6	0	10
0	C0 using Y3 position	7	1	110
0	C0 using Y1 position	3	2	1110
0	C0 using Y6 position	6	3	1111
1	C0 using Y0 position	16	0	10
1	C0 using Y0 position	8	1	110
1	C0 using Y1 position	8	2	1110

[0153] An example of Cb and Cr CTB offset set indices is given for 1280×720 sequence POCO (number of CTUs in a frame is 10×6 if the CTU size is 128×128). POCO Cb uses 4 offset sets and Cr uses 1 offset set. As shown in Table 11 below, when the offset set index is 0: CCSAO is not applied; when the offset set index is 1: set 0 is applied; when the offset set index is 2: set 1 is applied; when the offset set index is 3: set 2 is applied; when the offset set index is 4: set 3 is applied. Type means the position of the chosen collocated luma sample (Yi). Different offset sets can have different types, band_num and corresponding offsets.

TABLE 11

An example of Cb and Cr CTB offset set indices is given for 1280 × 720 sequence POCO (number of CTUs in a frame is 10 × 6 if the CTU size is 128 × 128)									
ccsao_on_frame	POC: 0,	TID: 0 comp: 0, on: 1, lcu_ctrl: 1	set_num: 4 9,	set: 0,	type: 3,	band_num: 6			
ccsao_on_frame	POC: 0,	TID: 0 comp: 0, on: 1, lcu_ctrl: 1	set_num: 4 9,	set: 0,	type: 3,	band_num: 7			
ccsao_on_frame	POC: 0,	TID: 0 comp: 0, on: 1, lcu_ctrl: 1	set_num: 4 9,	set: 0,	type: 3,	band_num: 8			
ccsao_on_frame	POC: 0,	TID: 0 comp: 0, on: 1, lcu_ctrl: 1	set_num: 4 9,	set: 0,	type: 3,	band_num: 9			
ccsao_on_frame	POC: 0,	TID: 0 comp: 0, on: 1, lcu_ctrl: 1	set_num: 4 9,	set: 0,	type: 3,	band_num: 10			
1 0 2 2 0 0 1 2 0 0		1 1 1 1 1 1 1 1 1 1							
0 0 0 0 1 1 1 1 2 4		1 1 1 1 1 1 1 1 1 1							
1 1 4 1 3 1 1 2 2 1		1 1 1 1 1 1 1 1 1 1							
4 3 1 1 4 2 1 1 1 4		1 1 1 1 1 1 1 1 1 1							
0 0 3 1 1 1 1 2 1 3		1 1 1 1 1 1 1 1 1 1							
0 0 3 3 3 1 1 3 4 1		1 1 1 1 1 1 1 1 1 1							
offset [0]	U:	6 0 -1	2	V: -2					
offset [1]	U:	2 1 -7	0	V: 0					
offset [2]	U:	0 -1 -6	-2	V: 0					
offset [3]	U:	-2 2 0	-1	V: 0					
offset [4]	U:	-3 2 0	-1	V: 0					
offset [5]	U:	-4 1 0	-7	V: 1					

TABLE 11-continued

An example of Cb and Cr CTB offset set indices is given for 1280 × 720 sequence POC0 (number of CTUs in a frame is 10 × 6 if the CTU size is 128 × 128)									
offset [6]	U:					V:	0		
offset [7]	U:					V:	0		
offset [8]	U:					V:	0		
offset [9]	U:					V:	−4		
offset [10]	U:					V:			
offset [11]	U:					V:			
offset [12]	U:					V:			
offset [13]	U:					V:			
offset [14]	U:					V:			
offset [15]	U:					V:			

[0154] In some embodiments, the maximum band_num can be fixed or signaled in SPS/APS/PPS/PH/SH/CTU/CU levels. For example, fixing max band_num=16 in the decoder and for each frame, 4 bits are signaled to indicate the CO band_num in a frame. Some other maximum band_num examples are listed below in Table 12.

TABLE 12

Maximum band_num and band_num bit examples		
Band_num_min	Band_num_max	Band_num bit
1	1	0
1	2	1
1	4	2
1	8	3
1	16	4
1	32	5
1	64	6
1	128	7
1	256	8

[0155] In some embodiments, a restriction can be applied on the CO classification, for example, restricting band_num to be only power of 2 values. Instead of explicitly signaling

[0156] Another operation example is taking rounding into account to reduce error.

$$\text{Class}(C0) = ((Y0 + (1 \ll (\text{band_num_shift} - 1))) \gg \text{band_num_shift}) \gg$$

bit_depth

[0157] For example, if band_num_max is 16, the possible band_num_shift candidates are 0, 1, 2, 3, 4, corresponding to band_num = 1, 2, 4, 8, 16, as shown in Table 13.

POC	Classifier	C0 band_num_shift	C0 band_num	Total classes
0	C0 using Y0 position	4	16	16
1	C0 using Y7 position	3	8	8

TABLE 13

Band_num and corresponding band_num_shift candidates		
Band_num_max	Valid band_num	Band_num_shift candidates
1	1	0
2	1, 2	0, 1
4	1, 2, 4	0, 1, 2
8	1, 2, 4, 8	0, 1, 2, 3
16	1, 2, 4, 8, 16	0, 1, 2, 3, 4
32	1, 2, 4, 8, 16, 32	0, 1, 2, 3, 4, 5
64	1, 2, 4, 8, 16, 32, 64	0, 1, 2, 3, 4, 5, 6
128	1, 2, 4, 8, 16, 32, 64, 128	0, 1, 2, 3, 4, 5, 6, 7
256	1, 2, 4, 8, 16, 32, 64, 128, 256	0, 1, 2, 3, 4, 5, 6, 7, 8

band_num, a syntax band_num_shift is signaled. Decoder can use shift operation to avoid multiplication.

$$\text{Class}(C0) = (Y0 \gg \text{band_num_shift}) \gg \text{bit_depth}$$

[0158] In some embodiments, the classifiers applied to Ch and Cr are different. The Ch and Cr offsets for all classes can be signaled separately. For example, different signaled offsets are applied to different chroma components as shown in Table 14 below.

TABLE 14

The Cb and Cr offsets for all classes can be signaled separately					
POC	Component	Classifier	C0	Total classes	Signaled offsets
0	Cb	C0	16	16	16
0	Cr	C0	5	5	5

[0159] In some embodiments, the max offset value is fixed or signaled in Sequence Parameter Set (SPS)/Adaptation parameter set (APS)/Picture parameter set (PPS)/Picture header (PH)/Slice header (SH). For example, the max offset is between $[-15, 15]$.

[0160] In some embodiments, the offset signaling can use Differential pulse-code modulation (DPCM). For example, offsets $\{3, 3, 2, 1, -1\}$ can be signaled as $\{3, 0, -1, -1, -2\}$.

[0161] In some embodiments, the offsets can be stored in APS or a memory buffer for the next picture/slice reuse. An index can be signaled to indicate which stored previous frame offsets are used for the current picture.

[0162] In some embodiments, the classifiers of Cb and Cr are the same. The Cb and Cr offsets for all classes can be signaled jointly, for example, as shown in Table 15 below.

TABLE 15

The Cb and Cr offsets for all classes can be signaled jointly					
POC	Component	Classifier	C0	Total classes	Signaled offsets
0	Cb and Cr	C0	8	8	8

[0163] In some embodiments, the classifier of Cb and Cr can be the same. The Cb and Cr offsets for all classes can be signaled jointly, with a sign flag difference, for example, as shown in Table 16 below. According to Table 16, when Cb offsets are $(3, 3, 2, -1)$, the derived Cr offsets are $(-3, -3, -2, 1)$.

TABLE 16

The Cb and Cr offsets for all classes can be signaled jointly with a sign flag difference					
POC	Component	Classifier	C0 band_num	Total classes	Signaled offsets
0	Cb and Cr	C0	4	4	4: $(3, 3, 2, -1)$

[0164] In some embodiments, the sign flag can be signaled for each class. For example, as shown in Table 17 below. According to Table 17, when Cb offsets are $(3, 3, 2, -1)$, the derived Cr offsets are $(-3, 3, 2, 1)$ according to the respective sign flag.

TABLE 17

The Cb and Cr offsets for all classes can be signaled jointly with a sign flag signaled for each class					
POC	Component	Classifier	C0 band_num	Total classes	Signaled offsets
0	Cb and Cr	C0	4	4	4: $(3, 3, 2, -1)$

[0165] In some embodiments, the classifiers of Cb and Cr can be the same. The Cb and Cr offsets for all classes can be signaled jointly, with a weight difference, for example, as shown in Table 18 below. The weight (w) can be selected in a limited table, for example, $+-1/4, +-1/2, 0, +-1, +-2, +-4, \dots$ etc., where $|w|$ only includes the power-of-2 values. According to Table 18, when Cb offsets are $(3, 3, 2, -1)$, the derived Cr offsets are $(-6, -6, -4, 2)$ according to the respective signed flag.

TABLE 18

The Cb and Cr offsets for all classes can be signaled jointly with a weight difference					
POC	Component	Classifier	C0 band_num	Total classes	Signaled offsets
0	Cb and Cr	C0	4	4	4: $(3, 3, 2, -1)$

[0166] In some embodiments, the weight can be signaled for each class. For example, as shown in Table 19 below. According to Table 19, when Cb offsets are $(3, 3, 2, -1)$, the derived Cr offsets are $(-6, 12, 0, -1)$ according to the respective signed flag.

TABLE 19

The Cb and Cr offsets for all classes can be signaled jointly with a weight signaled for each class					
POC	Component	Classifier	C0 band_num	Total classes	Signaled offsets
0	Cb and Cr	C0	4	4	4: $(3, 3, 2, -1)$

[0167] In some embodiments, if plural classifiers are used in the same POC, different offset sets are signaled separately or jointly.

[0168] In some embodiments, the previously decoded offsets can be stored for use of future frames. An index can be signaled to indicate which previously decoded offsets set is used for the current frame, to reduce offsets signaling overhead. For example, POC0 offsets can be reused by POC2 with signaling offsets set idx=0 as shown in Table 20 below.

TABLE 20

An index can be signaled to indicate which previously decoded offsets set is used for the current frame					
POC	Component	Classifier	C0 band_num	Total classes	Signaled offsets
0	Cb	C0	4	4	4: $(3, 3, 2, -1)$
0	Cr	C0	4	4	4: $(-2, 1, 0, 1)$
1	Cb	C0	4	4	4: $(0, 0, 1, -1)$
1	Cr	C0	4	4	4: $(1, 2, 0, 1)$
2	Cb	C0	4	4	Reuse offsets $(3, 3, 2, -1)$
2	Cr	C0	4	4	Reuse offsets $(-2, 1, 0, 1)$

[0169] In some embodiments, the reuse offsets set idx for Cb and Cr can be different. For example, as shown in Table 21 below.

TABLE 21

An index can be signaled to indicate which previously decoded offsets set is used for the current frame, and the index can be different for Cb and Cr components.						
POC	Com- ponent	Classifier	C0 band_num	Total classes	Signaled offsets	Stored offset set idx
0	Cb	C0	4	4	4: (3, 3, 2, -1)	0
0	Cr	C0	4	4	4: (-2, 1, 0, 1)	0
1	Cb	C0	4	4	4: (0, 0, 1, -1)	1
1	Cr	C0	4	4	4: (1, 2, 0, 1)	1
2	Cb	C0	4	4	Reuse offsets (3, 3, 2, -1)	Signal idx = 0
2	Cr	C0	4	4	Reuse offsets (1, 2, 0, 1)	Signal idx = 1

[0170] In some embodiments, the offset signaling can use additional syntax including start and length, to reduce signaling overhead. For example, when band_num=256, only offsets of band_idx=37-44 are signaled. In the example below in Table 22, the syntax of start and length both are 8 bits fixed-length coded that should match band_num bits.

TABLE 22

the offset signaling uses additional syntax including start and length				
band_idx	offset	band_num_max	band_num bits, start, length	
1	0			
2	0			
3	0			
...				
37	start = 37	offset[0]		
38		offset[1]		
39		offset[2]	1	0
40		offset[3]	2	1
41		offset[4]	4	2
42		offset[5]	8	3
43		offset[6]	16	4
44	length = 8	offset[7]	32	5
...			64	6
255		0	128	7
256		0	256	8

[0171] In some embodiments, if a sequence bit depth is higher than 10 (or a certain bit depth), the offset can be quantized before signaling. On the decoder side, the decoded offset is dequantized before applying it as shown in Table 23 below. For example, for a 12-bit sequence, the decoded offsets are left shifted (dequantized) by 2.

TABLE 23

The decoded offset is dequantized before applying it	
Signaled offset	Dequantized and applied offset
0	0
1	4
2	8
3	12
...	
14	56
15	60

[0172] In some embodiments, the offset can be calculated as $CcSaoOffsetVal = (1 - 2 * ccsao_offset_sign_flag) * (ccsao_offset_abs << (BitDepth - Min(10, BitDepth)))$

[0173] In some embodiments, a sample processing is described below. Let $R(x, y)$ be the input chroma sample value before CCSAO, $R'(x, y)$ be the output chroma sample value after CCSAO:

$$offset = ccsao_offset[class_index \text{ of } R(x, y)]$$

$$R'(x, y) = Clip3(0, (1 << bit_depth) - 1, R(x, y) + offset)$$

[0174] According the above equations, each chroma sample value $R(x, y)$ is classified using the indicated classifier of the current picture. The corresponding offset of the derived class index is added to each chroma sample value $R(x, y)$. A clip function Clip 3 is applied to the $(R(x, y) + offset)$ to make the output chroma sample value $R'(x, y)$ within the bit depth dynamic range, for example, range 0 to $(1 << bit_depth) - 1$.

[0175] In some embodiments, a boundary processing is described below. If any of the collocated and neighboring luma samples used for classification is outside the current

picture, CCSAO is not applied on the current chroma sample. FIG. 13A is a block diagram illustrating CCSAO is not applied on the current chroma sample if any of the collocated and neighboring luma samples used for classification is outside the current picture in accordance with some implementations of the present disclosure. For example, in FIG. 13A (a), if a classifier is used, CCSAO is not applied on the left 1 column chroma components of the current picture. For example, if C1' is used, CCSAO is not applied on the left 1 column and the top 1 row chroma components of the current picture, as shown in FIG. 13A (b).

[0176] FIG. 13B is a block diagram illustrating CCSAO is applied on the current chroma sample if any of the collocated and neighboring luma samples used for classification is outside the current picture in accordance with some implementations of the present disclosure. In some embodiments, a variation is, if any of the collocated and neighboring luma samples used for classification is outside the current picture, the missed samples are used repetitively as shown in FIG. 13B (a), or the missed samples are mirror padded to create samples for classification as shown in FIG. 13B (b), and CCSAO can be applied on the current chroma samples.

[0177] FIG. 14 is a block diagram illustrating CCSAO is not applied on the current chroma sample if a corresponding selected collocated or neighboring luma sample used for classification is outside a virtual space defined by a virtual boundary in accordance with some implementations of the present disclosure. In some embodiments, a virtual boundary (VB) is a virtual line that separates the space within a picture frame. In some embodiments, if a virtual boundary (VB) is applied in the current frame, CCSAO is not applied on the chroma samples that have selected corresponding luma position outside a virtual space defined by the virtual boundary. FIG. 14 shows an example with a virtual boundary for CO classifier with 9 luma position candidates. For each CTU, CCSAO is not applied on the chroma samples for which the corresponding selected luma position is outside a virtual space surrounded by the virtual boundary. For example, in FIG. 14 (a), CCSAO is not applied to the chroma sample 1402 when the selected Y7 luma sample position is on the other side of the horizontal virtual boundary 1406 which is located 4 pixel lines from the bottom side of the frame. For example, in FIG. 14 (b), CCSAO is not applied to the chroma sample 1404 when the selected Y5 luma sample position is located on the other side of the vertical virtual boundary 1408 which is located y pixel lines from the right side of the frame.

[0178] FIG. 15 shows repetitive or mirror padding can be applied on the luma samples that are outside the virtual boundary in accordance with some implementations of the present disclosure. FIG. 15 (a) shows an example of repetitive padding. If the original Y7 is chosen to be the classifier which is located on the bottom side of the VB 1502, the Y4 luma sample value is used for classification (copied to the Y7 position), instead of the original Y7 luma sample value. FIG. 15 (b) shows an example of mirror padding. If Y7 is chosen to be the classifier which is located on the bottom side of the VB 1504, the Y1 luma sample value which is symmetric to the Y7 value relative to the Y0 luma sample is used for classification, instead of the original Y7 luma sample value. The padding methods give more chroma samples possibility to apply CCSAO so more coding gain can be achieved.

[0179] In some embodiments, a restriction can be applied to reduce the CCSAO required line buffer, and to simplify the boundary processing condition check. FIG. 16 shows additional 1 luma line buffer, i.e., the whole line luma samples of line -5 above the current VB 1602, may be required if all 9 collocated neighboring luma samples are used for classification in accordance with some implementations of the present disclosure. FIG. 10B (a) shows an example using only 6 luma candidates for classification, which reduces the line buffer and does not need any additional boundary check in FIG. 13A and FIG. 13B.

[0180] In some embodiments, using luma samples for CCSAO classification may increase the luma line buffer and hence increase the decoder hardware implementation cost. FIG. 17 shows an illustration in AVS that 9 luma candidates CCSAO crossing VB 1702 may increase 2 additional luma line buffers in accordance with some implementations of the present disclosure. For luma and chroma samples above Virtual Boundary (VB) 1702, DBF/SAO/ALF are processed at the current CTU row. For luma and chroma samples below VB 1702, DBF/SAO/ALF are processed at the next CTU row. In AVS decoder hardware design, luma line -4 to -1 pre DBF samples, line -5 pre SAO samples, and chroma

line -3 to -1 pre DBF samples, line -4 pre SAO samples are stored as line buffers for next CTU row DBF/SAO/ALF processing. When processing the next CTU row, luma and chroma samples not in the line buffer are not available. However, for example, at chroma line -3 (b) position, the chroma sample is processed at the next CTU row, but CCSAO requires pre SAO luma sample lines -7, -6, and -5 for classification. Pre SAO luma sample lines -7, -6 are not in the line buffer so they are not available. And adding pre SAO luma samples line -7 and -6 to the line buffer will increase the decoder hardware implementation cost. In some examples, luma VB (line -4) and chroma VB (line-3) can be different (not aligned).

[0181] Similar as FIG. 17, FIG. 18 shows an illustration in VVC that 9 luma candidates CCSAO crossing VB 1802 may increase 1 additional luma line buffer in accordance with some implementations of the present disclosure. VB can be different in different standard. In VVC, luma VB is line -4 and chroma VB is line -2, so 9 candidate CCSAO may increase 1 luma line buffer.

[0182] In some embodiments, in a first solution, CCSAO is disabled for a chroma sample if any of the chroma sample's luma candidates is across VB (outside the current chroma sample VB). FIGS. 19A-19C show in AVS and VVC, CCSAO is disabled for a chroma sample if any of the chroma sample's luma candidates is across VB 1902 (outside the current chroma sample VB) in accordance with some implementations of the present disclosure. FIG. 14 also shows some examples of this implementation.

[0183] In some embodiments, in a second solution, repetitive padding is used for CCSAO from a luma line close to and on the other side of the VB, for example, the luma line -4, for "cross VB" luma candidates. FIGS. 20A-20C show in AVS and VVC, CCSAO is enabled using repetitive padding for a chroma sample if any of the chroma sample's luma candidates is across VB 2002 (outside the current chroma sample VB) in accordance with some implementations of the present disclosure. FIG. 14 (a) also shows some examples of this implementation.

[0184] In some embodiments, in a third solution, mirror padding is used for CCSAO from below luma VB for "cross VB" luma candidates. FIGS. 21A-21C show in AVS and VVC, CCSAO is enabled using mirror padding for a chroma sample if any of the chroma sample's luma candidates is across VB 2102 (outside the current chroma sample VB) in accordance with some implementations of the present disclosure. FIGS. 14 (b) and 13B (b) also show some examples of this implementation.

[0185] The padding methods give more chroma samples possibility to apply CCSAO so more coding gain can be achieved.

[0186] In some embodiments, at the bottom picture (or slice, tile, brick) boundary CTU row, the samples below VB are processed at the current CTU row, so the above special handling (Solution 1, 2, 3) is not applied at the bottom picture (or slice, tile, brick) boundary CTU row. For example, a frame of 1920×1080 is divided by CTUs of 128×128. A frame contains 15×9 CTUs (round up). The bottom CTU row is the 15th CTU row. The decoding process is CTU row by CTU row, and CTU by CTU for each CTU row. Deblocking needs to be applied along horizontal CTU boundaries between the current and next CTU row. CTB VB is applied for each CTU row since inside one CTU, at the bottom 4/2 luma/chroma line, DBF samples (VVC case) are

processed at the next CTU row and are not available for CCSAO at the current CTU row. However, at the bottom CTU row of the picture frame, the bottom 4/2 luma/chroma line DBF samples are available at the current CTU row since there is no next CTU row left and they are DBF processed at the current CTU row.

[0187] In some embodiments, a restriction can be applied to reduce the CCSAO required line buffer, and to simplify boundary processing condition check as explained in FIG. 16. FIG. 22 shows the restrictions of using a limited number of luma candidates for classification in accordance with some implementations of the present disclosure. FIG. 22 (a) shows the restriction of using only 6 luma candidates for classification. FIG. 22 (b) shows the restriction of using only 4 luma candidates for classification.

[0188] In some embodiments, CCSAO syntax implemented is shown in Table 24 below. In AVS3, the term patch is similar with slice, and patch header is similar with the slice header. FLC stands for fixed length code. TU stands for truncated unary code. EGk stands for exponential-golomb code with order k, where k can be fixed.

TABLE 24

A exemplary CCSAO syntax			
Level	Syntax element	Binarization	Meaning
SPS	cc_sao_enabled_flag	FLC	whether CCSAO is enabled in the sequence
PH/SH	ph_cc_sao_cb_flag ph_cc_sao_cr_flag	FLC	whether CCSAO is enabled in this picture/slice for Cb/Cr
PH/SH	ph_cc_sao_stored_offsets_set_idx	FLC	which previously decoded offsets set is used
PH/SH	ph_cc_sao_cb_ctb_control_flag ph_cc_sao_cr_ctb_control_flag	FLC	whether to enable Cb/Cr on/off control at CTB level
SPS/APS/ PPS/PH/ SH/CTU	ph_cc_sao_cb_band_num_minus1 ph_cc_sao_cr_band_num_minus1	FLC	adaptively changed band numbers for classification
SPS/APS/ PPS/PH/ SH/CTU	ph_cc_sao_cb_type ph_cc_sao_cr_type	FLC	Indicating classifier type 1. CCSAO C1 classifier luma type (C1 or C1') 2. CCSAO C0 classifier luma position (Y0, Y1 . . . etc.) 3. CCSAO weighted CO classifier (Yp, Yn . . .)
SPS/APS/ PPS/PH/ SH/CTU	cc_sao_cb_offset_sign_flag cc_sao_cb_offset_abs cc_sao_cr_offset_sign_flag cc_sao_cr_offset_abs	FLC TU or EGk FLC TU or EGk	CCSAO Cb and Cr offset values of each class
CTU	ctb_cc_sao_cb_flag ctb_cc_sao_cr_flag	CABAC, 1 or 2 (up & left) contexts	whether CCSAO is enabled for the current Cb or Cr CTB
CTU	ctb_cc_sao_cb_set_idx ctb_cc_sao_cr_set_idx	TU or EGk	which CCSAO offset set is used for the current Cb or Cr CTB (if CCSAO is enabled)
CTU	cc_sao_cb_merge_left_flag cc_sao_cb_merge_up_flag cc_sao_cr_merge_left_flag cc_sao_cr_merge_up_flag	CABAC	whether CCSAO offset is merged from the left or up CTU

[0189] If a higher-level flag is off, the lower level flags can be inferred from the off state of the flag and do not need to be signaled. For example, if ph_cc_sao_cb_flag is false in this picture, ph_cc_sao_cb_band_num_minus1, ph_cc_sao_cb_luma_type, cc_sao_cb_offset_sign_flag, cc_sao_cb_offset_abs, ctb_cc_sao_cb_flag, cc_sao_cb_merge_left_flag, and cc_sao_cb_merge_up_flag are not present and inferred to be false.

[0190] In some embodiments, the SPS ccsao_enabled_flag is conditioned on the SPS SAO enabled flag as show in Table 25 below.

TABLE 25

the SPS ccsao_enabled_flag is conditioned on the SPS SAO enabled flag	
sps_sao_enabled_flag	u(1)
if(sps_sao_enabled_flag && ChromaArrayType != 0)	
sps_ccsao_enabled_flag	u(1)
sps_alf_enabled_flag	u(1)
if(sps_alf_enabled_flag && ChromaArrayType != 0)	
sps_ccalf_enabled_flag	u(1)

[0191] In some embodiments, ph_cc_sao_cb_ctb_control_flag, ph_cc_sao_cr_ctb_control_flag indicate whether to enable Cb/Cr CTB on/off control granularity. If ph_cc_sao_cb_ctb_control_flag, ph_cc_sao_cr_ctb_control_flag are enabled, ctb_cc_sao_cb_flag and ctb_cc_sao_cr_flag can be further signaled. Otherwise, whether CCSAO is

applied in the current picture depends on ph_cc_sao_cb_flag, ph_cc_sao_cr_flag, without further signaling ctb_cc_sao_cb_flag and ctb_cc_sao_cr_flag at CTB level.

[0192] In some embodiments, for ph_cc_sao_cb_type and ph_cc_sao_cr_type, a flag can be further signaled to distinguish if the center collocated luma position is used (Y0 position in FIG. 10) for classification for a chroma sample,

to reduce bit overhead. Similarly, if `cc_sao_cb_type` and `cc_sao_cr_type` are signaled in CTB level, a flag can be further signaled with the same mechanism. For example, if the number of the C0 luma position candidates is 9, `cc_sao_cb_type0_flag` is further signaled to distinguish if the center

U/V uses the current U/V samples for classification as shown in Table 27 below. FIG. 24 is a block diagram illustrating that for post prediction SAO filter, each component can use the current and neighboring samples for classification in accordance with some implementations of the present disclosure.

TABLE 27

A corresponding classifier is chosen for each Y, U, and V component				
POC	Component	Classifier	C0 band_num	Offset derived from the current classes component
0	Y	combine C0 and C1	16	16*17 $h_Y[i]$
0	U	C0 using U0 position	8	8 $h_U[i]$
0	V	C0 using V0 position	32	32 $h_V[i]$

collocated luma position is used as shown in Table 26 below. If the center collocated luma position is not used, `cc_sao_cb_type_idc` is used to indicate which of the remaining 8 neighboring luma positions is used.

TABLE 26

cc_sao_cb_type0_flag is signaled to distinguish if the center collocated luma position is used	
<code>ctb_cc_sao_cb_flag</code>	<code>u(1)</code>
<code>if(ctb_cc_sao_cb_flag)</code>	
<code>cc_sao_cb_type0_flag</code>	<code>u(1)</code> , can be context coded
<code>if(!cc_sao_cb_type0_flag)</code>	
<code>cc_sao_cb_type_idc</code>	<code>u(3)</code> , can be context coded

[0195] In some embodiments, the refined prediction samples ($Y_{pred'}$, $U_{pred'}$, $V_{pred'}$) are updated by adding the corresponding class offset and are used for intra, inter, or other prediction thereafter.

$$Y_{pred'} = clip3(0, (1 << bit_depth) - 1, Y_{pred} + h_Y[i])$$

$$U_{pred'} = clip3(0, (1 << bit_depth) - 1, U_{pred} + h_U[i])$$

$$V_{pred'} = clip3(0, (1 << bit_depth) - 1, V_{pred} + h_V[i])$$

[0196] In some embodiments, for chroma U and V components, besides the current chroma component, the cross-component (Y) can be used for further offset classification. The additional cross-component offset (h'_U , h'_V) can be added on the current component offset (h_U , h_V), for example, as shown in Table 28 below.

TABLE 28

For chroma U and V components, besides the current chroma component, the cross-component (Y) can be used for further offset classification				
POC	Component	Classifier	C0 band_num	Offset derived from the current classes component
0	U	C0 using Y4 position	16	16 $h'_U[i]$
0	V	C0 using Y1 position	7	7 $h'_V[i]$

[0193] In some embodiments, an extension to the intra and inter post prediction SAO filter is illustrated further below. In some embodiments, the SAO classification methods disclosed in the present disclosure can serve as a post prediction filter, and the prediction can be intra, inter, or other prediction tools such as Intra Block Copy. FIG. 23 is a block diagram illustrating that the SAO classification methods disclosed in the present disclosure serve as a post prediction filter in accordance with some implementations of the present disclosure.

[0194] In some embodiments, for each Y, U, and V component, a corresponding classifier is chosen. And for each component prediction sample, it is first classified, and a corresponding offset is added. For example, each component can use the current and neighboring samples for classification. Y uses the current Y and neighboring Y samples, and

[0197] In some embodiments, the refined prediction samples ($U_{pred''}$, $V_{pred''}$) are updated by adding the corresponding class offset and are used for intra, inter, or other prediction thereafter.

$$U_{pred''} = clip3(0, (1 << bit_depth) - 1, U_{pred'} + h'_U[i])$$

$$V_{pred''} = clip3(0, (1 << bit_depth) - 1, V_{pred'} + h'_V[i])$$

[0198] In some embodiments, the intra and inter prediction can use different SAO filter offsets.

[0199] FIG. 25 is a flowchart illustrating an exemplary process 2500 of decoding video signal using cross-compo-

nent correlation when VB is present in accordance with some implementations of the present disclosure.

[0200] The video decoder 30 (as shown in FIG. 3), receives, from the video signal, a picture frame that includes a first component and a second component (2510).

[0201] The video decoder 30, determines a classifier for the second component from a set of one or more samples of the first component associated with a respective sample of the second component (2520).

[0202] The video decoder 30, determines whether to modify a value of the respective sample of the second component of a current block of the picture frame within a virtual boundary according to the classifier (2530).

[0203] The video decoder 30, in response to the determination to modify the value of the respective sample of the second component of the current block of the picture frame within the virtual boundary according to the classifier, determines a sample offset for the respective sample of the second component according to the classifier (2540).

[0204] The video decoder 30, modifies the value of the respective sample of the second component based on the determined sample offset (2550).

[0205] In some embodiments, the virtual boundary includes a first virtual boundary within a first block of the first component and a second virtual boundary within a second block of the second component.

[0206] In some embodiments, the virtual boundary includes a first virtual boundary within a first Coding Tree Block (CTB) of the first component and a second virtual boundary within a second CTB of the second component, and the first virtual boundary is not aligned with the second virtual boundary. In some examples, the first CTB and the second CTB are within a CTU.

[0207] In some embodiments, the virtual boundary is parallel with a block boundary and separated from the block boundary by at least one row or column of samples.

[0208] In some embodiments, determining whether to modify the value of the respective sample of the second component of the current block of the picture frame within the virtual boundary according to the classifier (2530) comprises: in accordance with a determination that the set of one or more samples of the first component associated with the respective sample of the second component is divided by the first virtual boundary, determining not to modify the value of the respective sample of the second component of the current block of the picture frame within the virtual boundary according to the classifier. For example, CCSAO is disabled for “cross VB” chroma samples.

[0209] In some embodiments, determining whether to modify the value of the respective sample of the second component of the current block of the picture frame within the virtual boundary according to the classifier (2530) comprises: in accordance with a determination that the set of one or more samples of the first component associated with the respective sample of the second component is located on a different side of the virtual boundary relative to the respective sample of the second component, determining not to modify the value of the respective sample of the second component of the current block of the picture frame within the virtual boundary according to the classifier. For example, CCSAO is disabled for “cross VB” chroma samples.

[0210] In some embodiments, determining whether to modify the value of the respective sample of the second component of the current block of the picture frame within

the virtual boundary according to the classifier (2530) comprises: in accordance with a determination that: a first subset of the set of one or more samples of the first component associated with the respective sample of the second component is located on a different side of the virtual boundary relative to the respective sample of the second component, and a remaining subset of the set of one or more samples of the first component associated with the respective sample of the second component is located on a same side of the virtual boundary relative to the respective sample of the second component, copying a second subset from the remaining subset of the set of one or more samples of the first component to replace the first subset; and determining to modify the value of the respective sample of the second component of the current block of the picture frame within the virtual boundary according to the classifier. For example, repetitive padding and/or mirror padding are used from luma line when the VB is cross the luma samples.

[0211] In some embodiments, the second subset from the remaining subset of the set of one or more samples of the first component is from a closest row to the first subset. For example, repetitive padding is used when the VB is cross the luma samples.

[0212] In some embodiments, the second subset from the remaining subset of the set of one or more samples of the first component is located in a symmetric location of the first virtual boundary or the respective sample of the second component to the first subset. For example, mirror padding is used when the VB is cross the luma samples.

[0213] In some embodiments, determining whether to modify the value of the respective sample of the second component of the current block of the picture frame within the virtual boundary according to the classifier (2530) comprises: in accordance with a determination that the set of one or more samples of the first component associated with the respective sample of the second component is located on a different side of the virtual boundary relative to the respective sample of the second component, copying a second set of samples of the first component on a same side of the virtual boundary relative to the respective sample of the second component to replace the set of one or more samples of the first component; and determining to modify the value of the respective sample of the second component of the current block of the picture frame within the virtual boundary according to the classifier. For example, repetitive padding and/or mirror padding is used from luma line when the VB is below the luma samples.

[0214] In some embodiments, the second set of samples on the same side of the virtual boundary relative to the respective sample of the second component is from a closest row on the same side of the virtual boundary relative to the respective sample of the second component to the set of one or more samples of the first component. For example, repetitive padding is used from luma line when the VB is below the luma samples.

[0215] In some embodiments, the second set of samples on the same side of the virtual boundary relative to the respective sample of the second component is located in a symmetric location of the first virtual boundary or the respective sample of the second component to the set of one or more samples of the first component. For example, mirror padding is used from luma line when the VB is below the luma samples.

[0216] In some embodiments, determining whether to modify the value of the respective sample of the second component of the current block of the picture frame within the virtual boundary according to the classifier (2530) comprises: in accordance with a determination that the set of one or more samples of the first component associated with the respective sample of the second component is divided by the virtual boundary, a subset of the one or more samples of the first component is located on a different side of the virtual boundary relative to the respective sample of the second component, and the subset of the one or more samples of the first component is within a boundary coding tree unit (CTU) row at a processing bottom of the picture frame, determining to modify the value of the respective sample of the second component of the current block of the picture frame within the virtual boundary according to the classifier. For example, at the bottom picture (or slice, tile, brick) boundary CTU row, the samples below VB are processed at the current CTU row, so the special handling (Solution 1, 2, 3) is not applied at the bottom picture (or slice, tile, brick) boundary CTU row.

[0217] Further embodiments also include various subsets of the above embodiments combined or otherwise re-arranged in various other embodiments.

[0218] In one or more examples, the functions described may be implemented in hardware, software, firmware, or any combination thereof. If implemented in software, the functions may be stored on or transmitted over, as one or more instructions or code, a computer-readable medium and executed by a hardware-based processing unit. Computer-readable media may include computer-readable storage media, which corresponds to a tangible medium such as data storage media, or communication media including any medium that facilitates transfer of a computer program from one place to another, e.g., according to a communication protocol. In this manner, computer-readable media generally may correspond to (1) tangible computer-readable storage media which is non-transitory or (2) a communication medium such as a signal or carrier wave. Data storage media may be any available media that can be accessed by one or more computers or one or more processors to retrieve instructions, code and/or data structures for implementation of the implementations described in the present application. A computer program product may include a computer-readable medium.

[0219] The terminology used in the description of the implementations herein is for the purpose of describing particular implementations only and is not intended to limit the scope of claims. As used in the description of the implementations and the appended claims, the singular forms “a,” “an,” and “the” are intended to include the plural forms as well, unless the context clearly indicates otherwise. It will also be understood that the term “and/or” as used herein refers to and encompasses any and all possible combinations of one or more of the associated listed items. It will be further understood that the terms “comprises” and/or “comprising,” when used in this specification, specify the presence of stated features, elements, and/or components, but do not preclude the presence or addition of one or more other features, elements, components, and/or groups thereof.

[0220] It will also be understood that, although the terms first, second, etc. may be used herein to describe various elements, these elements should not be limited by these

terms. These terms are only used to distinguish one element from another. For example, a first electrode could be termed a second electrode, and, similarly, a second electrode could be termed a first electrode, without departing from the scope of the implementations. The first electrode and the second electrode are both electrodes, but they are not the same electrode.

[0221] Reference throughout this specification to “one example,” “an example,” “exemplary example,” or the like in the singular or plural means that one or more particular features, structures, or characteristics described in connection with an example is included in at least one example of the present disclosure. Thus, the appearances of the phrases “in one example” or “in an example,” “in an exemplary example,” or the like in the singular or plural in various places throughout this specification are not necessarily all referring to the same example. Furthermore, the particular features, structures, or characteristics in one or more examples may include combined in any suitable manner.

[0222] The description of the present application has been presented for purposes of illustration and description, and is not intended to be exhaustive or limited to the invention in the form disclosed. Many modifications, variations, and alternative implementations will be apparent to those of ordinary skill in the art having the benefit of the teachings presented in the foregoing descriptions and the associated drawings. The embodiment was chosen and described in order to best explain the principles of the invention, the practical application, and to enable others skilled in the art to understand the invention for various implementations and to best utilize the underlying principles and various implementations with various modifications as are suited to the particular use contemplated. Therefore, it is to be understood that the scope of claims is not to be limited to the specific examples of the implementations disclosed and that modifications and other implementations are intended to be included within the scope of the appended claims.

What is claimed is:

1. A method of video coding, comprising:

in response to a current Coding Tree Unit (CTU) being not located within a bottom row of CTUs of a frame, slice, tile, subpicture or patch, determining a set of one or more samples of a first component of a picture frame according to a positional relationship between a respective sample of a second component of the picture frame and a virtual boundary;

determining a class index for the second component from the set of one or more samples of the first component associated with the respective sample of the second component;

selecting a sample offset from a plurality of sample offsets for the respective sample of the second component according to the class index; and

obtaining a cross-component offsetted sample value of the second component based on the selected sample offset.

2. The method of claim 1, further comprising:

in response to the current CTU being located within the bottom row of CTUs of the frame, slice, tile, subpicture or patch, determining the set of one or more samples of the first component regardless of the virtual boundary, wherein the set of one or more samples comprises a collocated sample of the first component relative to the respective sample of the second component or a neigh-

boring sample of the collocated sample of the first component relative to the respective sample of the second component.

3. The method of claim 1, wherein determining the set of one or more samples of the first component according to the positional relationship between the respective sample of the second component and the virtual boundary comprises:

determining a first set of one or more samples of the first component as the set of one or more samples of the first component when the respective sample of the second component is above the virtual boundary; and

determining a second set of one or more samples of the first component as the set of one or more samples of the first component when the respective sample of the second component is on or below the virtual boundary, wherein the second set of one or more samples is different from the first set of one or more samples.

4. The method of claim 3, wherein the first set of one or more samples comprises a collocated sample of the first component relative to the respective sample of the second component or a neighboring sample of the collocated sample of the first component relative to the respective sample of the second component.

5. The method of claim 3, wherein the virtual boundary comprises a virtual boundary within a Coding Tree Block (CTB) of the second component.

6. The method of claim 3, wherein the virtual boundary is parallel with a block boundary and separated from the block boundary by at least one row or column of samples.

7. An electronic apparatus comprising:

one or more processing units;

memory coupled to the one or more processing units; and a plurality of programs stored in the memory that, when executed by the one or more

processing units, cause the electronic apparatus to perform a method of video coding, the method comprises:

in response to a current Coding Tree Unit (CTU) being not located within a bottom row of CTUs of a frame, slice, tile, subpicture or patch, determining a set of one or more samples of a first component of a picture frame according to a positional relationship between a respective sample of a second component of the picture frame and a virtual boundary;

determining a class index for the second component from the set of one or more samples of the first component associated with the respective sample of the second component;

selecting a sample offset from a plurality of sample offsets for the respective sample of the second component according to the class index; and

obtaining a cross-component offsetted sample value of the second component based on the selected sample offset.

8. The electronic apparatus of claim 7, the method further comprising:

in response to the current CTU being located within the bottom row of CTUs of the frame, slice, tile, subpicture or patch, determining the set of one or more samples of the first component regardless of the virtual boundary, wherein the set of one or more samples comprises a collocated sample of the first component relative to the respective sample of the second component or a neighboring sample of the collocated sample of the first component relative to the respective sample of the second component.

9. The electronic apparatus of claim 7, wherein determining the set of one or more samples of the first component according to the positional relationship between the respective sample of the second component and the virtual boundary comprises:

determining a first set of one or more samples of the first component as the set of one or more samples of the first component when the respective sample of the second component is above the virtual boundary; and

determining a second set of one or more samples of the first component as the set of one or more samples of the first component when the respective sample of the second component is on or below the virtual boundary, wherein the second set of one or more samples is different from the first set of one or more samples.

10. The electronic apparatus of claim 9, wherein the first set of one or more samples comprises a collocated sample of the first component relative to the respective sample of the second component or a neighboring sample of the collocated sample of the first component relative to the respective sample of the second component.

11. The electronic apparatus of claim 9, wherein the virtual boundary comprises a virtual boundary within a Coding Tree Block (CTB) of the second component.

12. The electronic apparatus of claim 9, wherein the virtual boundary is parallel with a block boundary and separated from the block boundary by at least one row or column of samples.

13. A non-transitory computer readable storage medium storing a bitstream generated by a method of video coding, the method comprising:

in response to a current Coding Tree Unit (CTU) being not located within a bottom row of CTUs of a frame, slice, tile, subpicture or patch, determining a set of one or more samples of a first component of a picture frame according to a positional relationship between a respective sample of a second component of the picture frame and a virtual boundary;

determining a class index for the second component from the set of one or more samples of the first component associated with the respective sample of the second component;

selecting a sample offset from a plurality of sample offsets for the respective sample of the second component according to the class index; and

obtaining a cross-component offsetted sample value of the second component based on the selected sample offset.

14. The storage medium of claim 13, the method further comprising:

in response to the current CTU being located within the bottom row of CTUs of the frame, slice, tile, subpicture or patch, determining the set of one or more samples of the first component regardless of the virtual boundary, wherein the set of one or more samples comprises a collocated sample of the first component relative to the respective sample of the second component or a neighboring sample of the collocated sample of the first component relative to the respective sample of the second component.

15. The storage medium of claim 13, wherein determining the set of one or more samples of the first component according to the positional relationship between the respective sample of the second component and the virtual boundary comprises:

determining a first set of one or more samples of the first component as the set of one or more samples of the first component when the respective sample of the second component is above the virtual boundary; and
determining a second set of one or more samples of the first component as the set of one or more samples of the first component when the respective sample of the second component is on or below the virtual boundary, wherein the second set of one or more samples is different from the first set of one or more samples.

16. The storage medium of claim **15**, wherein the first set of one or more samples comprises a collocated sample of the first component relative to the respective sample of the second component or a neighboring sample of the collocated sample of the first component relative to the respective sample of the second component.

17. The storage medium of claim **15**, wherein the virtual boundary comprises a virtual boundary within a Coding Tree Block (CTB) of the second component.

18. The storage medium of claim **15**, wherein the virtual boundary is parallel with a block boundary and separated from the block boundary by at least one row or column of samples.

19. A method for storing a bitstream, comprising:
performing the method according to claim **1** to generate a bitstream; and
storing the bitstream.

* * * * *